



SCHOOL OF COMPUTATION, INFORMATION
AND TECHNOLOGY - INFORMATICS

TECHNICAL UNIVERSITY OF MUNICH

Master's Thesis in Data Engineering and Analytics

Exploring Pedagogical Alignment of LLMs Using Student Errors and Tutor Moves

Jaeyeop Chung





SCHOOL OF COMPUTATION, INFORMATION
AND TECHNOLOGY - INFORMATICS

TECHNICAL UNIVERSITY OF MUNICH

Master's Thesis in Data Engineering and Analytics

Exploring Pedagogical Alignment of LLMs Using Student Errors and Tutor Moves

Untersuchung des pädagogischen Alignments von LLMs anhand von Schülerfehlern und Tutor Moves

Author:	Jaeyeop Chung
Supervisor:	Prof. Dr. Enkelejda Kasneci
Advisor:	Isabel Schorr, MSc
Submission Date:	25.09.2026



I confirm that this master's thesis in the MSc in Data Engineering and Analytics is my own work and that I have documented all sources and materials used.

Berlin, 25.09.2026

Location, Submission Date

Jaeyeop Chung 장재엽

Author

AI Assistant Usage Disclosure

Use of AI Assistants for Research Purposes

Explanation: I used AI assistants, including Cursor, ChatGPT, and Perplexity to support the writing of this thesis in the following areas: language and style enhancement, clarity and conciseness, and debugging and code-style enhancement, LaTeX formatting, and initial literature search. I reviewed and revised all AI-assisted text before including it in this thesis. The research design, analysis, interpretation of results, and the scientific content remain my own responsibility.

By signing below, I confirm that I have reported all use of AI assistants in my research and that this disclosure is truthful and complete.

Berlin, 25.09.2026

Location, Date

Jaeyeop Chung 정재엽

Author

Acknowledgments

First and foremost, I would like to express my sincere gratitude to my supervisor, Isabel Schorr, for giving me the opportunity to pursue my own research interests through this thesis. Her calm assurance, thoughtful guidance, and warm support have shaped this work and given me confidence throughout the process.

I am deeply grateful to my girlfriend, Afrida, for always being by my side throughout this journey. Her patience and encouragement have been a constant source of strength. In moments of doubt, her belief in me helped me find the confidence to keep going.

To my family, I owe my heartfelt appreciation for their love, encouragement, and unconditional support. I would not be where I am today without them, and this milestone belongs as much to them as it does to me.

I extend my sincere thanks to everyone around me and to all those whose paths have crossed mine. Whether through lasting companionship or a brief moment of kindness, each of you has, in your own way, been part of the journey that brought me here.

Abstract

Pedagogical alignment of large language models (LLMs) requires attention not only to the quality of tutor responses but also to the difficulties indicated by student errors. This thesis investigates empirical associations between student error types and tutor moves by combining the MathDial and StepVerify datasets. RoBERTa-base classifiers are used to estimate tutor-move posteriors. These posteriors are combined with positive pointwise mutual information (PPMI) derived from observed student error–tutor move co-occurrences to score candidate tutor responses.

Llama-3-8B-Instruct serves as the base model and is first adapted through supervised fine-tuning (SFT) on human-authored tutor responses from a subset of MathDial dialogues and then trained through Direct Preference Optimization (DPO). Separate DPO models use the top 10%, 30%, and 50% of preference pairs ranked by score margin within each student error type. The base, SFT, and DPO models are evaluated using gpt-4.1-mini as an LLM-as-a-Judge, following an eight-dimensional pedagogical taxonomy. Pedagogical quality is measured through the Desired Annotation Match Rate (DAMR). The analyses examine how the observed associations are reflected in the preference dataset, how changes in pedagogical quality vary across student error types, and whether the resulting behavioral changes correspond to the tutor-move preferences encoded for those error types.

Kurzfassung

Das pädagogische Alignment großer Sprachmodelle (Large Language Models, LLMs) erfordert die Berücksichtigung sowohl der Qualität von Tutorantworten als auch der durch Schülerfehler angezeigten Lernschwierigkeiten. Diese Arbeit untersucht empirische Zusammenhänge zwischen Schülerfehlertypen und Tutor Moves, indem die Datensätze MathDial und StepVerify kombiniert werden. RoBERTa-base-Klassifikatoren werden verwendet, um Posterior-Wahrscheinlichkeiten für Tutor Moves zu schätzen. Diese Posterior-Wahrscheinlichkeiten werden mit der Positive Pointwise Mutual Information (PPMI) kombiniert, die aus beobachteten Ko-Okkurrenzen von Schülerfehlertypen und Tutor Moves abgeleitet wird, um potenzielle Tutorantworten zu bewerten.

Llama-3-8B-Instruct dient als Basismodell und wird zunächst durch Supervised Fine-Tuning (SFT) auf von Menschen verfassten Tutorantworten aus einer Teilmenge der MathDial-Dialoge angepasst und anschließend mittels Direct Preference Optimization (DPO) weitertrainiert. Separate DPO-Modelle verwenden jeweils die obersten 10%, 30% und 50% der Präferenzpaare, die innerhalb jedes Schülerfehlertyps nach ihrer Scoredifferenz geordnet werden. Das Basis-, SFT- und die DPO-Modelle werden mithilfe von gpt-4.1-mini als LLM-as-a-Judge anhand einer achtdimensionalen pädagogischen Taxonomie evaluiert. Die pädagogische Qualität wird anhand der Desired Annotation Match Rate (DAMR) gemessen. Die Analysen untersuchen, wie sich die beobachteten Zusammenhänge im Präferenzdatensatz widerspiegeln, wie Veränderungen der pädagogischen Qualität zwischen verschiedenen Schülerfehlertypen variieren und ob die daraus resultierenden Verhaltensänderungen den für diese Fehlertypen kodierten Präferenzen für Tutor Moves entsprechen.

Contents

Acknowledgments	iv
Abstract	v
Kurzfassung	vi
1. Introduction	1
2. Background and Related Work	2
2.1. Learning-Science Foundations	2
2.2. From Learning Theory to Intelligent Tutoring Systems	4
2.3. From Intelligent Tutoring Systems to LLM-Based Tutors	5
2.4. Educational Dialogue Data	7
2.4.1. Settings: Classroom Discourse and Tutoring Dialogue	7
2.4.2. Differences Between the Two Settings	7
2.4.3. Publicly Available Datasets	7
2.5. Dialogue Acts in Educational Dialogue	10
2.5.1. Uses of Dialogue Acts in Educational Dialogue	10
2.5.2. Dialogue Act Classification	11
2.6. Modeling Students and Tutors in Educational Dialogue	11
2.6.1. Modeling the Student	11
2.6.2. Modeling the Tutor	13
2.6.3. Joint Modeling of Student and Tutor	13
2.7. Pedagogical Alignment of Large Language Models	14
2.7.1. Prompting	14
2.7.2. Supervised Fine-Tuning	15
2.7.3. Preference Optimization and Reinforcement Learning	15
2.8. Pedagogical Evaluation	15
2.8.1. Human and Expert Evaluation	16
2.8.2. Rubric-Based and Automated Evaluation	16
2.8.3. Learner-Centred Validation	16
3. Research Gap and Research Questions	17
3.1. Research Gap	17
3.2. Research Questions	18

4. Methodology	19
4.1. Pipeline Overview	19
4.2. Dataset Construction	20
4.2.1. Source Dataset for Annotation Transfer	21
4.2.2. Annotation Transfer Pipeline	23
4.2.3. Aligned Dataset	25
4.3. Tutor-Turn Pedagogy Move Classification for Preference Scoring	27
4.3.1. Task Formulation and Input Representation	28
4.3.2. Base Model and Domain-Adaptive Pretraining	31
4.3.3. Loss Functions and Training Configurations	35
4.3.4. Evaluation and Model Selection	38
4.4. Preference Dataset Construction	40
4.4.1. Candidate Tutor Response Generation	40
4.4.2. Error-Conditioned Tutor-Move Scoring Method	43
4.4.3. Preference Pair Construction	46
4.5. Pedagogical Alignment and Evaluation of LLM Tutors	48
4.5.1. Task Definition and Experimental Setup	48
4.5.2. Supervised-Fine-Tuning	49
4.5.3. Direct Preference Optimization	52
4.5.4. Automated Pedagogical Evaluation	56
5. Results	60
5.1. Tutor-Move Classification Results	60
5.1.1. Comparison of Model Configurations	60
5.1.2. The Selected Classifier	65
5.2. Error-Conditioned Pedagogical Signals in the Constructed Preference Dataset	67
5.2.1. Observed Error-Pedagogy Associations in the Source Dataset	67
5.2.2. Examining Error-Pedagogy Associations in Chosen Responses	68
5.3. Pedagogical Alignment Results	71
5.3.1. Overall Pedagogical Quality	71
5.3.2. Variation across Student Error Types	73
5.3.3. Student Error Types behind the Steering	76
6. Discussion	80
6.1. Findings	80
6.2. Limitations and Future Work	84
7. Conclusion	88
A. Appendix	92
A.1. Ranked Candidates for Each Student Error Type	92
List of Figures	93

Contents

List of Tables	96
Bibliography	98

1. Introduction

Research in the learning sciences suggests that student errors provide diagnostic signals that teachers can use to identify students' difficulties and select appropriate instructional responses. Errors may reflect systematic misconceptions or gaps in prerequisite knowledge rather than random failures (Brown & Burton, 1978; Radatz, 1979). Feedback informed by this diagnosis can help students understand their errors and determine how to proceed. Effective feedback therefore needs to address the task and the reasoning process, while providing guidance that supports improvement (Hattie & Timperley, 2007). The instructional challenge is not simply to recognize an incorrect answer, but to decide what support the student needs next.

Large language models (LLMs) offer opportunities to provide such support through flexible, natural-language tutoring dialogue. However, generating a correct solution is not equivalent to helping a student learn. General-purpose models may reveal answers instead of supporting students' reasoning. This distinction has practical consequences: Bastani et al. (2025) found that access to an unrestricted GPT-4 interface improved students' practice performance but reduced their performance on a subsequent unassisted exam, whereas teacher-designed safeguards largely avoided this harm. These findings suggest that the educational value of an LLM tutor depends on how its behavior is directed toward learning.

This challenge motivates pedagogical alignment, which aims to make LLMs behave as tutors rather than question-answering assistants (Sonkar et al., 2024). A preference signal should capture not only desirable response characteristics but also the student's instructional needs. This raises a question: can the relationship between the type of error a student makes and the tutor's subsequent response provide an empirical basis for constructing such preferences?

This thesis investigates that possibility using student error types and tutor moves by combining complementary annotations from MathDial and StepVerify (Daheim et al., 2024; Macina, Daheim, Pal Chowdhury, et al., 2023). Specifically, this thesis examines (1) whether associations can be observed between student error types and tutor moves, (2) whether these association signals are reflected in the constructed preference dataset, and (3) which student error types show the strongest changes in the pedagogical behavior of the aligned LLM tutors.

 https://github.com/JaeyeopC/Thesis_Repository

2. Background and Related Work

This chapter provides the background for the thesis and reviews the work it builds on. It moves from the general to the specific. Section 2.1 summarizes the learning-science theories that describe how a tutor should respond to a learner, and in particular to a learner’s error. Sections 2.2 and 2.3 show how these theories were operationalized in Intelligent Tutoring Systems (ITS) and how ITS evolved into today’s tutors based on large language models (LLMs). Section 2.4 reviews educational dialogue data, the settings in which it is collected, and the publicly available datasets. Section 2.5 introduces dialogue acts and reviews their uses in educational dialogue and how they are classified, and Section 2.6 reviews how prior work models the student side, the tutor side, and both sides jointly. Section 2.7 reviews how LLMs have been steered towards pedagogically desirable behavior through prompting, supervised fine-tuning, and preference optimization. Section 3.1 summarizes the chapter and identifies the research gap.

2.1. Learning-Science Foundations

Research on tutoring systems rests on a small number of learning theories. This section introduces only those that later matter for tutoring dialogue, and states for each what it implies for the way a tutor should react to a student.

From Behaviorism to Cognitive Theories

Behaviorism views learning as a change in observable behavior that is shaped by reinforcement. Skinner’s teaching machines and programmed instruction translated this view into instructional design: material is split into small steps, the learner responds to each step, and feedback follows immediately (Skinner, 1954). This design is an important intellectual precursor to computer-assisted instruction, and its implication for tutoring is simple: learner errors should be detected and addressed promptly.

Cognitive theories shifted attention from observable behavior to the mental representations underlying it. Anderson’s ACT-R theory describes skill acquisition as the gradual compilation of declarative knowledge into procedural production rules. In model-based tutoring, this allows both a skill and a learner’s deviation from it to be represented at the level of individual rules (Anderson et al., 1995). Cognitive Load Theory further emphasizes that working memory is limited and that instruction should avoid imposing unnecessary processing demands (Sweller, 1988). Together, these perspectives suggest that feedback on an error should focus on the specific step or knowledge component involved rather than restating the entire solution.

Constructivism, the Zone of Proximal Development, and Scaffolding

Constructivist theories hold that learners actively build knowledge rather than receiving it. Vygotsky's social constructivism locates learning in interaction with a more knowledgeable other and defines the Zone of Proximal Development (ZPD) as the distance between what a learner can do alone and what that learner can do with guidance (Vygotsky, 1978). Wood et al. (1976) introduced the term *scaffolding* for the support a tutor gives within this zone: the tutor reduces the degrees of freedom of the task, keeps the learner oriented towards the goal, marks critical features, and withdraws support as the learner's competence grows (Wood et al., 1976). Scaffolding is *contingent*: the amount of help depends on what the learner has just done. The implication for responding to errors is that the tutor should increase support step by step and should not hand over the answer if a smaller intervention can still succeed.

Mastery Learning and the Two-Sigma Problem

Bloom (1984) reported that students who received one-to-one tutoring combined with mastery learning performed substantially better than students in conventional classrooms and framed the search for scalable methods with a comparable effect as the *two-sigma problem* (Bloom, 1984). This result has motivated the development of tutoring systems for four decades. However, later evidence has suggested more moderate effects. VanLehn's review found that both human tutoring and step-based ITS produced substantial benefits relative to no tutoring, but concluded that Bloom's two-sigma result was not confirmed (VanLehn, 2011).

Feedback and Errors as Learning Opportunities

Effective feedback is specific and focuses on the task and process rather than on the learner as a person (Hattie & Timperley, 2007). It also gives learners clear guidance on how to improve or proceed (Hattie & Timperley, 2007). A related tradition treats errors as diagnostic information. Brown and Burton (1978) showed with BUGGY that many wrong answers in arithmetic are not random but follow from systematic bugs in an otherwise correct procedure (Brown & Burton, 1978), and VanLehn's repair theory explained how such bugs arise (VanLehn, 1990). In mathematics education, Radatz (1979) classified errors by their cause, for example language difficulties, deficient prerequisite skills, or the rigid application of a familiar rule (Radatz, 1979). Research on learning from errors suggests that generating errors and receiving corrective feedback can support learning more effectively than attempting to avoid errors altogether (Metcalfe, 2017). Taken together, these literatures imply that the type of error carries information about what help is needed.

Human Tutoring and Classroom Discourse

Empirical studies of human tutors describe what effective responses look like in practice. Graesser et al. (1995) found that naturalistic tutoring follows a recurring five-step frame in which the tutor asks a question, the student answers, the tutor gives short feedback, both collaboratively improve the answer, and the tutor assesses understanding (Graesser et al., 1995).

M. T. H. Chi et al. (2001) showed that learning in tutoring depends on the student's own constructive activity and on the interaction, and not only on the tutor's explanations (M. T. H. Chi et al., 2001); the ICAP framework generalizes this into the claim that interactive and constructive engagement produce more learning than active or passive engagement (M. T. H. Chi & Wylie, 2014).

Classroom research developed its own vocabulary. Sinclair and Coulthard (1975) described the dominant Initiation–Response–Feedback (IRF) exchange, in which a teacher initiates an exchange, a student responds, and the teacher provides feedback. Mehan (1979) described the closely related Initiation–Response–Evaluation (IRE) exchange, in which the teacher's third turn evaluates the student's response rather than extending or building on it (Mehan, 1979; Sinclair & Coulthard, 1975). Work on dialogic instruction argued that learning benefits when teachers ask authentic questions and take up students' ideas (Nystrand et al., 1997). Building on these principles, the Accountable Talk framework specified concrete teacher moves, such as revoicing and pressing for reasoning, that make students' thinking public (Michaels et al., 2008).

2.2. From Learning Theory to Intelligent Tutoring Systems

Intelligent tutoring systems (ITSs) are computer systems that adapt instruction to individual learners. A conventional description distinguishes four components (Woolf, 2009): a domain model that represents the knowledge to be taught, a student model that estimates what the learner knows, a tutoring (or pedagogical) model that selects instructional actions, and a user interface through which the learner and system interact. The theories discussed in Section 2.1 map onto these components.

Cognitive Tutors

A canonical example is the Cognitive Tutor approach, which is grounded in ACT-R and represents procedural knowledge as production rules (Anderson et al., 1995). Its cognitive model encodes correct solution procedures and may also include buggy production rules representing common errors or misconceptions. Model tracing relates each student step to actions that can be generated by the cognitive model, enabling immediate feedback and, when a recognized buggy production matches the step, an error-specific message. The tutor estimates learners' developing knowledge to individualize problem selection and support progression toward mastery. Help is available on demand, with multiple levels providing increasingly detailed assistance as needed.

Constraint-Based Tutors

Constraint-based modeling represents domain knowledge as constraints that every correct solution must satisfy. When a student solution violates a relevant constraint, the tutor diagnoses an error and delivers feedback associated with that constraint, without requiring an explicit model of every valid solution path (Mitrovic & Ohlsson, 1999).

Inner and Outer Loops

VanLehn (2006) described tutoring systems as having two loops. The outer loop selects the next task, whereas the inner loop operates on steps within a task. The inner loop can provide minimal or error-specific feedback and hints, assess the learner's evolving knowledge, and update the student model used by the outer loop to select an appropriate next task. VanLehn (2011) later distinguished tutoring systems by the granularity of their interaction and found that step-based ITSs were substantially more effective than no tutoring and nearly as effective as human tutoring.

Dialogue-Based Tutors

Unlike tutoring systems that provide step-level guidance through structured problem-solving models, dialogue-based tutoring systems implement tutoring strategies through natural-language interaction. AutoTutor guides learners through a natural-language dialogue organized around expectations (components of a good answer) and misconceptions. It progresses from a pump to a hint, a prompt, and finally an assertion until an expectation is covered, and it corrects misconceptions when they are detected (Nye et al., 2014). Why2-Atlas analyzes student essays in qualitative physics to identify misconceptions and missing ideas, then engages the learner in a dialogue to address them (VanLehn et al., 2002); ITSPOKE adds a spoken interface to this system (D. Litman & Silliman, 2004); Affective AutoTutor extended AutoTutor by adapting tutorial dialogue to learners' affective states (D'Mello & Graesser, 2012).

Effectiveness

Meta-analyses generally report positive effects of ITS (Kulik & Fletcher, 2016; Ma et al., 2014). ITS outperform teacher-led group instruction, non-adaptive computer-based instruction, and textbooks, and do not differ significantly from individualized human tutoring (Ma et al., 2014). A review of fifty controlled evaluations likewise found positive effects overall (Kulik & Fletcher, 2016). Evidence for K–12 mathematics is more mixed, suggesting no negative and perhaps very small positive effects (Steenbergen-Hu & Cooper, 2013).

2.3. From Intelligent Tutoring Systems to LLM-Based Tutors

Traditional ITS achieved fine-grained adaptation through hand-authored domain, student, and pedagogical models, but this approach was costly to scale across domains (Murray, 2003). Subsequent work increasingly incorporated data-driven, neural, and language-model-based components.

Data-Driven ITS

As interaction logs became more widely available, hand-authored student models were increasingly complemented by models learned from data. Deep Knowledge Tracing introduced

recurrent neural networks for knowledge tracing, learning a representation of students' latent knowledge states from sequences of interactions (Piech et al., 2015). Reinforcement learning was also used to learn pedagogical policies, which map student states to tutor actions, directly from interaction data (M. Chi et al., 2011).

Neural Dialogue Tutoring

Neural response generation reduced reliance on scripted dialogue, but raised the question of whether generated responses are pedagogically sound. Tack and Piech (2022) proposed the AI Teacher Test, which compares model responses with real teacher responses on three abilities: speaking like a teacher, understanding the student, and helping the student. They found that Blender and GPT-3 performed significantly worse than teachers, especially on helping the student (Tack & Piech, 2022). The BEA 2023 shared task extended this evaluation to a broader range of systems (Tack et al., 2023). Macina, Daheim, Wang, et al. (2023) identified limited and low-quality tutoring data, along with inadequate modeling of student understanding and pedagogical strategies, as central obstacles for neural dialogue tutoring (Macina, Daheim, Wang, et al., 2023).

LLM-Based Tutors

Large language models made fluent, open-ended tutoring dialogue possible across a broader range of domains. Deployed systems such as Khanmigo rely on carefully engineered prompts (Khan Academy, 2023); research systems combine prompting with learning-science design principles (Schmucker et al., 2024; Sonkar et al., 2023), combine an LLM with an AutoTutor-style dialogue structure and guardrails (Pal Chowdhury et al., 2024), apply pedagogy-informed post-training or fine-tuning to improve tutoring behavior (LearnLM Team, 2024; J. Liu et al., 2024), or assist human tutors in real time (R. E. Wang et al., 2024).

Why Alignment Is Needed

General-purpose LLMs are trained to be helpful assistants, which can conflict with tutoring strategies that require withholding answers and guiding students through productive struggle. In MathDial, ChatGPT prompted to act as a teacher often revealed solutions directly and provided incorrect feedback (Macina, Daheim, Pal Chowdhury, et al., 2023). A field experiment with high-school students found that access to a standard GPT-4 interface improved practice performance but reduced performance on a later unassisted exam, whereas a version with teacher-designed safeguards largely avoided this harm (Bastani et al., 2025). In contrast, a randomized controlled trial at university level found that a carefully designed AI tutor led to larger learning gains in less time than in-class active learning (Kestin et al., 2025). Together, these findings suggest that the educational value of an LLM tutor depends substantially on how its behavior is designed and constrained for learning.

2.4. Educational Dialogue Data

2.4.1. Settings: Classroom Discourse and Tutoring Dialogue

While this distinction is only mentioned in a few papers (Cao et al., 2025), we noticed that educational dialogue are commonly distinguished in two settings. *Classroom discourse* involves whole-class interaction between one teacher and multiple students and is typically derived from recorded classroom lessons that are subsequently transcribed (Demszky & Hill, 2023; Suresh, Jacobs, Harty, et al., 2022). *Tutoring dialogues* involve one-to-one, and sometimes small-group, interaction between a tutor and a learner (Caines et al., 2020; Han et al., 2024; Macina, Daheim, Pal Chowdhury, et al., 2023; Stasaski et al., 2020; R. Wang et al., 2024). The contrast is useful for interpreting the meaning of annotations, but it does not capture the full range of educational dialogue. In addition, recent work has used LLMs to generate partly or fully synthetic tutoring dialogues, for example in MathDial, CLASS, and SocraticLM (J. Liu et al., 2024; Macina, Daheim, Pal Chowdhury, et al., 2023; Sonkar et al., 2023).

2.4.2. Differences Between the Two Settings

The two settings differ in ways that shape what can be annotated and modeled. In classroom discourse, a teacher interacts with multiple students, and interaction often follows initiation–response–feedback (IRF) sequences. Teachers must also manage participation across the class. Consequently, analyses often focus on the quality of teacher talk, including discourse moves, uptake of student ideas, and question types, and may relate these measures to instructional-observation scores or class-level learning outcomes (Demszky & Hill, 2023; Suresh, Jacobs, Harty, et al., 2022; Zent et al., 2025). Because individual students may contribute only a small number of turns in whole-class recordings or interact with class peers, tracing a particular student’s errors and the teacher’s subsequent response is difficult.

The two settings also pose different data-collection constraints. To our best knowledge, the publicly available classroom corpora are frequently derived from audio or video recordings and subsequent transcripts of naturally occurring lessons, which can contain transcription noise and require careful privacy protection, particularly when minors are recorded. Tutoring corpora are often smaller and more text-based, but can provide denser evidence about a learner’s reasoning and a tutor’s targeted response. Because this thesis investigates how a tutor should respond to a specific student error, tutoring dialogue is the more directly relevant setting.

2.4.3. Publicly Available Datasets

Access to educational dialogue data is limited because collecting and sharing recordings of real instructional interactions is resource-intensive and raises substantial privacy concerns, particularly when minors are involved (Demszky & Hill, 2023). To the best of our knowledge, Table 2.1 lists the publicly available datasets that have been most explored in the analyses of educational dialogue (Cao et al., 2025; Macina et al., 2025; Maurya et al., 2025b; Scarlatos, Liu,

2.4. Educational Dialogue Data

Dataset	Set.	Domain	Size	Participants	Access
NCTE (Demszky & Hill, 2023)	C	Math, grades 4–5	1,660 lesson transcripts	Real classes	Request form
TalkMoves (Suresh, Jacobs, Harty, et al., 2022)	C	K–12 math	567 lesson transcripts	Real classes	Open
CIMA (Stasaski et al., 2020)	T	Italian for English speakers	391 dialogues	Crowdworkers (role-play)	Open
TSCC (Caines et al., 2020, 2022)	T	English as a second language	260 chat lessons (v2)	Real teachers and learners	Request form
MathDial (Macina, Daheim, Pal Chowdhury, et al., 2023)	T	Math word problems	ca. 3k dialogues	Human teachers, LLM student	Open
StepVerify (Daheim et al., 2024)	T	K–12 math word problems	1,002 solutions (612 problems)	Teachers; LLM solutions from MathDial	Open
Bridge (R. Wang et al., 2024)	T	Elementary math	700 conversation excerpts	Real tutoring sessions	Open

Table 2.1.: Publicly accessible educational dialogue datasets. Not all publicly available educational dialogue datasets are shown. *C* = classroom, *T* = tutoring.

et al., 2025). They differ in the extent to which they capture interaction between human participants in naturally occurring educational settings, a distinction that matters when interpreting findings. NCTE (Demszky & Hill, 2023) and TalkMoves (Suresh, Jacobs, Harty, et al., 2022) contain transcripts of real classroom interaction between teachers and students. TSCC (Caines et al., 2020, 2022) contains written transcripts of online English lessons between teachers and learners. CIMA (Stasaski et al., 2020) consists of tutoring dialogues in which crowdworkers role-play both the tutor and student roles. MathDial (Macina, Daheim, Pal Chowdhury, et al., 2023) pairs human teachers with LLM-simulated students. StepVerify (Daheim et al., 2024) adds teacher annotations of the first erroneous step, an error type, and an error description to student solutions from MathDial. We did not include the datasets that were not publicly available.

NCTE and TalkMoves

The NCTE (Demszky & Hill, 2023) corpus contains transcripts of elementary mathematics lessons collected between 2010 and 2013, together with classroom observation scores, teacher and student questionnaires, and student test scores; a subset of turns is annotated for discourse moves such as teacher uptake, focusing questions, and student reasoning. A record is an utterance with a speaker role, a transcript identifier, and a teacher identifier that links to the metadata. TalkMoves (Suresh, Jacobs, Harty, et al., 2022) contains K–12 mathematics lesson transcripts in which each teacher and student utterance carries one Accountable Talk label and a general dialogue-act label.

CIMA and TSCC

In CIMA, a crowdworker in the student role translates a phrase into Italian and a crowdworker in the tutor role helps. Each student turn is labeled with a student action and each tutor turn with a tutor action, and several tutors respond to the same context, which shows that more than one response can be appropriate (Stasaski et al., 2020). The Teacher-Student Chatroom Corpus contains one-to-one English lessons held in a chat room; turns are annotated with conversational sequence types, the teaching focus, and the use of feedback, and learners are graded for proficiency (Caines et al., 2020, 2022).

MathDial and StepVerify

MathDial is grounded in multi-step mathematics word problems (Macina, Daheim, Pal Chowdhury, et al., 2023). An LLM simulates a student by generating an incorrect solution conditioned on a student profile, and a human teacher then tutors the simulated student towards the correct solution. Each record contains the problem, the ground-truth solution, the incorrect student solution, the student profile, the annotated student confusion, and the tutoring dialogue. Teacher turns are annotated with tutor moves: Focus and Probing operationalize scaffolding strategies inspired by Reiser’s distinction between structuring and problematizing, whereas Telling and Generic capture answer-revealing and general conversational moves (Macina, Daheim, Pal Chowdhury, et al., 2023; Reiser, 2018).

StepVerify extends MathDial with teacher annotations of stepwise student solutions (Daheim et al., 2024). The release contains 1,002 solutions across 612 K–12 mathematics word problems. For each solution, a teacher identifies the first erroneous step, assigns an error type, and writes an error description. Later steps are not annotated because they often follow from the first error. StepVerify includes incorrect student solutions and rephrased reference solutions derived from MathDial, together with the associated MathDial dialogue context.

Bridge

Bridge contains 700 excerpts from real chat-based elementary mathematics tutoring sessions in which a tutor responds to a student mistake (R. Wang et al., 2024). Through cognitive task analysis with experienced teachers, the authors model the expert’s remediation process as three decisions: identifying the error type, selecting a remediation strategy, and specifying the intention behind it. Each record includes the lesson topic, conversation history, original tutor response, expert annotations for the three decisions, and an expert-written response. The annotation scheme covers categories of student errors, remediation strategies, and pedagogical intentions. Among the publicly available datasets reviewed here, Bridge is the only one that jointly annotates student error types and tutor remediation strategies with expert judgments on real tutoring data (R. Wang et al., 2024).

2.5. Dialogue Acts in Educational Dialogue

A dialogue act (DA) is the communicative function of an utterance, such as a question, a statement, or an acknowledgment (Bunt et al., 2012). The concept goes back to speech act theory, which holds that utterances are actions (Austin, 1962; Searle, 1969). Annotation schemes such as DAMSL and its Switchboard adaptation made the concept operational (Core & Allen, 1997; Jurafsky et al., 1997). Jurafsky et al. (1997) consolidated the Switchboard scheme into 42 tags, and Stolcke et al. (2000) established statistical DA tagging on the Switchboard corpus using this tag set (Jurafsky et al., 1997; Stolcke et al., 2000). ISO 24617-2 subsequently standardized a domain-independent scheme (Bunt et al., 2012). General-purpose schemes can identify that a tutor asks a question, but they do not indicate whether the question probes the student’s reasoning or leads the student to the answer. Educational research therefore uses domain-specific act inventories, referred to as talk moves, tutor moves, or pedagogical strategies (Caines et al., 2020; Macina, Daheim, Pal Chowdhury, et al., 2023; Michaels et al., 2008).

2.5.1. Uses of Dialogue Acts in Educational Dialogue

Prior work uses dialogue acts for four main purposes that are particularly relevant to this thesis.

Linking Discourse to Learning

Dialogue acts turn free text into variables that can be related to outcomes. In spoken tutoring, the frequencies of specific student and tutor acts correlate with learning gains (D. J. Litman & Forbes-Riley, 2006). Hidden Markov models over act sequences reveal tutoring modes whose frequency is associated with learning (Boyer et al., 2011), and sequence mining over act-annotated dialogues identifies tutor strategies that distinguish successful from unsuccessful sessions (J. Lin et al., 2022). In classroom settings, automatic measures of authentic questions and uptake are associated with instructional quality and student outcomes (Demszky et al., 2021; Kelly et al., 2018).

Feedback to Teachers and Tutors

Once acts can be detected automatically, they can be reported back. The TalkMoves application gives mathematics teachers a summary of their talk moves after each lesson (Suresh et al., 2019); similar feedback on teacher discourse is described by (Jensen et al., 2020); and automated feedback on uptake improved tutors’ practice in a randomized trial (Demszky & Liu, 2023).

Control Signal for Generation

A dialogue act provides an interpretable plan for the next turn. In CIMA and MathDial, tutor-response generators can be conditioned on a specified tutor action or tutor move, enabling responses to be generated in accordance with that intended move (Macina, Daheim, Pal Chowdhury, et al., 2023; Stasaski et al., 2020). In Bridge, providing GPT-4 with an expert’s diagnosis

of the student’s error, remediation strategy, and strategy intention produced responses that teachers markedly preferred over responses generated without decision-making; randomly selected decisions performed worse than omitting decisions altogether (R. Wang et al., 2024). In a field experiment, human tutors given strategy-based suggestions from Tutor CoPilot used more high-quality instructional strategies, and their students were more likely to demonstrate topic mastery (R. E. Wang et al., 2024).

Evaluation of AI Tutors

AI-tutor evaluation often operationalizes pedagogical quality as response-level functional properties: whether a response identifies and locates a student’s mistake, reveals the answer, provides relevant guidance, and makes clear what the student should do next (Kochmar et al., 2025; Maurya et al., 2025b; Tack & Piech, 2022).

2.5.2. Dialogue Act Classification

Automatic classification of educational dialogue acts has developed alongside advances in text classification. Early work in ITS used hand-crafted lexical and syntactic features with classical classifiers (Marineau et al., 2000), and examined the use of preceding dialogue context for speech-act classification (Samei et al., 2014). Later work applied neural classifiers, while more recent studies fine-tuned pre-trained language models, including BERT and RoBERTa, to classify talk moves, uptake, and question types in classroom discourse (Alic et al., 2022; Demszky et al., 2021; Suresh, Jacobs, Harty, et al., 2022).

2.6. Modeling Students and Tutors in Educational Dialogue

This section reviews how prior work represents students, models tutor behavior, and connects student-side information with tutor responses. The first two parts consider student modeling and tutor modeling separately. The third part examines approaches that explicitly connect the students and tutors.

2.6.1. Modeling the Student

Prior work represents different aspects of the student, including dialogue acts, errors and misconceptions, knowledge states, and affective states (Pavlik et al., 2013). A related line of research simulates student responses and behavior rather than representing individual aspects of an observed student (Macina, Daheim, Pal Chowdhury, et al., 2023; Markel et al., 2023).

Student Dialogue Acts

Student dialogue acts characterize the function of a student’s utterance within an interaction, such as making a guess, asking a question, or contributing reasoning (Stasaski et al., 2020; Suresh, Jacobs, Harty, et al., 2022). These acts are obtained through annotation or classification and provide a representation of how the student participates in the dialogue. They describe

the student’s observable contribution rather than directly estimating the student’s underlying knowledge or understanding.

Errors and Misconceptions

In the ITS approaches discussed in Section 2.2, student errors can be represented through predefined buggy rules or violations of domain constraints. Dialogue-based resources use different representations. MathDial includes a free-text description of the student’s confusion (Macina, Daheim, Pal Chowdhury, et al., 2023), whereas Bridge identifies error types through expert teachers’ analyses of student mistakes (R. Wang et al., 2024). Daheim et al. (2024) annotate the first erroneous step in a student’s solution and investigate how verifying solution steps before generating a response can make tutor responses more targeted and reduce hallucinations. MRBench evaluates whether a tutor response identifies the student’s mistake and locates it within the solution (Maurya et al., 2025b). A distinction relevant to this thesis is between identifying whether and where a mistake occurred and using the category of that mistake to guide the tutor’s response. Bridge explicitly address the latter by connecting student errors with remediation strategies and their intentions (R. Wang et al., 2024).

Knowledge State

Knowledge tracing models a learner’s knowledge from a history of observed responses, commonly represented as correct or incorrect, and uses this history to predict subsequent performance (Corbett & Anderson, 1995). Scarlatos, Baker, and Lan (2025) apply this approach to open-ended tutoring dialogue; LLMKT uses LLM-based annotations of knowledge components and student-response correctness, together with the dialogue history, to estimate mastery and predict whether the student will respond correctly to the tutor’s next task.

Affect and Uncertainty

Spoken and multimodal tutoring systems also consider uncertainty, confusion, and disengagement when adapting tutor responses. Studies of these systems suggest that responding to such signals can improve learning for some groups of students (D’Mello & Graesser, 2012; Forbes-Riley & Litman, 2011). These approaches therefore consider aspects of the student’s experience that are not captured by the correctness of a response alone.

Simulated Students

Several recent studies use LLMs to simulate students. MathDial conditions a simulated student on a profile and an incorrect solution (Macina, Daheim, Pal Chowdhury, et al., 2023); GPTeach uses GPT-simulated students to support teacher training (Markel et al., 2023); and Generative Students uses LLM-simulated student profiles and their responses to support question-item evaluation (Lu & Wang, 2024). Such simulations make it possible to conduct studies without recruiting real learners, but they may not fully reproduce the errors, knowledge states, or learning behavior of real students. The CLASS framework likewise uses GPT-4 to generate simulated student–tutor conversations.

2.6.2. Modeling the Tutor

Whereas student modeling represents aspects of the learner, tutor modeling concerns the tutor's behavior and the pedagogical functions of tutor utterances. The studies reviewed here differ both in the behaviors they represent—including talk moves, uptake, questions, tutor actions, and pedagogical strategies—and in how those representations are used for analysis, prediction, or response generation (Alic et al., 2022; Demszky et al., 2021; Macina, Daheim, Pal Chowdhury, et al., 2023; Stasaski et al., 2020; Suresh, Jacobs, Harty, et al., 2022; R. Wang et al., 2024).

What Is Modeled

Classroom research examines Accountable Talk moves (Suresh, Jacobs, Harty, et al., 2022), conversational uptake, defined as the extent to which a teacher builds on a student's contribution (Demszyk et al., 2021), and the distinction between funneling and focusing questions. Funneling questions lead the student toward a predetermined answer, whereas focusing questions make the student's own thinking the object of the conversation (Alic et al., 2022).

Research on tutoring dialogue represents behaviors such as hints, corrections, questions, and confirmations (Stasaski et al., 2020), and examines remediation strategies together with their intentions (R. Wang et al., 2024). Research on tutor training also evaluates specific skills, such as responding to a student's mathematical error in a motivating and non-discouraging manner, and investigates whether LLMs can assess tutors' responses (Kakarla et al., 2024).

How It Is Modeled

Three approaches can be distinguished. The first classifies tutor utterances, as described in Section 2.5, and uses the labels for analysis or feedback (Suresh, Jacobs, Harty, et al., 2022). The second predicts the next tutor move from the dialogue context and then generates the response conditioned on it; Macina, Daheim, Wang, et al. (2023) evaluate this setting across CIMA, TSCC, and TalkMoves and find that strategy prediction is difficult, partly because several moves are acceptable in the same context (Macina, Daheim, Wang, et al., 2023). The third makes the expert's reasoning explicit. Bridge decomposes the expert's reaction into error, strategy, and intention, and shows that novice tutors and LLMs fall short mainly because they skip this reasoning (R. Wang et al., 2024).

2.6.3. Joint Modeling of Student and Tutor

This subsection reviews work that connects student-side information with tutor actions or responses.

Before LLMs

D. J. Litman and Forbes-Riley (2006) used correlation analyses to relate the frequencies of student responses and questions, and of tutor feedback and questions, to student learning gains (D. J. Litman & Forbes-Riley, 2006). Boyer et al. (2011) trained hidden Markov models

on sequences of student and tutor dialogue acts, together with task actions, to identify latent tutoring modes whose frequencies were associated with learning (Boyer et al., 2011). J. Lin et al. (2022) used sequential pattern mining to compare combinations of student responses with tutor actions, such as questions, hints, and feedback, across more and less successful tutoring sessions (J. Lin et al., 2022). M. Chi et al. (2011) used reinforcement learning to learn a policy that maps a tutoring state, including the difficulty of the current step and features of the ongoing dialogue, to a tutor action: eliciting the next step from the student or telling it to the student (M. Chi et al., 2011). Forbes-Riley and Litman (2011) used the correctness and detected uncertainty of a student’s spoken response to select an adaptive tutor response (Forbes-Riley & Litman, 2011).

With LLMs

Bridge links a student’s error type to a tutor’s remediation strategy and instructional intention, and includes these decisions in an LLM prompt (R. Wang et al., 2024). Daheim et al. (2024) connect a verified erroneous step in a student’s solution to the generated tutor response (Daheim et al., 2024). Park et al. (2024) incorporate an explicit student model into a conversational tutor and use it to personalize the prompt (Park et al., 2024). Scarlatos, Liu, et al. (2025) use a dialogue knowledge-tracing model to estimate the probability that a student will answer the tutor’s next task correctly, and combine this estimate with a pedagogical-quality score when training an LLM-based tutor (Scarlatos, Liu, et al., 2025). Dinucu-Jianu et al. (2025) similarly train a tutor model through interactions with simulated students, using whether the student solves the problem without answer leakage as a learning signal (Dinucu-Jianu et al., 2025).

2.7. Pedagogical Alignment of Large Language Models

Pedagogical alignment denotes methods that make an LLM behave like a good tutor and not like a question-answering assistant (Sonkar et al., 2024). Three groups of methods are used.

2.7.1. Prompting

Prompting specifies the desired tutoring behavior in the inference-time context without updating the model parameters. For example, Khan Academy (2023) instructs the model to guide students through questions and hints rather than directly provide an answer (Khan Academy, 2023). Conditioning prompts can also provide a representation of the current dialogue state, such as a pedagogical dialogue act in MathDial (Macina, Daheim, Pal Chowdhury, et al., 2023), an expert annotation of the student’s error, strategy, and intention in Bridge (R. Wang et al., 2024), or a verified erroneous step in StepwiseDaheim et al., 2024. Prompting requires no additional task-specific training, but it requires developers to encode pedagogical principles as a detailed natural-language policy because effective pedagogy is nuanced and context-dependent, such policies are difficult to specific (Jurenka et al., 2024a).

2.7.2. Supervised Fine-Tuning

Supervised fine-tuning (SFT) trains a model to reproduce reference tutor responses. References may come from human tutors, as in models fine-tuned on MathDial dialogues (Macina, Daheim, Pal Chowdhury, et al., 2023); from a stronger LLM, as in CLASS, where GPT-4 generates tutoring dialogues that are used to train a tutor model (Sonkar et al., 2023); or from multi-agent simulation, as in SocraticLM, which is trained on simulated tutoring dialogues (J. Liu et al., 2024). SFT has two limitations. First, it imitates reference behavior regardless of its pedagogical quality. Scarlatos, Liu, et al. (2025) found that a model fine-tuned on human MathDial utterances scored lower on their pedagogical rubric than the untuned base model because many human tutor turns were short and provided little guidance (Scarlatos, Liu, et al., 2025). Second, SFT shows the model what to do, but not which plausible responses it should avoid.

2.7.3. Preference Optimization and Reinforcement Learning

Preference-based methods learn from comparisons between candidate responses. Reinforcement learning from human feedback first trains a reward model on human preference comparisons and then optimizes the language model against that reward (Ouyang et al., 2022). Direct Preference Optimization (DPO) instead directly optimizes a policy from pairs of preferred and dispreferred responses, without first learning a separate reward model (Rafailov et al., 2023). For pedagogical alignment, the central problem is how to obtain comparisons that reflect desirable tutoring behavior without collecting preferences from expert teachers at scale.

Sonkar et al. (2024) construct preference pairs from synthetic tutoring dialogues following the CLASS framework. Responses that follow the framework’s pedagogical behavior are treated as preferred, whereas responses that violate it are treated as dispreferred. Ashok Kumar and Lan (2024) augment a code-debugging dataset with invalid Socratic questions, including questions that repeat earlier questions or reveal the solution directly. They use the invalid questions as dispreferred examples for DPO (Ashok Kumar & Lan, 2024). Scarlatos et al. (2024) define a rubric for feedback on math answers, let GPT-4 score human-written and generated feedback messages, and form DPO pairs from the scores (Scarlatos et al., 2024). Scarlatos, Liu, et al. (2025) combine the rubric score with the prediction of a student model Scarlatos, Liu, et al., 2025, and Dinucu-Jianu et al. (2025) use the success of a simulated student as reward Dinucu-Jianu et al., 2025. LearnLM collects preference judgments from human raters for reinforcement learning from human feedback LearnLM Team, 2024.

2.8. Pedagogical Evaluation

Evaluating pedagogical alignment requires more than assessing whether a tutor produces a correct answer. Recent work evaluates LLM tutors through three complementary forms of evidence: human judgments of tutor behavior, rubric-based automated evaluation, and learner-centred measures of progress or learning outcomes.

2.8.1. Human and Expert Evaluation

The most direct approach asks teachers, pedagogical experts, or learners to evaluate tutor responses in context. The BEA 2023 shared task used human evaluation of generated teacher responses and showed that conventional reference-based generation metrics were poorly aligned with pedagogical quality judgments (Tack et al., 2023). MathDial instead annotates tutor turns with pedagogical actions, enabling analysis of the kinds of support a tutor provides (Macina, Daheim, Pal Chowdhury, et al., 2023). Bridge grounds evaluation in expert decision-making about how to remediate a student’s error, while LearnLM uses pedagogy experts to assess complete conversations and compare responses through pairwise preferences (Jurenka et al., 2024a; LearnLM Team, 2024; R. Wang et al., 2024). These methods capture context-sensitive instructional quality, but are expensive and depend on the raters and evaluation protocol.

2.8.2. Rubric-Based and Automated Evaluation

A second line of work formalizes pedagogical quality through explicit rubrics or labels and applies them with human annotators, LLM judges, reward models, or supervised classifiers. MRBench provides a unified taxonomy for response-level evaluation and measures whether responses receive the desired human annotation (Maurya et al., 2025b). TutorBench evaluates generated responses with an LLM judge against expert-written pedagogical rubrics, whereas MathTutorBench uses a pedagogical reward model to compare generated and teacher responses (Macina et al., 2025; Srinivasa et al., 2025). The BEA 2025 shared task operationalizes selected pedagogical properties as classification tasks using human annotations as gold labels (Kochmar et al., 2025). These methods enable scalable comparison, although the agreement between automatic and human judgments remains uneven (Maurya et al., 2025b).

2.8.3. Learner-Centred Validation

The strongest evidence evaluates effects on learners rather than the apparent quality of individual tutor responses. Randomized and controlled studies measure independent performance, learning gains, or mastery after tutoring: unrestricted GPT-4 access improved practice performance but harmed later unassisted exam performance, whereas pedagogical guardrails avoided this effect (Bastani et al., 2025); other studies report learning or mastery benefits for a purpose-built tutor and AI support for human tutors (Kestin et al., 2025; R. E. Wang et al., 2024). LLMKT complements these outcome studies by estimating learner mastery and predicting correctness on a subsequent task from dialogue history (Scarlatos, Baker, & Lan, 2025). It therefore evaluates student-state modeling rather than the tutor’s causal effect on learning.

3. Research Gap and Research Questions

In this chapter, we present the research gaps identified in the previous chapter and the research questions this thesis intends to answer.

3.1. Research Gap

Learning-science and ITS research treat learner errors as diagnostic evidence rather than merely incorrect outcomes: errors may reflect systematic bugs, misconceptions, or deficient prerequisite knowledge, and effective tutoring should identify the relevant error, provide contingent scaffolded feedback, and guide the learner towards a corrective next step rather than prematurely disclose the answer (Brown & Burton, 1978; Hattie & Timperley, 2007; Metcalfe, 2017; Radatz, 1979; VanLehn, 1990; Wood et al., 1976). Classical ITS operationalized this principle through student models, buggy rules, constraint violations, and error-specific feedback, enabling tutors to diagnose particular deviations from a target solution and select corresponding support (Anderson et al., 1995; Mitrovic & Ohlsson, 1999; Woolf, 2009). LLM tutors offer more flexible dialogue, but general-purpose models are optimized to be helpful assistants rather than tutors and therefore may provide feedback that is insufficiently grounded in student reasoning (Jurenka et al., 2024a; R. Wang et al., 2024). Such behavior also undermines learning outcomes in real educational settings (Bastani et al., 2025). Existing resources make important aspects of both sides of tutoring observable: MathDial represents student confusion and tutor moves; Bridge links student error types with expert remediation strategies and instructional intentions; and MRBench evaluates the pedagogical quality of tutor responses to student mistakes (Macina, Daheim, Pal Chowdhury, et al., 2023; Maurya et al., 2025a; R. Wang et al., 2024).

Existing alignment methods derive preference signals from different forms of tutor-side and student-side evidence. Tutor-side approaches use predefined pedagogical behaviors, synthetic contrasts, or rubric-based evaluations of response quality (Scarlatos et al., 2024; Sonkar et al., 2024). Student-side approaches use simulated student success or predicted student correctness as learning-related optimization signals (Dinucu-Jianu et al., 2025; Scarlatos, Liu, et al., 2025). Scarlatos, Liu, et al. (2025) further combine a tutor-side pedagogical-quality score with a dialogue knowledge-tracing model’s prediction of next-turn student correctness. This demonstrates that tutor-side and student-side signals can be jointly incorporated when constructing preferences (Scarlatos, Liu, et al., 2025). However, this joint signal represents the learner through a predicted outcome rather than through an interpretable error type, and it does not connect particular error types to particular tutor moves.

Bridge provides the ingredients for such a connection by linking student error types to

expert-selected remediation strategies and instructional intentions, but its reported generation setup uses these decisions as prompt inputs rather than as an empirical basis for preference construction (R. Wang et al., 2024). To the best of our knowledge, no prior work constructs tutor-alignment preferences from observed associations between student error types and tutor moves. This thesis addresses that gap by combining student-side error types and tutor-side tutor moves in a preference dataset, evaluating whether preference optimization changes pedagogical quality across error types, and identifying which error types contribute most to the resulting steering.

3.2. Research Questions

RQ 1. What associations are observed between student error types and tutor moves, and how are these associations reflected in the constructed preference dataset?

RQ 2. How do changes in the pedagogical quality of tutor responses following preference optimization vary across student error types?

RQ 3. If a preference-optimized model is pedagogically steered, which student error types exert the influence on this steering?

4. Methodology

This chapter describes the methodology used to answer the research questions of this thesis. We begin with an overview of the pipeline of this work. We then describe the annotation transfer process used to construct the main dataset (aligned dataset) and introduce the tutoring-domain-specific corpus used for adaptive pretraining of the tutor-move classifier. Next, we describe the two components of preference dataset construction: training and selecting tutor-move classifiers under various configurations in low-resource class-imbalance settings, and generating candidate tutor responses that are scored using an error-conditioned tutor-move scoring method. Finally, we describe how we align LLMs with pedagogical principles through supervised fine-tuning and preference optimization, and how their pedagogical quality is evaluated.

4.1. Pipeline Overview

The overview of the pipeline is shown in Figure 4.1. Our main sources of data are MathDial (Macina, Daheim, Pal Chowdhury, et al., 2023) and StepVerify (Daheim et al., 2024), the latter of which is a derived subset of MathDial that contains richer student annotations, including student error types in addition to the original tutor-turn pedagogy annotations in MathDial. Through annotation transfer, we construct an aligned dataset that pairs student utterances labeled with specific error types with corresponding tutor-turn utterances labeled with tutor moves (Section 4.2). This aligned dataset serves as the empirical source for analyzing associations between student errors and tutor moves throughout the thesis.

After the aligned dataset is constructed, the remaining data points from MathDial, which are not used to construct the aligned dataset, are held out and used as a training dataset for supervised fine-tuning (Section 4.5.2). The aligned dataset is split into training and test splits, with the test split of the aligned dataset reserved for evaluating pedagogically aligned models (Section 4.5).

The entire aligned dataset is used to train tutor-move classifiers under multiple input representations, loss functions, and training configurations (Section 4.3). In parallel, a tutoring-domain corpus from QATD2k (Zent et al., 2025) and MathMentorDB (Ion, 2026) is used for domain-adaptive pretraining of the classifier (Section 4.3.2). The best-performing classifier is selected based on our chosen evaluation metrics. The selected classifier takes a tutor response and outputs a probability distribution over tutor moves, which serves as the interface for the error-conditioned scoring method (Section 4.4.2).

Using each dialogue context in the training split of the aligned dataset, we generate synthetic tutor responses and score them with the error-conditioned tutor-move scoring method

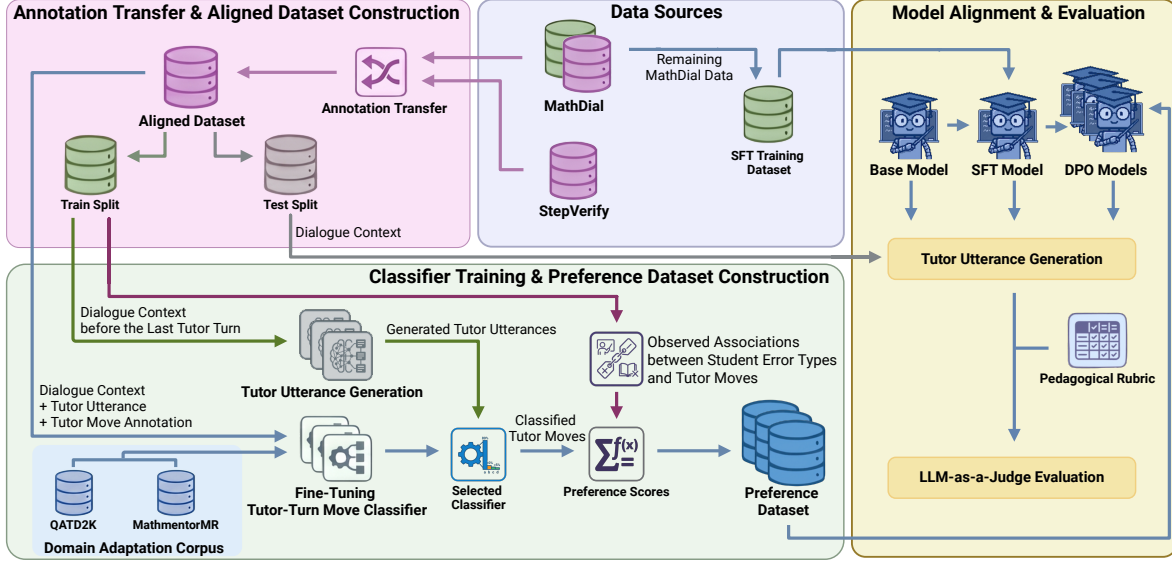


Figure 4.1.: Overview of the methodology pipeline, including preference dataset construction using error-conditioned scoring, LLM-based tutor training through supervised fine-tuning and preference optimization, and rubric-based pedagogical evaluation.

(Section 4.4.2), which provides the basis for answering Research Question 1. Preference pairs are constructed using these scores so that the preference dataset encodes the observed association between student error types and tutor moves as a pairwise empirical training signal.

With the constructed preference dataset, a base LLM is aligned in two stages: supervised fine-tuning followed by preference optimization. For preference optimization, preference pairs are ranked by score margins within each error type, and the highest-ranked pairs are selected using different selection ratios. For each selection ratio, the selected pairs are pooled across error types to form a training dataset, which is then used to train separate DPO models. The pedagogical quality of the SFT and DPO models, as well as the base model, is evaluated by an LLM-as-a-Judge model using the chosen pedagogical rubrics (Section 4.5). This evaluation aims to answer Research Questions 2–3.

4.2. Dataset Construction

In this section, we introduce the source datasets that form the basis for constructing the main dataset used throughout this thesis, and we describe how this dataset is constructed. For clarity, we refer to this constructed dataset as the *aligned dataset*, which distinguishes it from the original source datasets.

4.2.1. Source Dataset for Annotation Transfer

To investigate associations between student error types and tutor moves, this thesis requires dataset that links student-error annotations to corresponding tutor utterances and their tutor-move labels. These associations are subsequently used to construct a preference dataset that allows us to examine how error-conditioned preference optimization of LLMs influences the pedagogical quality of the generated tutor responses. During data selection, we aim to use datasets that not only incorporate human teacher–student multi-turn interactions but also have a large volume of utterances. However, datasets of human-to-human tutoring interactions with fine-grained annotations of both student errors and tutor moves are limited. To meet these requirements, we select MathDial (Macina, Daheim, Pal Chowdhury, et al., 2023) and StepVerify (Daheim et al., 2024) as a practical compromise.

MathDial provides multi-turn tutoring dialogues with human-authored tutor responses and utterance-level tutor-move annotations. However, a limitation of the dataset is that the student utterances are generated by LLMs. StepVerify provides additional human-tutor annotations of student solutions drawn from MathDial, including student error types. The dialogue contexts of both datasets come from GSM8k (Cobbe et al., 2021) multi-step math word problems. MathDial’s human-authored tutor responses and their annotations, and StepVerify’s human-authored annotations for students’ solutions provide a basis for identifying corresponding instances and transferring tutor-move annotations from MathDial to StepVerify instances.

MathDial

MathDial (Macina, Daheim, Pal Chowdhury, et al., 2023) was collected through interactions between human teachers and LLM-simulated students using problems from GSM8k, a collection of elementary-level multi-step math word problems (Cobbe et al., 2021). Each dialogue context was grounded in an initial incorrect solution generated by `gpt-3.5-turbo`. Student utterances were generated by `text-davinci-003`, conditioned on the dialogue history, initial incorrect solution, and one of six student profiles representing common algebraic misconceptions. Teachers had access to the problem, reference solution, and initial incorrect solution, and selected a tutor move from a predefined taxonomy before giving each response. Each conversation ended when the student correctly solved the problem or the conversation exceeded 10 minutes. Teachers then recorded whether the student had solved the problem correctly, distinguishing cases where they had revealed the answer. Although the annotation procedure included classifying initial incorrect solutions into seven error types, these annotations are not included in the public release.

Each record contains a complete dialogue together with the problem, reference solution, initial incorrect student solution, and student profile. The dataset also provides teacher-written descriptions of student confusion during the conversation and metadata such as problem identifiers. Tutor utterances are annotated with one of four tutor-move labels (Focus, Probing, Telling, and Generic), whose descriptions and examples are shown in Table 4.1. The dataset contains 2,861 tutoring dialogues covering 1,111 unique math problems, with 16,737 student

4.2. Dataset Construction

Category	Intent	Example
Focus	Guiding Student Focus	Can you calculate ...?
	Recall Relevant Information	Can you reread the question and tell me what is ...?
	Seek Strategy	So what should you do next?
Probing	Asking for Explanation	Why do you think you need to add these numbers?
	Perturbing the Question	How would things change if they had ...items instead?
	Seeking Self Correction	Are you sure you need to add here?
	Seeking World Knowledge	How do you calculate the perimeter of a square?
Telling	Revealing Answer	No, he had ...items.
	Revealing Strategy	You need to add ...to ...to get your answer.
Generic	General inquiry	Can you go walk me through your solution?
	Greeting/Farewell	Hi ..., how are you doing with the word problem?
		Good Job! Is there anything else I can help with?

Table 4.1.: Teacher moves with examples of utterances and their intents from the MathDial dataset. Adapted from Table 2 (Macina, Daheim, Pal Chowdhury, et al., 2023).*

* Copyright 2023 by the Association for Computational Linguistics. Licensed under CC BY 4.0.

utterances and 18,609 tutor utterances, and is publicly available on Hugging Face ¹ and the authors’ GitHub repository ². In this thesis, MathDial serves as the source of tutor-move annotations, which are transferred to the corresponding tutor utterances in StepVerify through the annotation-transfer pipeline described in Section 4.2.2.

StepVerify

StepVerify (Daheim et al., 2024) extends MathDial by adding teacher-authored annotations of eight student error types and corresponding descriptions to the initial incorrect student solution. Subsequent errors were not annotated because the authors claimed that they could result from the initial mistake. To assess student error annotation reliability, a random sample comprising 10% of the incorrect student solutions was independently annotated by an additional teacher and compared with the original annotations, yielding Cohen’s $\kappa = 0.75$, which indicates substantial inter-rater agreement.

Although the original StepVerify paper describes eight student error types, the released dataset combines *missing quantity* and *extra quantity* into a single category. Accordingly, this thesis uses the following seven student error types:

- *missing or incorrect factual knowledge*
- *misunderstanding of the question*
- *reached the correct solution but proceeded further*
- *missing or extra quantity*
- *unit conversion error*
- *numerical calculation error*
- *none of the above*

Each StepVerify record inherits attributes from MathDial, including the problem, lesson

¹<https://huggingface.co/datasets/eth-nlped/mathdial>

²<https://github.com/eth-nlped/mathdial>

topic, reference solution, incorrect student solution, and a short dialogue excerpt, together with additional teacher annotations of the student error type and of the reasoning step where the first mistake was made. The dialogue excerpts retain human-authored tutor utterances from MathDial but do not include the corresponding tutor-move annotations. This thesis therefore uses both annotation schemes: the seven student error types represented in the released StepVerify dataset and the four tutor-move labels from MathDial. Through manual comparison of corresponding records, we confirm that the dialogue excerpts in StepVerify are compatible with the original MathDial dialogues. Figure 4.2 shows an example from both datasets, with tutor-move labels from MathDial and the student error type from StepVerify.

The public release of StepVerify contains 1,002 records covering 612 distinct math problems. Each dialogue consists of three turns. As in MathDial, the first utterance is the tutor’s response to the student’s initial incorrect solution. The StepVerify dataset is available on Hugging Face ³ and the authors’ GitHub repository ⁴.

4.2.2. Annotation Transfer Pipeline

As discussed in the previous section and shown in Figure 4.2, MathDial and StepVerify provide complementary teacher-authored annotations. MathDial provides tutor-move labels for tutor utterances but does not include student error-type annotations. In contrast, StepVerify contains student error-type annotations for the initial incorrect student solutions but omits the corresponding tutor-move labels. Since StepVerify was constructed using math problems, incorrect student solutions, and dialogue excerpts from MathDial, the shared content allows the missing pedagogical annotations in StepVerify to be recovered and transferred from MathDial to the corresponding tutor utterances in StepVerify through hierarchical matching. StepVerify serves as the anchor dataset in this pipeline. The annotation-transfer process consists of four stages: preprocessing, problem matching, dialogue matching, and tutor-utterance matching.

Text-Similarity Measure

Since MathDial and StepVerify share much of the same dialogue text, we use normalized character-sequence similarity to match corresponding utterances. The score ranges from 0 to 1, where 1 indicates identical strings and 0 indicates that no matching character blocks are found. For example, a score of 0.90 means that the number of matched characters equals 90% of the average length of the two strings.

$$\text{sim}(a, b) = \frac{2M(a, b)}{|a| + |b|} \quad \begin{array}{l} M(a, b) : \text{matched character blocks,} \\ |a| : \text{length of } a, \\ |b| : \text{length of } b. \end{array}$$

Preprocessing

In StepVerify, 35 dialogues contain more than three turns. In these dialogues, the actual tutor-student interaction is confined to the final three turns, as the preceding turns are either empty

³<https://huggingface.co/datasets/eth-nlped/stepverify>

⁴<https://github.com/eth-lre/verify-then-generate>

```

[StepVerify only]
error_category: None of the above
[Shared: MathDial question / StepVerify problem]
Javier is selling donuts to raise money for a new game. He wants to raise $96. He buys each dozen
donuts for $2.40 and then sells each donut for $1. How many dozen donuts does he need to buy and
sell to reach his goal?
[Shared: student_incorrect_solution]
Javier makes $1 - $2.40 = -$1.40 loss for each dozen donuts. To raise $96, he needs to sell 96 / $1 =
96 donuts. He needs to buy and sell 96 / 12 = 8 dozen donuts. Since he makes a loss, he needs to
spend 8 x $2.40 = $19.2 to buy the donuts. So, his profit will be $96 - $19.2 = $76.8.
[MathDial conversation: begin]
  [Shared: StepVerify dialog_history begins]
    Teacher: (probing)You have made an error at the start. Each dozen donuts sold at $1 per donut
would be how much
    Student: Each dozen donuts sold at $1 per donut would be $12
    Teacher: (focus)So he had spent $2.40 per dozen and makes a profit of 9.60 per dozen do you
see that
  [Shared: StepVerify dialog_history ends]
    - remaining dialogues... -
[MathDial conversation: end]
[Shared: MathDial ground_truth / StepVerify reference_solution]
Every donut costs him $.2 because 2.4 divided by 12 equals .2. So he makes $.8 for every donut he
sells because $1 minus $.2 equals .8. He needs to sell 120 donuts because 96 divided by .8 equals
120. He needs to buy and sell 10 dozen donuts because 120 divided by 12 equals 10
[MathDial only meta data]
student_profile: ... / teacher_described_confusion: ...
self-correctness: ... / self-typical-confusion: ...
self-typical-interactions: ... / qid: ... / scenario: ...
[StepVerify only meta data]
topic: Math Word Problem / incorrect_index: ...
incorrect_step: ... / error_description: ... / student_correct_response: ...

```

Figure 4.2.: An example from the 1,932nd record in MathDial and the 969th record in StepVerify, showing shared and dataset-specific fields. Tutor tutor-move annotations in parentheses appear only in MathDial, while the student error type appears only in StepVerify. Other metadata are omitted for readability.

or repetitions of the first tutor utterance of the remaining dialogue turns. Thus, only the last three turns of each dialogue are retained. The text fields are first preprocessed into a consistent format. For each dialogue, student utterances are stored in their original order in a separate field. In addition, each tutor utterance is paired with the nearest preceding and following student utterances, when available, and stored in another field. These fields are then used to identify similar dialogue contexts between MathDial and StepVerify in the next steps.

Math Problem Matching

For each StepVerify record, MathDial records containing the same math problem as the StepVerify record are retrieved. In MathDial, multiple tutoring dialogues were collected for the same underlying math problems. Consequently, a set of MathDial records sharing the same math problem is collected for each StepVerify record.

Dialogue Matching: Student Utterance Matching

To identify which of the retrieved MathDial records most closely corresponds to each StepVerify record, the similarity scores of student utterances from StepVerify and the candidate MathDial records are calculated in the original conversation order. The average of all similarity scores of student utterances is used to select the most corresponding MathDial record; each StepVerify record is matched to a MathDial record.

As a result, all 1,002 StepVerify records are successfully matched to a MathDial dialogue. These matches represent 999 distinct MathDial records, as three matches refer to MathDial records that are also matched to other StepVerify records. All 1,002 matches are retained at this stage and further processed in the next stage.

Tutor-Utterance Matching and Filtering

Once each StepVerify record is matched to a MathDial record, each of its tutor utterances is compared with all labeled tutor utterances in the matched MathDial record. To account for the context of each tutor utterance, the nearest preceding and following student utterances are also compared between the two records, where available. The similarity score of the tutor utterance is computed as the mean of these scores, and the corresponding MathDial tutor utterance with the highest similarity score is selected as a matched tutor utterance.

To reduce noisy matches, a StepVerify record is retained only if all of its tutor utterances have similarity scores of at least 0.9. In total, 2,004 tutor utterances from 1,002 StepVerify records are compared with the corresponding tutor utterances from the matched MathDial records. These matches involve 999 distinct MathDial records, with three duplicate matches. In nine StepVerify records, both tutor utterances have scores below the threshold, while only one tutor utterance falls below the threshold in two records. Consequently, these 11 records are excluded and 991 StepVerify records are retained. Tutor-move labels are then transferred from the corresponding MathDial tutor utterances to the matched StepVerify tutor utterances.

4.2.3. Aligned Dataset

Following annotation transfer and filtering, 991 of the 1,002 StepVerify records (98.90%) are retained. A record is included only when tutor utterances from both MathDial and StepVerify dialogues satisfy a similarity threshold of 0.9. In the aligned dataset, each record consists of a student’s initial incorrect answer to a math problem, together with the corresponding error type, followed by a three-turn tutor–student–tutor interaction. Tutor-move annotations are transferred from MathDial to both tutor utterances. The resulting aligned dataset comprises 1,982 labeled tutor utterances and 991 student turns. Figure 4.3 shows an instance of the aligned dataset.

Remaining MathDial Dataset after Annotation Transfer

Of the 2,861 MathDial records, 999 unique records are selected as matching sources during annotation transfer. Given the limited availability of suitable training data with human-authored

```

"topic": "Math Word Problem",
"problem": "javier is selling donuts to raise money for a new game. he wants to raise 96. he buys each dozen donuts for 2.40 and then sells each donut for 1. how many dozen donuts does he need to buy and sell to reach his goal?",
"reference_solution": "Every donut costs him $.2 because 2.4 divided by 12 equals .2. So he makes $.8 for every donut he sells because $1 minus $.2 equals .8. He needs to sell 120 donuts because 96 divided by .8 equals 120. He needs to buy and sell 10 dozen donuts because 120 divided by 12 equals 10. 10",
"student_incorrect_solution": "'javier makes 1 - 2.40 - 1.40 loss for each dozen donuts.', 'to raise 96, he needs to sell 96 1 96 donuts.', 'he needs to buy and sell 96 12 8 dozen donuts.', 'since he makes a loss, he needs to spend 8 x 2.40 19.2 to buy the donuts.', 'so, his profit will be 96 - 19.2 76.8.', ' 8'",
"error_category": "none_of_the_above",
"error_description": null,
"dialog_history": [
{"text": "you have made an error at the start. each dozen donuts sold at $1 per donut would be how much",
"user": "teacher",
"pedagogy": "probing" },
{"text": "each dozen donuts sold at $1 per donut would be $12",
"user": "student" },
{"text": "so he had spent $2.40 per dozen and makes a profit of 9.60 per dozen do you see that",
"user": "teacher",
"pedagogy": "focus" }
],
"incorrect_index": ..., "incorrect_step": ..., "student_correct_response": ...

```

Figure 4.3.: An example record from the aligned dataset, combining student error annotations from the 969th record in StepVerify with tutor-move labels transferred from the 1,932nd record in MathDial. Only the contents of fields relevant to illustrating the annotation transfer are shown.

tutor utterances, we retain the remaining 1,862 records as training data for the supervised fine-tuning in Section 4.5.2 instead of discarding them.

Train and Test Split

For preference dataset construction and downstream pedagogical LLM evaluation, the aligned dataset of 991 records is partitioned into a training split of 693 records (70%) and a test split of 298 records (30%). This split is stratified by student error type to preserve the relative frequencies of error types across the two subsets. The training split is used to estimate error-move associations (Section 4.4.2), generate candidate tutor responses, and construct the preference dataset (Section 4.4). The test split is reserved for evaluating pedagogically aligned LLM tutors (Section 4.5).

Tutor-move classification uses labeled tutor utterances from all 991 aligned records, with a separate utterance-level 60:20:20 train-validation-test split stratified by tutor-move label (Section 4.3). The selected classifier predicts tutor-move probabilities for generated candidate responses, which are then used to compute preference scores.

These task-specific splits allow the limited annotated data to support both classifier development and preference optimization.

Student error type	Dialogues	Focus	Generic	Probing	Telling	Total
Numerical calculation error	125	63	87	93	7	250
Missing or extra quantity	238	145	196	118	17	476
Missing or incorrect factual knowledge	139	76	104	89	9	278
Misunderstanding of the question	284	147	254	152	15	568
None of the above	86	46	76	46	4	172
Reached the correct solution but proceeded further	70	32	63	38	7	140
Unit conversion error	49	27	40	27	4	98
Total	991	536	820	563	63	1,982

Table 4.2.: Distribution of student error types and tutor-move labels in the aligned dataset. Tutor-move counts include both tutor turns in each dialogue. Each dialogue has one student error type for the initial incorrect student solution.

Distributions of Tutor-Move Labels and Student Error Types

Each dialogue contributes one student error type and two tutor-move labels. Among pedagogical tutor moves, Generic is the most frequently observed tutor move, whereas Telling is the least frequent. In terms of student error types, *misunderstanding of the question* is the most common, while *unit conversion error* is the least common.

For each dialogue, the tutor move of the first tutor utterance is a response to the student’s initial incorrect solution, for which an error type is annotated. The co-occurrence of these first-turn tutor moves and the corresponding error types is treated as an empirical association between student error types and tutor moves, and is used as the basis for the error-conditioned scoring method in Section 4.4.2. The distribution of student error types and tutor-move labels over all tutor turns in the aligned dataset is shown in Table 4.2.

4.3. Tutor-Turn Pedagogy Move Classification for Preference Scoring

This section describes the tutor-move classifier that maps a tutor utterance, optionally together with preceding dialogue context, to a probability distribution over the four pedagogical tutor moves in MathDial. The classifier is not an end task in itself. It is a component that converts a candidate tutor response r into a posterior $p_{\theta}(m \mid r)$ over tutor moves $\mathcal{M} = \{\text{Focus, Generic, Probing, Telling}\}$. These posteriors later play a role as the move probabilities in the expected-utility calculation in Section 4.4.2, where they are combined with student error-tutor move association to assign a scalar score to each LLM-generated tutor utterance. While this association is computed deterministically from the co-occurrence of student error types and pedagogical tutor moves observed in the training split of the aligned dataset, the reliability of these posteriors depends on the performance of the classifier.

However, due to the challenging nature of obtaining tutoring data, tutoring datasets often

suffer from limited training data and class imbalance (J. Lin et al., 2023; Macina, Daheim, Pal Chowdhury, et al., 2023). The aligned dataset exhibits the same constraints. It contains only 991 records and is highly imbalanced across tutor-move labels. To examine how these constraints affect tutor-move classification, we compare two encoder models (RoBERTa-base and a domain-adapted variant of the same architecture) under three input representations and three objective functions. Each trained model is evaluated based on the chosen metrics and the best-performing model is used in the subsequent downstream tasks.

4.3.1. Task Formulation and Input Representation

This section formulates tutor-move classification as a sequence-classification problem over tutoring dialogues, describes the dataset used for the classification task, and introduces the input representations used by the classifiers.

Task Formulation

Let $\mathcal{M} = \{\text{Focus, Generic, Probing, Telling}\}$ denote the tutor moves. A tutoring dialogue for a given math problem context $\mathcal{C}$ is a sequence of utterances:

$$H = (u_1, \dots, u_N)$$

where each u_j is either a tutor or student utterance, and N denotes the length of the dialogue. In the aligned dataset, N is fixed to 3, and each dialogue follows a fixed turn order: tutor–student–tutor.

Let r denote a target tutor utterance, with ground-truth tutor-move label $y \in \mathcal{M}$. The classification objective is to predict y . The input to a classifier is a sequence x that serializes r with $k \geq 0$ preceding utterances:

$$x = \begin{cases} r, & \text{if } k = 0, \\ u_{N-k} \oplus \dots \oplus u_{N-1} \oplus r, & \text{if } k \geq 1, \end{cases}$$

where $\oplus$ denotes concatenation. When $k = 0$, every labeled tutor utterance is used as a target. When $k \geq 1$, the target is always the last tutor utterance, so $r := u_N$. Let $z_\theta(x) \in \mathbb{R}^{|\mathcal{M}|}$ denote the logit vector produced from x . A classifier f_θ maps these logits to a posterior over tutor moves $m \in \mathcal{M}$ by

$$p_\theta(m \mid x) := f_\theta(x) = \text{softmax}(z_\theta(x)).$$

The final prediction

$$\hat{y} = \arg \max_{m \in \mathcal{M}} p_\theta(m \mid x)$$

is used for classifier evaluation.

Training Dataset

To train and evaluate a classifier, the data are typically divided into training, validation, and test sets. The training set is used to learn the model parameters, the validation set to tune hyperparameters and select the best model, and the test set to evaluate the selected model’s performance on unseen data.

The aligned dataset contains 1,982 labeled tutor utterances, of which only 63 are labeled as Telling. Restricting classification to the training split defined in Section 4.2.3 would shrink this sample further and would risk leaving the minority class too sparsely represented in each split. We therefore use labeled tutor utterances from all 991 records and adopt a classification-task-specific split in order to use every available data point. Because each record contains two tutor-move labels, this split is applied to labeled utterances rather than to records. The utterances are partitioned into training, validation, and test sets in a 60:20:20 ratio, stratified by tutor-move label. The final counts for each split vary depending on the input representation. Thus, they are reported at the end of this section.

The rationale behind using the classification-task-specific split is that the classifier learns from existing labeled tutor utterances and is later applied to unseen tutor responses. As later described in Section 4.4.1, these tutor responses are generated from the existing context in the aligned dataset—the math problem, the student’s initial incorrect solution, and the reference solution (see Figure 4.3). The goal of the classification task is to assign probabilities to predicted labels, which are then used to score the generated responses and construct preference pairs. Thus, there are no overlapping tutor responses between the classifier’s training dataset and the classifier’s target tutor utterances used for preference construction.

Text Normalization

Text normalization converts raw utterances into a consistent format before they are passed to the classifier. Without this step, two utterances that express the same instructional intent may still look different to the model. Text normalization is therefore applied so that the classifier can attend to the consistent formats of input sequences. Sequences of raw utterances are lowercased, and URLs, HTML, emoji, and non-alphanumeric characters other than basic punctuation are removed. Utterances that are empty after this procedure are discarded. The same normalization is applied not only to the aligned dataset of 991 records but also to the domain corpus used for domain-adaptive pretraining in the subsequent section.

Input Representations

In tutoring dialogues, the same expression can have different meanings depending on the context. For example, Buckley and Wolska (2008) show that short responses such as “Really?” or “Yes?” may appear to be information requests, but within a given context they can instead encourage the student to continue. Similarly, Macina, Daheim, Pal Chowdhury, et al. (2023) note that the tutor moves Probing and Focus are particularly difficult to distinguish, as the boundary between the two can be subjective. These observations suggest that distinguishing tutor moves may require information beyond the tutor utterance itself.

Prior work on mathematics classroom discourse has shown that incorporating preceding and following utterances alongside the target teacher utterance improves transformer-based talk-move classification (Suresh, Jacobs, Perkoff, et al., 2022). Related work on tutoring dialogue has reported that adding the immediately preceding student turn improved classification compared with using the tutor response alone, whereas additional context did not lead to a further improvement (He & Xu, 2026). Suresh, Jacobs, Perkoff, et al. (2022) further showed that inserting separator tokens (i.e., `</s>` in RoBERTa) between consecutive utterances helps the model incorporate local dialogue context.

Explicit speaker markers, such as `T:` or `S:`, have also been found to help models distinguish student utterances from tutor utterances (Cao et al., 2025; Kim & Vossen, 2021). Longer dialogue context, however, does not necessarily improve classification performance, as the usefulness of additional context depends on the task and the dataset (Ganesh et al., 2021; Ikram et al., 2025; Suresh, Jacobs, Perkoff, et al., 2022). In this thesis, we therefore compare different context lengths empirically.

Following Cao et al. (2025) and Suresh, Jacobs, Perkoff, et al. (2022), we prepend `T:` or `S:` to each utterance to indicate whether it was produced by a tutor or a student, respectively, and insert the RoBERTa separator token `</s>` between utterances when more than one utterance is included in the input sequence. In the aligned dataset, every tutor–student dialogue consists of three turns in a fixed order: tutor–student–tutor, and both tutor turns have a tutor-move label. When $k = 0$, each labeled tutor utterance is classified without dialogue context. When $k \geq 1$, the target is always the last tutor turn. We therefore compare the following three input representations and report the total number of examples for each:

- *Tutor-turn only* ($k = 0$).

$$x = r = [\text{tutor utterance}]$$

The input consists of a single normalized tutor utterance without preceding context or a speaker prefix. Both labeled tutor utterances in each dialogue are used as separate inputs (1,982 examples).

- *Preceding-1* ($k = 1$).

$$x = \text{spk}_{\text{student}}(u_1) \oplus \text{spk}_{\text{tutor}}(u_2 = r)$$

$$= \text{S: } [\text{student utterance}] \text{ </s> T: } [\text{tutor utterance}]$$

The input consists of the immediately preceding student utterance (u_1) followed by the last tutor utterance (u_2), each with a speaker prefix. The model predicts the tutor-move label of the last tutor utterance (991 examples).

- *Preceding-2* ($k = 2$).

4.3. Tutor-Turn Pedagogy Move Classification for Preference Scoring

Input representation	Split	Focus	Generic	Probing	Telling	Total
Tutor-turn only	Train	322	492	337	37	1,188
	Validation	107	164	113	13	397
	Test	107	164	113	13	397
	Total	536	820	563	63	1,982
Preceding-1 and Preceding-2	Train	279	43	244	28	594
	Validation	93	14	81	10	198
	Test	93	15	82	9	199
	Total	465	72	407	47	991

Table 4.3.: Train-validation-test counts by tutor-move label. Tutor-turn only includes both labeled tutor utterances. Preceding-1 and Preceding-2 use only last-turn labels and therefore share the same stratified split.

$$x = \text{spk}_{\text{tutor}}(u_0) \oplus \text{spk}_{\text{student}}(u_1) \oplus \text{spk}_{\text{tutor}}(u_2 = r)$$

= T: [tutor utterance] </s> S: [student utterance] </s> T: [tutor utterance]

The input consists of the full three-turn dialogue in its original order: the first tutor utterance (u_0), the student utterance (u_1), and the last tutor utterance (u_2). Each utterance has a speaker prefix, and the model predicts the tutor-move label of the last tutor utterance (991 examples).

Train-Validation-Test Split per Input Representation

For each input representation, the labeled examples are partitioned into training, validation, and test sets in a 60:20:20 ratio. The split is stratified by tutor-move label and uses a fixed random seed of 42. Tutor-turn only comprises 1,188 training, 397 validation, and 397 test examples. Preceding-1 and Preceding-2 comprise 594 training, 198 validation, and 199 test examples. The corresponding counts by tutor-move label are shown in Table 4.3.

4.3.2. Base Model and Domain-Adaptive Pretraining

In the tutor-move classification task, we use RoBERTa as the backbone of the classifier. However, there is a domain mismatch between our tutoring data and the data used to pretrain RoBERTa. Domain-adaptive pretraining (DAPT) addresses this issue through continued masked language modeling on in-domain data. Gururangan et al. (2020) show that this approach can improve downstream classification performance in both high- and low-resource settings. Similar approaches have been reported to be beneficial in educational settings. For example, MathBERT improves performance on mathematics education tasks through further pretraining on mathematical text (Shen et al., 2021), while EduBERT improves student forum post classification through further pretraining on educational forum data (Clavié & Gal, 2019).

The aligned dataset contains 991 records, with 991 labeled examples for each of the Preceding-

1 and Preceding-2 representations and 1,982 labeled examples for the Tutor-turn only representation before splitting (see Table 4.3). This limited amount of labeled data provides an additional motivation for DAPT, which allows the encoder to learn from additional domain-relevant text. We therefore compare the original RoBERTa encoder with a domain-adapted version to examine whether continued pretraining improves tutor-move classification.

RoBERTa

All classifiers in this thesis use RoBERTa-base (Y. Liu et al., 2019) as the backbone. RoBERTa is a Transformer encoder pretrained with masked language modeling on a large general-domain corpus. Encoder-only models of this family are a common choice for utterance-level talk-move classification in mathematics tutoring and classroom discourse (Cao et al., 2025; Suresh, Jacobs, Perkoff, et al., 2022). Given an input sequence x constructed in Section 4.3.1, the model encodes x and a linear classification head is applied to the $\langle s \rangle$ representation. The head produces four logits over $\mathcal{M} = \{\text{Focus, Generic, Probing, Telling}\}$, which a softmax converts to the posterior $p_{\theta}(m \mid x)$. This posterior is later used as the move-probability term of the expected-utility score in Section 4.4.2. The same architecture is used with both the original RoBERTa-base encoder and the domain-adapted encoder to train a classifier.

Domain Corpus

We construct the domain corpus from Question-Anchored Tutoring Dialogues (QATD2k) and MathMentorDB, both of which consist of online human tutoring dialogues (Ion, 2026; Zent et al., 2025). QATD2k provides 68,717 messages from question-centered, one-to-one tutoring on the British learning platform called Eedi, whereas MathMentorDB contains 5,452,947 messages collected from interactions between volunteer tutors and students on a Discord mathematics help channel. We use these two datasets as complementary domain corpora to adapt the base RoBERTa-base model to the mathematics tutoring domain.

QATD2k

QATD2k (Zent et al., 2025) consists of one-to-one human tutoring sessions between students and expert tutors, collected on Eedi, a UK-based online mathematics platform used in schools. A session begins when a student requests help from an on-demand expert tutor while working on a teacher-assigned lesson, typically a diagnostic multiple-choice question with one correct answer and three misconception-based distractors. The students are predominantly based in the UK, with a reported average age of 12 years. Tutor messages were automatically assigned one of seven labels, including *None*, using the GPT-based talk-move classifier from Moreau-Pernet et al. (2024). However, the authors acknowledge limitations in the automatically assigned talk-move labels. The public release contains 1,971 dialogues, 68,717 messages (22,468 student turns and 23,781 tutor turns after grouping consecutive messages by the same speaker), and data from 1,073 students. The public release of the dataset is available on Hugging Face ⁵. Figure 4.4 shows a complete one-to-one tutoring session from QATD2k.

⁵<https://huggingface.co/datasets/Eedi/Question-Anchored-Tutoring-Dialogues-2k>

```

QATD2k (InterventionId 42; 20 messages)
Tutor: (Keep Together) Hi How can I help?
Student: I'm a bit confused with this question
Tutor: (None) Ok, let's take a look together!
Tutor: (Press for Accuracy) So, what is  $p \times -q$ ?
Student: Um well I know it would be a negative but I don't really know the answer
Tutor: (Revoicing) That's what we need really, so it would be  $-pq$ 
Tutor: (None) As if we multiply two letters we write them next to each other
Student: Ok
Tutor: (Keep Together) Does  $-p \times q$  give us the same answer?
Student: Yes
Tutor: (None) Great, so statement 1 is true
Student: Ok
Tutor: (None) Now statement 2
Student: I don't think it works
Tutor: (Revoicing) I don't think it does either, as the left one stays negative, but the right one is positive
Tutor: (Press for Accuracy) So which answer do you think we need?
Student: A
Tutor: (None) I agree!
Tutor: (None) Well done Sophia :)
Student: Ok thank you

```

Figure 4.4.: An example of a complete tutoring session from QATD2k (Zent et al., 2025). Tutor messages are shown with their talk-move labels in parentheses.

MathMentorDB

MathMentorDB (Ion, 2026) consists of mathematics tutoring dialogues collected from public help channels of the Mathematics Discord Server, a global community in which students post questions and volunteer peer mentors reply in real time. A conversation begins when a student occupies an available help channel by posting a question, often with a screenshot. It is closed by a bot-generated channel-closed marker, triggered when a participant types `.close` or the channel remains idle for 15 minutes. Unlike QATD2k, the helpers are volunteer peers rather than professional tutors, and 43.6% of conversations involve more than two participants. Usernames and user identifiers are replaced with consistent pseudonyms. The mathematical range is broader than QATD2k or MathDial, spanning high-school precalculus to graduate-level topics, and the chat is informal, with abbreviations, emoji, LaTeX, and images. The public release contains 5,452,947 messages in 200,332 conversations from 43,249 users (36,682 students and 22,554 tutors), with a mean of 27.2 messages per conversation (median 13). The dataset is available on Hugging Face.⁶ Figure 4.5 shows a complete question-answering multi-party conversation.

Processing the Domain Corpus

We use 55,322 messages across 1,576 dialogues from QATD2k and 5,452,947 messages in 200,332 dialogues from MathMentorDB. We first apply the same text normalization used for the aligned dataset. For MathMentorDB, messages containing `.close` are further removed. Messages belonging to the same dialogue are grouped by conversation identifier. Within each dialogue,

⁶https://huggingface.co/datasets/mikeion/mathconverse_pseudonyms

```

MathMentorDB (ID 130153; 4 participants; 13 messages)
2022-11-20 14:37:51 UTC Student: [attaches image of a proof problem]
2022-11-20 14:37:52 UTC Student: this is weird
2022-11-20 14:38:16 UTC Student: can some explain what are the two steps?
2022-11-20 14:40:35 UTC Helper1: A derivative is (and I'm sure some smartie will come and correct me
    with their "ackthwally...", but) the analysis of instantaneous rate of change of a function.
2022-11-20 14:40:59 UTC Helper1: With that being, an instantaneous rate of change of 0 means there is
    no change at all.
2022-11-20 14:41:47 UTC Helper1: And  $F(x_0) = 0$  just says that that function intersects the  $x$  axis
    somewhere.
2022-11-20 14:44:10 UTC Helper2: eh yeh  $f(x) = 0$  is just the horizontal line at  $y = 0$ 
2022-11-20 14:44:27 UTC Helper2: step 2 is odd
2022-11-20 14:45:36 UTC Helper3: in step 1 you show that  $F$  is constant, in step 2 you show that  $F$  is
    in fact the constant 0 and not any other constant.
2022-11-20 14:46:19 UTC Helper2: ic ann
2022-11-20 14:46:19 UTC Helper2: thx
2022-11-20 14:46:30 UTC Helper2: can u help 14
2022-11-20 14:46:44 UTC Student: .close

```

Figure 4.5.: An example of a complete question-answering session from MathMentorDB (Ion, 2026). The student shares a screenshot of a proof problem, three volunteer helpers offer complementary explanations, and the student then closes the channel with .close. Speaker labels follow the source transcript.

consecutive messages with the same tutor or student role are concatenated into a single turn; consecutive contributions from different MathMentorDB helpers are merged into a single tutor turn. To keep the format of the domain corpus consistent with the aligned dataset, if a dialogue ends with a student turn, the final student turn is removed. We then retain dialogues that contain at least three turns, in which tutor and student turns alternate and the last turn is a tutor turn. The resulting domain corpus comprises 117,171 dialogues, of which 1,576 originate from QATD2k (36,289 turns) and 115,595 from MathMentorDB (2,054,622 turns).

This corpus is then rendered in the same input representations used for the aligned dataset, using only the utterances. Preceding-1 consists of the last tutor utterance together with the immediately preceding student turn, and Preceding-2 consists of the last tutor turn together with the two preceding tutor–student turns, yielding 117,171 sequences for each of these representations. Tutor-turn only keeps all individual tutor utterances and yields 944,641 sequences.

To illustrate how a source dialogue is transformed, Figure 4.4 is shown below after rendering into the three input representations:

- Preceding-2 ($k = 2$).
 $x = T$: i don't think it does either, as the left one stays negative,
 but the right one is positive so which answer do you think we need?
 $\text{</s> } S$: a $\text{</s> } T$: i agree! well done sophia
- Preceding-1 ($k = 1$).
 $x = S$: a $\text{</s> } T$: i agree! well done sophia
- Tutor-turn only ($k = 0$).

$x = i$ don't think it does either, as the left one stays negative,
but the right one is positive so which answer do you think we need?
 $x = i$ agree! well done sophia

Domain-Adaptive Pretraining Configurations

For each input representation, RoBERTa-base is further pretrained as a masked language model on the corresponding corpus, following the domain-adaptive pretraining approach of Gururangan et al. (2020). Tokens are masked independently with probability 0.15. Training uses the AdamW optimizer for three epochs with a learning rate of 4×10^{-5} , a warmup ratio of 0.06, weight decay of 0.01, and a batch size of 32, in mixed precision. The corpus is split 99:1 into training and validation sequences, and the model with the lowest validation masked-language-modeling loss is retained. The resulting encoder is then used to initialize the corresponding classification model.

4.3.3. Loss Functions and Training Configurations

As shown in Table 4.3, the classifier training data are highly imbalanced across tutor-move labels. In the Tutor-turn only representation, only 37 of the 1,188 training examples are labeled as Telling. In each of the Preceding-1 and Preceding-2 representations, Telling and Generic account for just 28 and 43 of the 594 training examples, respectively.

The error-conditioned scoring method described later in Section 4.4.2 uses the classifier's predicted probabilities over tutor moves to score candidate tutor responses. Poor classifier performance on tutor-move prediction may affect the reliability of these scores and, consequently, the selection of preferred and rejected responses. In this section, we describe different loss functions used for classifiers to address the class imbalance, together with the training configurations associated with each objective.

Loss Functions and Class Imbalance

In this thesis, we compare three loss functions to mitigate the adverse effects of class imbalance: weighted cross-entropy, focal loss, and AUC maximization via a surrogate loss. Weighted cross-entropy serves as the baseline against which the other loss functions are compared.

We use the following notation in this section: $z_{i,c}$ denotes the logit for class c on example i , y_i denotes the true label, and $p_{i,c}$ denotes the softmax probability.

- *Weighted Cross Entropy Loss*

While cross-entropy loss assigns smaller losses to samples with a high predicted probability for the true class, these small losses can accumulate over samples from the majority class. In class imbalance settings, the cumulative contribution may dominate the total loss (T.-Y. Lin et al., 2017). A common method for addressing class imbalance is to add a weighting factor to the cross entropy that computes weights inversely proportional to the frequency of the true labels, so that rare moves contribute more to the average loss; this method is known as weighted cross-entropy loss (WCE). In educational settings,

Ganesh et al. (2021) showed that WCE mitigated the effects of class imbalance in the future talk-move prediction for classroom discourse.

The weighting factor is defined as

$$w_c = \frac{N_{\text{train}}}{C \cdot n_c},$$

where N_{train} is the number of training examples, $C = 4$ is the number of tutor moves, and n_c is the number of training examples of class c .

$$\ell_i^{\text{WCE}} = -w_{y_i} \log p_{i,y_i} = -w_{y_i} \log \left(\frac{\exp(z_{i,y_i})}{\sum_{j=1}^C \exp(z_{i,j})} \right).$$

We use PyTorch to implement this loss, and the batch loss in PyTorch is computed as a weight-normalized mean of ℓ_i^{WCE} :

$$L_{\text{Batch}}^{\text{WCE}} = \frac{\sum_{i=1}^B \ell_i^{\text{WCE}}}{\sum_{i=1}^B w_{y_i}} = -\frac{\sum_{i=1}^B w_{y_i} \log p_{i,y_i}}{\sum_{i=1}^B w_{y_i}},$$

where B is the batch size.

- *Focal Loss*

While weighted cross-entropy loss may alleviate the adverse effect of the majority class on parameter optimization, the nature of cross-entropy does not differentiate easy and difficult samples. T.-Y. Lin et al. (2017) introduced focal loss which adds a *down weighting* term $(1 - p_i)^\gamma$ with γ being a *focusing parameter*. When $\gamma \geq 1$, easy examples are down weighted, reducing their impact on the loss function. In class imbalance settings, focal loss is often compared with WCE, with some studies reporting improvements over WCE (Cao et al., 2019). Focal loss is defined as follows:

$$\ell_i^{\text{Focal}} = -(1 - p_{i,y_i})^\gamma \log p_{i,y_i} = -\left(1 - \frac{\exp(z_{i,y_i})}{\sum_{j=1}^C \exp(z_{i,j})}\right)^\gamma \log \left(\frac{\exp(z_{i,y_i})}{\sum_{j=1}^C \exp(z_{i,j})} \right)$$

Following T.-Y. Lin et al. (2017), we use $\gamma=2$. Analogously to the weighted cross-entropy loss, the weighted version of focal loss is defined as

$$L_{\text{Batch}}^{\text{Balanced Focal}} = -\frac{\sum_{i=1}^B \alpha_{y_i} (1 - p_{i,y_i})^\gamma \log p_{i,y_i}}{\sum_{i=1}^B \alpha_{y_i}}$$

where B is the batch size and $\alpha = w$; the authors call this form α -balanced variant. We use the α -balanced variant of focal loss.

- *AUC Maximization*

Unlike cross-entropy and focal loss, which compute losses for individual examples, AUC

maximization aims to rank positive examples above negative examples. In educational dialogue act classification, J. Lin et al. (2023) showed that an AUC maximization approach improved performance over WCE under low-resource and imbalanced conditions.

Directly optimizing AUC is difficult due to its non-differentiable nature. Thus, we use a pairwise AUC square loss as a surrogate loss (J. Lin et al., 2023; Yuan et al., 2021). For each class within a batch, we apply the sigmoid function $\sigma(z) = 1/(1 + \exp(-z))$ to each class logit and compare positive and negative scores in the one-vs-rest fashion.

Let P_c and N_c denote the batch indices with $y_i = c$ and $y_i \neq c$, respectively, and let $\mathcal{A} = \{c \mid |P_c| > 0, |N_c| > 0\}$ denote the set of classes for which the batch contains at least one positive and one negative example.

For each class $c \in \mathcal{A}$, the per-class loss within a batch is computed as follows:

$$L_c^{\text{AUC}} = \frac{1}{|P_c||N_c|} \sum_{i \in P_c} \sum_{j \in N_c} [\delta - \sigma(z_{i,c}) + \sigma(z_{j,c})]_+^2, \quad c \in \mathcal{A},$$

where δ is the margin and $[*]_+ = \max(*, 0)$. The margin δ specifies the minimum desired separation between positive and negative scores as the loss stays active only when the score difference between the positive and negative samples is less than δ . Following J. Lin et al. (2023), we set $\delta = 1$.

For each batch with $|\mathcal{A}| > 0$, we compute the batch loss as the sum of the per-class losses over the active classes in $\mathcal{A}$:

$$L_{\text{Batch}}^{\text{AUC}} = \sum_{c \in \mathcal{A}} L_c^{\text{AUC}} = \sum_{c \in \mathcal{A}} \frac{1}{|P_c||N_c|} \sum_{i \in P_c} \sum_{j \in N_c} [\delta - \sigma(z_{i,c}) + \sigma(z_{j,c})]_+^2,$$

Training Configurations

All classifiers are trained in Google Colab with GPU acceleration (NVIDIA RTX PRO 6000 Blackwell Server Edition or NVIDIA A100 GPUs) using the AdamW optimizer for 10 epochs, with a warmup ratio of 0.10, weight decay of 0.01, and random seed 42. The configuration for each combination of input representation and learning objective is shared by the base and domain-adapted models, as summarized in Table 4.4.

Batch sizes are adjusted for training-set size and learning objective. Within each learning objective, the Tutor-turn only representation uses twice the batch size of Preceding-1 and Preceding-2 representations to account for its twice as large training set so that the number of updates per epoch is approximately the same across input representations. For each input representation, AUC maximization uses twice the batch size of WCE and focal loss to increase the probability that both positive and negative examples for each pedagogical tutor move are included in each batch.

For models trained with WCE or focal loss, we use macro-F1 to monitor tutor-move predictions. For models trained with an AUC maximization approach, we use macro one-vs-rest ROC-AUC to monitor class discrimination across decision thresholds.

4.3. Tutor-Turn Pedagogy Move Classification for Preference Scoring

Input Representation	Learning Objective	Learning Rate	Batch Size	Epoch	Validation Metric
Preceding-1/-2	WCE, focal loss	2×10^{-5}	8	10	macro-F1
Tutor-turn only	WCE, focal loss	1×10^{-4}	16	10	macro-F1
Preceding-1/-2	AUC maximization	2×10^{-5}	16	10	OvR macro-AUC
Tutor-turn only	AUC maximization	2×10^{-5}	32	10	OvR macro-AUC

Table 4.4.: Classifier training configurations. All classifiers are trained using the AdamW optimizer, with a warmup ratio of 0.10, weight decay of 0.01, and a random seed of 42. OvR AUC denotes macro one-vs-rest ROC-AUC.

Learning rates are determined empirically by using $\{1e-6, 2e-6, 1e-5, 2e-5, 1e-4, 2e-4\}$ in ascending order. For WCE and focal loss, a learning rate of $2e-5$ is used for Preceding-1 and Preceding-2 representations, and $1e-4$ is used for the Tutor-turn only representation. For AUC maximization, $2e-5$ is used for all input representations.

4.3.4. Evaluation and Model Selection

The experimental grid comprises 18 classifiers: two encoder models (RoBERTa-base and its domain-adapted variant), three input representations (Tutor-turn only, Preceding-1, and Preceding-2), and three learning objectives (weighted cross-entropy, focal loss, and AUC maximization). This section describes the final model selection criteria and how the final classifier is used as an interface in the error-conditioned preference scoring method in Section 4.4.2.

Evaluation Metrics and Comparative Analysis

J. Lin et al. (2023) and Nguyen et al. (2023) show that optimizing models with an AUC-maximization objective may improve F1 in low-resource and class-imbalance settings, and may increase recall without compromising precision. We therefore use macro-F1 and macro one-vs-rest ROC-AUC as our primary evaluation metrics. Macro-F1 serves as a criterion for comparing the correctness of discrete tutor-move predictions, while macro one-vs-rest ROC-AUC provides a threshold-agnostic measure of class separability. We additionally report the multiclass Matthews correlation coefficient (MCC) as an auxiliary measure of overall agreement between predicted and ground-truth tutor-move labels.

- *Macro-F1*

We obtain the predicted tutor-move label using $\hat{y}_i = \arg \max_m p_\theta(m \mid x_i)$. Macro-F1 is the unweighted mean of the class-specific F1 scores:

$$F1_{\text{macro}} = \frac{1}{C} \sum_{m=1}^C \frac{2 TP_m}{2 TP_m + FP_m + FN_m},$$

where $C = 4$ and TP_m , FP_m , and FN_m are the true positives, false positives, and false negatives for tutor move m .

- *Macro One-vs-Rest ROC-AUC*

For each tutor move m , we treat examples whose true label is m as positive and all remaining examples as negative, using $p_\theta(m | x_i)$ as the prediction score. The class-specific the one-vs-rest AUC is computed with scikit-learn’s trapezoidal integration over the ROC curve. This AUC is equivalent to the normalized Mann–Whitney U statistic (Hanley & McNeil, 1982). Therefore, one-vs-rest ROC-AUC score measures how well positive samples are ranked above negative samples for class m . It can be expressed as:

$$\text{AUC}_m^{\text{OvR}} = \frac{U_m}{n_m^+ n_m^-} = \frac{1}{n_m^+ n_m^-} \sum_{i: y_i = m} \sum_{j: y_j \neq m} \left(\mathbf{1}[s_{i,m} > s_{j,m}] + \frac{1}{2} \mathbf{1}[s_{i,m} = s_{j,m}] \right),$$

where $s_{i,m} = p_\theta(m | x_i)$, n_m^+ and n_m^- are the numbers of positive and negative examples for class m , and $\mathbf{1}[\cdot]$ denotes the indicator function, which equals 1 if its argument is true and 0 otherwise. Analogously to macro-F1, we report the unweighted mean across the C tutor moves:

$$\text{AUC}_{\text{macro}}^{\text{OvR}} = \frac{1}{C} \sum_{m=1}^C \text{AUC}_m^{\text{OvR}}.$$

- *Matthews Correlation Coefficient*

Using the same predicted labels $\hat{y}_i$ as for macro-F1, we report the multiclass Matthews correlation coefficient. Following scikit-learn’s implementation of the generalization in Gorodkin (2004),

$$\text{MCC} = \frac{N \sum_{m=1}^C \text{TP}_m - \sum_{m=1}^C (\text{TP}_m + \text{FP}_m)(\text{TP}_m + \text{FN}_m)}{\sqrt{\left[N^2 - \sum_{m=1}^C (\text{TP}_m + \text{FN}_m)^2 \right] \left[N^2 - \sum_{m=1}^C (\text{TP}_m + \text{FP}_m)^2 \right]}},$$

where N is the number of evaluated examples, and TP_m , FP_m , and FN_m are defined as for macro-F1. For each class m , true-negative counts are determined by $\text{TN}_m = N - \text{TP}_m - \text{FP}_m - \text{FN}_m$. Thus, MCC provides a complementary evaluation by accounting for the entire confusion matrix. Equivalently, multiclass MCC can be written as the correlation between the true and predicted labels (Jurman et al., 2012).

Following J. Lin et al. (2023), we further compute Spearman’s rank correlation coefficient between macro-F1 and macro one-vs-rest ROC-AUC across the 18 classifier configurations to quantify the rank-based association between the two metrics. Spearman’s rank correlation is a nonparametric measure equivalent to the Pearson correlation between the ranked values of the two variables. Thus, in our analysis,

$$r_s = \text{corr}\left(\text{rank}(\text{F1}_{\text{macro}}), \text{rank}(\text{AUC}_{\text{macro}}^{\text{OvR}})\right),$$

Spearman’s rank correlation ranges from -1 to 1 . A value of 1 indicates identical rankings, 0 indicates no rank-based association, and -1 indicates exactly reversed rankings; thus, values closer to 1 indicate greater agreement between the orderings produced by the two metrics.

Downstream Input Representation

The input representation of the classifier selected according to the criteria above is retained throughout the downstream pipeline. Candidate responses are formatted according to the selected representation before classifier inference. The same response unit is then used to construct the chosen and rejected responses in the preference dataset and the target outputs in the supervised fine-tuning and DPO datasets. As reported in Section 5.1.1, the selected classifier uses the Tutor-turn only representation. Consequently, all downstream targets consist of a single tutor response.

4.4. Preference Dataset Construction

This thesis aims to align LLMs with the pedagogical value inferred from observed associations between student errors and tutor moves. We express this objective through a preference dataset in which each example consists of a dialogue context and a pair of tutor responses. Within each pair, the response that better satisfies the target tutor move is selected as the chosen response, while the other is designated as the rejected response.

Previous work suggests that constructing preference pairs that encode pedagogical signals, and aligning LLMs on those pairs, may improve the pedagogical behavior of LLMs. For example, Petukhova et al. (2026) and Sonkar et al. (2024) constructed preference pairs of tutor responses based on predefined pedagogical rubrics and reported that LLMs trained on such datasets showed improved pedagogical behavior, such as error identification or guidance provision. Additionally, Scarlatos, Liu, et al. (2025) combined pedagogical rubric scores of generated tutor responses with student outcome predictions to construct a preference dataset, and showed that LLMs preference-optimized on this dataset generated tutor responses that elicited higher predicted next-turn student correctness.

Building on this line of work, our aligned dataset provides an empirical basis for such alignment by capturing how tutors respond to different student errors. However, these observations do not directly tell us which tutor responses are better suited to a given context. We therefore translate the tutor moves associated with these error types into a preference signal, thereby addressing Research Question 1.

In this section, we first describe how we generate candidate tutor responses from the 693 contexts of the aligned dataset. We then define an error-conditioned scoring method that encodes these associations into a single scalar via expected utility. Finally, we describe how the resulting scores are used to rank candidates and construct the preference dataset. The preference dataset constructed in this section is used for Direct Preference Optimization (DPO) in Section 4.5.3.

4.4.1. Candidate Tutor Response Generation

Preference data quality and selection can substantially affect the effectiveness of DPO. Prior work suggests that preference pairs with larger quality gaps can provide clearer learning sig-

nals, and that margin-based data selection can help mitigate the effects of noisy preferences (Deng et al., 2025; Pattnaik et al., 2024). However, the absolute quality of preferred responses remains important and can have a greater influence on DPO performance than the preference gap itself (Pan et al., 2025). Motivated by these findings, we generate multiple candidate tutor responses for each dialogue context using a heterogeneous set of LLMs to construct a pool of responses with varying quality.

Generation Models and Configurations

To ensure a diverse range of response quality and style, we generate candidate tutor responses with six LLMs. Model selection follows two dimensions: (1) proprietary versus open-weight local models, and (2) the number of parameters among open-weight models. On the proprietary side, we use `gpt-5.6-terra`, the second most powerful OpenAI model available at the time of the experiments, together with `gemini-3.5-flash-lite`, a lightweight model with LearnLM capabilities integrated (LearnLM Team, 2025). On the open-weight side, we use `mistral-nemo:12b`, `llama3.1:8b`, `llama3.2:3b`, and `llama3.2:1b`, ranging from mid-sized to very small models. The temperature is set to 1.0 for all models.

Two-Stage Strategy-and-Intention Prompt

We adopt the Bridge framework, which uses cognitive task analysis to model expert decision-making when responding to errors made by first- to fifth-grade students during mathematics tutoring sessions (R. Wang et al., 2024). The authors identified three steps in experts’ reasoning: (1) inferring the student error e , (2) selecting a remediation strategy z_{what} , and (3) identifying the intention z_{why} behind that strategy. The strategy specifies how the teacher intervenes, whereas the intention captures the purpose of that intervention. Given a conversation history c_h , they formalize expert tutor responses c_r^* as being generated from the following computational model:

$$c_r^* \sim p(c_r \mid c_h, \underbrace{e}_{\text{Step (1)}}, \underbrace{z_{\text{what}}}_{\text{Step (2)}}, \underbrace{z_{\text{why}}}_{\text{Step (3)}}),$$

To generate more pedagogically sound candidate tutor responses, we follow this framework with a two-stage prompt design. Student error types are already annotated in the aligned dataset, so we omit Bridge’s first step of inferring the student error and retain only strategy selection and intention identification. We use Bridge’s taxonomy of remediation strategies and intentions, which is included in the Stage 1 prompt. Both stages receive the lesson topic, the math problem, and the student’s initial incorrect solution for each dialogue context in the training split of the aligned dataset. Additionally, Stage 1 receives the reference solution.

In Stage 1, `gpt-5.6-terra` is prompted to select one strategy and one corresponding intention for each dialogue context in the training split of the aligned dataset. In Stage 2, these six LLMs (`gpt-5.6-terra`, `gemini-3.5-flash-lite`, `mistral-nemo:12b`, `llama3.1:8b`, `llama3.2:3b`, and `llama3.2:1b`) generate candidate tutor responses for each dialogue context conditioned on the chosen strategy and intention. In this stage, these six models do not receive

4.4. Preference Dataset Construction

(a) Stage 1: strategy-and-intention selection

```
[ fontupper=] System
you are an experienced elementary math teacher. your take is to read a student's mistake. your task is to read the
student mistake. you should then determine what strategy you want to use to remediate the student's error.

User
we have a list of common strategies and intentions that teachers use, which you can pick from. we also give you the
option to write in your own strategy and intention if none of the options apply.

strategies:                                intentions:
0. explain a concept                        0. motivate the student
1. ask a question                          1. get the student to elaborate their answer
2. provide a hint                          2. correct the student's mistake
3. provide a strategy                      3. hint at the student's mistake
4. provide a worked example                4. clarify a student's misunderstanding
5. provide a minor correction              5. help the student understand the lesson topic or
6. provide a similar problem               solution strategy
7. simplify the question                   6. diagnose the student's mistake
8. affirm the correct answer                7. support the student in their thinking or
9. encourage the student                   problem-solving
10. other (specify in your reasoning)       8. explain the student's mistake (example: what is wrong
                                           in their answer or why is it incorrect)
                                           9. signal to the student that they have solved or not
                                           solved the problem
                                           10. other (specify in your reasoning)

the problem in {topic} you are student is solving is : {problem}
the reference solution is: {reference_solution}
the student made a following mistake in solving the problem: {student_mistake}
how would you remediate the student's error? pick the option number from the list of strategies and intentions.
```

(b) Stage 2: tutor-response generation

```
[ fontupper=] System
You are an experienced elementary mathematics tutor.
Your role is not merely to correct the student's mistake.
Your broader goal is to help the student remain curious, confident, and capable of thinking for themselves.

User
The lesson topic : {topic}
The problem you are student is solving is : {problem}
The student made a following mistake in solving the problem: {student_mistake}
The commonly used strategies and intentions are as follows:
- strategy: {strategy}
- intention: {intention}
Your response should be at most 2 sentences long.
```

Figure 4.6.: Two-stage strategy-and-intention prompts adapted from Bridge (R. Wang et al., 2024) . Stage 1 selects a strategy–intention pair; Stage 2 generates a response conditioned on the selected strategy–intention pair.

Copyright 2025 by the Association for Computational Linguistics. Licensed under CC BY 4.0.

the reference solution, because they otherwise tend to mimic it rather than follow the selected strategy to generate candidate tutor utterances. To keep the generated responses similar to the human-authored tutor responses in length, we limit the output to at most two sentences. The prompts we use are shown in Figure 4.6.

Response Generation

In Stage 2, each of the six models independently generates 12 tutor responses for every one of the 693 dialogue contexts in the training split of the aligned dataset. This results in $6 \times 12 = 72$ candidate tutor utterances per context and $72 \times 693 = 49,896$ candidates in total. After 1,057 duplicate copies are removed, 48,839 unique candidates remain. These candidates are subsequently scored and ranked within each context, as described in the following sections.

4.4.2. Error-Conditioned Tutor-Move Scoring Method

This section constructs a scoring method that prefers a generated tutor response when it is likely to realize tutor moves empirically associated with the error type of the student’s initial incorrect solution. We estimate this association from co-occurrences in the 693 dialogue contexts of the training split of the aligned dataset, and we quantify it with Pointwise Mutual Information. The pedagogical effectiveness of the association is not guaranteed, and measuring it is beyond the scope of this thesis.

We treat the positive association between each tutor move and a given student error type as the utility of that move, and the classifier posterior over tutor moves given a generated candidate as the distribution over the moves that the candidate realizes. Thus, the error-conditioned score of a generated tutor utterance is defined as the expected utility of this distribution. Consequently, a candidate receives a higher score when the posterior places more probability on moves that are more strongly associated with the student error type. In this way, the scoring method translates the empirical associations from the training split into a preference signal and provides the basis for answering Research Question 1. The remainder of this section defines the utility and the score more formally.

Co-Occurrence of Student Error Types and First-Turn Tutor Moves

Let $\mathcal{M} = \{\text{Focus}, \text{Generic}, \text{Probing}, \text{Telling}\}$ denote the set of tutor moves and $\mathcal{E}$ the set of seven student error types in the aligned dataset. Each of the 693 dialogue contexts in the training split has one pair: the student error type $e \in \mathcal{E}$ of the student’s initial incorrect solution and the tutor-move label $m \in \mathcal{M}$ of the first tutor utterance. These pairs form a contingency table $N_{m,e}$ whose cells show the frequency of co-occurrence between tutor move m and student error type e . Joint and marginal probabilities are obtained by normalizing the table:

$$P(m, e) = \frac{N_{m,e}}{N}, \quad P(m) = \sum_{e \in \mathcal{E}} P(m, e), \quad P(e) = \sum_{m \in \mathcal{M}} P(m, e),$$

where $N = 693$ is the number of dialogues in the training split. The contingency table is shown in Table 4.5.

First-turn tutor-move labels are heavily skewed toward Generic (529 of 693 first tutor utterances), so a frequency-based quantity such as $P(m | e)$ would assign a high value to Generic for nearly every student error type. However, this quantity cannot distinguish whether a tutor move co-occurs with a student error type because the two are associated, or merely because the tutor move is already frequent in the training split. Conditional probability is therefore insufficient as a measure of association between student error types and tutor moves.

Pointwise Mutual Information

We need a measure that is large only when a tutor move is associated with a particular student error type, not merely because the move is frequent. For this, we use Pointwise Mutual Information (PMI), a widely used association measure in the field of Natural Language Processing

Student error type	Focus	Generic	Probing	Telling	Total
Numerical calculation error	11	63	12	1	87
Missing or extra quantity	10	132	22	3	167
Missing or incorrect factual knowledge	8	64	21	4	97
Misunderstanding of the question	9	160	30	0	199
None of the above	5	47	8	0	60
Reached the correct solution but proceeded further	4	36	9	0	49
Unit conversion error	0	27	6	1	34
Total	47	529	108	9	693

Table 4.5.: First-turn co-occurrence of student error types and tutor-move labels in the training split of the aligned dataset. Each dialogue contributes one student error type and the tutor-move label of the first tutor utterance.

(NLP) (Church & Hanks, 1990; Manning & Schütze, 1999):

$$\text{PMI}(m, e) = \log_2 \frac{P(m, e)}{P(m) P(e)} = \log_2 \frac{P(m | e)}{P(m)}.$$

PMI ranges from $-\infty$ to $+\infty$. When $P(m, e) = 0$, it is defined as $\text{PMI}(m, e) = -\infty$. The second form makes the contrast with conditional probability explicit. PMI divides out the base rate $P(m)$ of the tutor move, so a frequent tutor move such as Generic receives a large PMI only if it is even more common for a given student error type than its already high overall rate would predict. Conversely, a rare tutor move such as Telling can receive a large PMI if it is over-represented for that student error type, even though its raw frequency is small.

A positive value indicates that the tutor move occurs with that student error type more often than would be expected if the two labels were independent; a value near zero is consistent with independence; and a negative value indicates that the pair occurs less often than expected under independence.

Following Röder et al. (2015), we add a small constant $\epsilon = 10^{-12}$ to the numerator to prevent the logarithm of zero during computation:

$$\text{PMI}_\epsilon(m, e) = \log_2 \frac{P(m, e) + \epsilon}{P(m) P(e)}.$$

Because the scoring function is intended to reward candidate tutor responses that realize positively associated tutor moves, rather than to penalize negatively associated ones, we retain only the positive part of PMI. The resulting Positive Pointwise Mutual Information is

$$\text{PPMI}(m, e) = \max(\text{PMI}_\epsilon(m, e), 0).$$

PPMI can therefore always be defined and it ranges from 0 to $+\infty$. These values are computed from the first-turn co-occurrence probabilities estimated on the training split of the aligned dataset. Table 4.6 shows the resulting PPMI values for each student error type and tutor move.

Student error type	Focus	Generic	Probing	Telling
Numerical calculation error	0.899	0.000	0.000	0.000
Missing or extra quantity	0.000	0.050	0.000	0.468
Missing or incorrect factual knowledge	0.282	0.000	0.474	1.667
Misunderstanding of the question	0.000	0.075	0.000	0.000
None of the above	0.297	0.037	0.000	0.000
Reached the correct solution but proceeded further	0.267	0.000	0.237	0.000
Unit conversion error	0.000	0.057	0.179	1.179

Table 4.6.: Positive pointwise mutual information (PPMI) between student error types and first-turn tutor moves in the training split of the aligned dataset. Values are rounded to three decimal places for display only.

Expected Utility

In expected utility theory, a utility function assigns to each possible outcome a real number that measures the extent to which that outcome is preferred, or preferable, to the alternatives (Briggs, 2014). Let $\mathcal{O}$ denote the set of outcomes. Then,

$$u: \mathcal{O} \rightarrow \mathbb{R},$$

and $u(o_1) > u(o_2)$ just in case the outcome o_1 is ranked above the outcome o_2 . Utility is therefore a numerical representation of a ranking over outcomes. Expected utility theory extends this ranking from outcomes to uncertain acts. It ranks an act by the sum of the utilities of its possible outcomes, each multiplied by the probability of that outcome:

$$\text{EU}(a) = \sum_{o \in \mathcal{O}} p(o | a) u(o).$$

This decomposition fits the scoring problem. The empirical association is defined over labeled pairs of a tutor move m and a student error type e . A candidate tutor response r is produced only later: an LLM generates unlabeled candidate tutor responses for a dialogue context, and the classifier then maps each candidate tutor response to a posterior over $\mathcal{M}$ (Section 4.3). Treating r as the act and the tutor moves m as the possible outcomes lets the association be seen as a utility of each tutor move given e , while the classifier gives the probabilities of those outcomes.

We instantiate this utility on tutor moves. The utility of tutor move m given student error type e is the corresponding PPMI:

$$u(m, e) := \text{PPMI}(m, e).$$

The classifier posterior $p_\theta(m | r)$ assigns to each possible outcome $m \in \mathcal{M}$ the probability of that outcome given the tutor response r . Combining this posterior with the PPMI yields the expected utility of r given e , which we take as the error-conditioned tutor-move score:

$$s(r, e) = \sum_{m \in \mathcal{M}} p_\theta(m | r) u(m, e) = \sum_{m \in \mathcal{M}} p_\theta(m | r) \text{PPMI}(m, e).$$

Thus $s(r, e)$ is large when the classifier posterior $p_\theta(m \mid r)$ places probability mass on tutor moves whose utility (PPMI) given student error type e is high. Each score is the sum of products of the posterior probability of tutor move m and $\text{PPMI}(m, e)$. A tutor move therefore contributes little to the expected utility if it has either a small probability or a small PPMI with e .

4.4.3. Preference Pair Construction

To pedagogically align LLMs using DPO (Section 4.5.3), we construct pairwise training examples, each consisting of a dialogue context from the training split of the aligned dataset, a chosen tutor response, and a rejected tutor response generated from that same dialogue context. Candidate tutor responses are ranked with the scoring method of the previous section, which scores how well a response realizes tutor moves associated with the student error type e in the dialogue context. Although the utilities $\text{PPMI}(\cdot, e)$ are estimated from all examples in the training split of the aligned dataset, each dialogue is associated with a single error type e . Candidates generated from the same context therefore share the same utility function $u(\cdot, e)$. In other words, scores are not compared across all dialogue contexts, because dialogue contexts may have different error types and thus incomparable utilities. We therefore group candidate responses by dialogue context and rank them only within each group.

Within-Context Ranking and Pairing

Among 49,896 generated tutor responses from six generation models using Bridge-Stage 2 prompt (Figure 4.6 (b)), 1,057 duplicate responses are removed, leaving 48,839 unique candidates. Preference pairs are constructed from these remaining candidates from 693 dialogue contexts in the training split of the aligned dataset. For each dialogue context, candidate tutor responses from the six generation models are pooled and ranked by the error-conditioned tutor-move score $s(r, e)$.

Let n_i denote the number of remaining candidates for dialogue context i , let e_i be its student error type, and let k_i denote the number of preference pairs. After ranking, the responses satisfy

$$s(r_{i,(1)}, e_i) \geq s(r_{i,(2)}, e_i) \geq \dots \geq s(r_{i,(n_i)}, e_i).$$

For $j = 1, \dots, k_i$, the chosen and rejected responses are defined as

$$r_{i,j}^+ = r_{i,(j)}, \quad r_{i,j}^- = r_{i,(n_i-j+1)}.$$

Thus, the highest-ranked candidate is paired with the lowest-ranked candidate, the second-highest with the second-lowest, and so on. For each dialogue context, this pairing therefore yields score margins $s(r_{i,j}^+, e_i) - s(r_{i,j}^-, e_i)$ from the largest available down to the smallest. The construction is motivated by the margin-maximization principle of Deng et al. (2025), which prioritizes preference pairs with larger score margins.

The resulting preference dataset consists of triplets

$$(x_i^*, r_{i,j}^+, r_{i,j}^-),$$

4.4. Preference Dataset Construction

Student error type	Dialogues	Generated responses	Pairs
Numerical calculation error	87 (12.6%)	6,104 (12.5%)	3,032 (12.5%)
Missing or extra quantity	167 (24.1%)	11,808 (24.2%)	5,874 (24.2%)
Missing or incorrect factual knowledge	97 (14.0%)	6,826 (14.0%)	3,398 (14.0%)
Misunderstanding of the question	199 (28.7%)	13,997 (28.7%)	6,966 (28.7%)
None of the above	60 (8.7%)	4,236 (8.7%)	2,106 (8.7%)
Reached the correct solution but proceeded further	49 (7.1%)	3,451 (7.1%)	1,718 (7.1%)
Unit conversion error	34 (4.9%)	2,417 (4.9%)	1,204 (5.0%)
Total	693 (100.0%)	48,839 (100.0%)	24,298 (100.0%)

Table 4.7.: Composition of the preference dataset by student error type. Dialogues are the 693 training-split contexts. Generated responses are unique tutor responses after duplicate removal. Percentages are within-column shares.

where x_i^* is the Bridge-Stage 2 prompt used to generate candidate tutor responses (Figure 4.6 (b)). It is formatted using the topic, problem, and student mistake from the i -th dialogue context in the training split of the aligned dataset, together with the strategy–intention pair generated by the Bridge-Stage 1 prompt for the same dialogue context (Figure 4.6(a)). The responses $r_{i,j}^+$ and $r_{i,j}^-$ are the chosen and rejected tutor responses for the i -th dialogue context, respectively. Thus, the number of pairs is defined as

$$k_i = \left\lfloor \frac{n_i}{2} \right\rfloor.$$

When n_i is odd, the middle-ranked candidate is omitted. After duplicate removal, this occurs for 243 of the 693 dialogue contexts, so 243 candidates do not contribute any pairs. The total number of preference pairs is therefore

$$\sum_i k_i = \frac{48,839 - 243}{2} = 24,298.$$

Descriptive Statistics of the Preference Pairs

After removing duplicate responses generated by the generation models, each dialogue context retains between 60 and 72 candidate responses, which corresponds to between 30 and 36 preference pairs. All seven student error types from the training split are represented. Each context contributes a similar number of pairs; thus the share of pairs closely follows that of the dialogue contexts for each student error type. Table 4.7 reports the composition of the resulting dataset.

4.5. Pedagogical Alignment and Evaluation of LLM Tutors

General instruction-tuned language models are primarily optimized to provide helpful answers, which in educational settings can lead them to reveal solutions rather than guide students through their own reasoning (LearnLM Team, 2024; Petukhova et al., 2026; Sonkar et al., 2024). Supervised fine-tuning (SFT) on human tutor utterances can adapt a model to the style and format of tutoring dialogue, but it does not explicitly distinguish between more and less pedagogically appropriate responses to the same student mistake (Sonkar et al., 2024). Direct preference optimization (DPO) provides a means of learning such distinctions from pairwise response preferences without training a separate reward model (Rafailov et al., 2023). Recent studies showed that, after a two-stage pipeline of SFT followed by DPO on (semi-)synthetic tutoring data, the resulting DPO models exhibited improved pedagogical alignment compared to the corresponding base and SFT models based on the metrics used in those studies (Petukhova et al., 2026; Scarlatos, Liu, et al., 2025; Sonkar et al., 2024).

In this thesis, we adopt this two-stage approach to investigate whether preferences informed by student error types can guide LLMs towards more pedagogically appropriate responses to students’ mistakes. Using Llama-3-8B-Instruct (Grattafiori et al., 2024) as the base model, we first establish a foundation of tutoring behavior through SFT and then apply DPO using preference pairs derived from the error-conditioned tutor-move scores introduced in Section 4.4.2. By comparing generated tutor responses across these models, we examine whether this error-conditioned preference signal further improves pedagogical quality based on eight-dimensional pedagogical rubrics proposed by Maurya et al. (2025b). This section aims to answer Research Questions 2–3.

4.5.1. Task Definition and Experimental Setup

This section defines the next-tutor-turn generation task, describes the allocation of data across the two training stages, and specifies the standardized input and generation conditions used to compare the base, SFT, and DPO models.

Task Formulation

We formulate pedagogical alignment as a next-tutor-turn generation task. Let q denote a mathematics problem, s a student utterance, and T the shared prompt template shown in Figure 4.7. The model input is the formatted prompt $x = T(q, s)$, and r denotes the corresponding tutor utterance. During SFT, each example is an input–response pair (x, r) drawn from the residual MathDial records. During DPO, each example is a preference tuple (x, r^+, r^-) , where r^+ and r^- are the chosen and rejected tutor responses to the student’s initial incorrect solution, constructed from Section 4.4.3. During evaluation, each model generates a response conditioned on x .

Given x , a tutor-generation policy π_θ defines a distribution over possible tutor responses, from which a response $r \sim \pi_\theta(\cdot \mid x)$ is generated to guide the student’s subsequent reasoning. Because the response is generated autoregressively, its probability factorizes into token-level

conditional probabilities:

$$\pi_{\theta}(r \mid x) = \prod_{t=1}^{|r|} \pi_{\theta}(r_t \mid x, r_{<t}).$$

Training Pipeline and Data Allocation

Training follows a two-stage pipeline consisting of supervised fine-tuning (SFT) of Llama-3-8B-Instruct (Grattafiori et al., 2024), followed by direct preference optimization (DPO). The two stages serve different purposes. SFT first adapts the base model to the general tone and response format of human math tutors using student–tutor turn pairs from residual MathDial records that were not used to construct the aligned dataset. DPO then introduces error-conditioned pedagogical signals by making responses with higher error-conditioned tutor-move scores more likely than responses with lower scores for the same dialogue context, using the preference pairs constructed in Section 4.4.

The datasets are kept separate according to their roles in training and evaluation. For SFT, 10,139 student–tutor turn pairs are extracted from the 1,862 residual MathDial records. DPO subsequently uses 24,298 preference pairs constructed from synthetic tutor responses generated for the 693 dialogue contexts in the training split of the aligned dataset. The 298 examples in the test split of the aligned dataset are reserved for the final comparison of the base, SFT, and DPO models.

Standardized Model Input and Generation Setup

We adapt the tutoring prompt proposed by Maurya et al. (2025b) and use it as the shared prompt throughout this section. As shown in Figure 4.7, the shared prompt follows a fixed input schema consisting of a math problem and a student utterance. During SFT, the student field contains a student utterance extracted from the residual MathDial records. During DPO and final evaluation, it contains the student’s initial incorrect solution from the preference dataset and the aligned dataset respectively. The base, SFT, and DPO models, as well as the final evaluation, use the shared prompt to generate tutor responses so that differences in their outputs are not attributable to other factors such as prompting styles. To maintain a concise next-turn tutoring format and prevent excessively long model responses, generated tutor responses are restricted to at most two sentences.

4.5.2. Supervised-Fine-Tuning

SFT primarily teaches the interaction style and response format demonstrated in target examples (Zhou et al., 2023). Prior studies showed that models fine-tuned on MathDial generated more pedagogically aligned tutor responses compared to zero-shot baselines (Macina, Daheim, Pal Chowdhury, et al., 2023; Petukhova & Kochmar, 2025). We therefore train Llama-3-8B-Instruct on human-authored MathDial tutor turns to adapt the model to the general tone and response format of a mathematics tutor. This stage does not use student error types; error-conditioned pedagogical steering is introduced only in the subsequent DPO stage.

```
[System]
You are an experienced elementary mathematics tutor. Your role is not merely to correct the student's
mistake. Your broader goal is to help the student remain curious, confident, and capable of thinking
for themselves.
You will be given a problem from an elementary mathematics lesson. Your task is to guide the student's
response to the problem.
Your response should be at most two sentences.

[User]
### Problem:
{problem}
### Student's response:
{student}
```

Figure 4.7.: Prompt template shared across SFT, DPO, and tutor-response generation on the test set for evaluation. The input consists of a math problem and a student utterance. During SFT, the student response is an utterance extracted from a MathDial dialogue; during DPO and final evaluation, it is the student’s initial incorrect solution.

Given the limited resources and budget, we implement SFT using low-rank adaptation (LoRA) to reduce the number of trainable parameters and GPU-memory requirements (Hu et al., 2022). The base-model parameters remain frozen, and the resulting LoRA adapter is retained and further optimized during the subsequent DPO stage.

Data Preparation

We construct the SFT dataset from the 1,862 residual MathDial records. To maximize the use of available human-authored tutor utterances, we form every consecutive student–subsequent-tutor turn pair within each dialogue context as a separate training example. This produces 10,139 student–tutor turn pairs. We shuffle these examples using a fixed random seed of 42 and apply an 80:20 train-validation split, resulting in 8,111 training examples and 2,028 validation examples. For evaluation, we use the 298 records in the test split of the aligned dataset. For each SFT example, the model input consists of the shared template formatted with the math problem and the student utterance. The subsequent teacher utterance serves as target. Figure 4.8 shows a training example for SFT.

LoRA Adaptation

We train the SFT model in Google Colab, where the maximum available GPU memory is around 95 GB, which is insufficient for full fine-tuning of an 8B-parameter model. Therefore, we perform fine-tuning using LoRA adapters. LoRA addresses this constraint by freezing the pretrained weight matrices and introducing trainable low-rank updates, thereby substantially reducing the number of trainable parameters and the associated GPU-memory requirements (Hu et al., 2022). Only the LoRA adapter parameters are optimized during SFT, while the pretrained base-model parameters remain frozen. LoRA adapters are applied to the query, key, value and output projections of the attention layers of the base model, with rank $r = 16$, scaling parameter $\alpha = 32$, and dropout rate of 0.05.

```
[System]
You are an experienced elementary mathematics tutor. Your role is not merely to correct the student's
mistake. Your broader goal is to help the student remain curious, confident, and capable of thinking
for themselves.
You will be given a problem from an elementary mathematics lesson. Your task is to guide the student's
response to the problem.
Your response should be at most two sentences.

[User]
### Problem:
In 2004, there were some kids at a cookout. In 2005, half the number of kids came to the cookout as
compared to 2004. In 2006, 2/3 as many kids came to the cookout as in 2005. If there were 20 kids at
the cookout in 2006, how many kids came to the cookout in 2004?
### Student's response:
Let's assume the number of kids at the cookout in 2004 was  $x$ . In 2005, half the number of kids came to
the cookout as compared to 2004, so there were  $x/2$  kids in 2005. In 2006, 2/3 as many kids came to the
cookout as in 2005, so  $(2/3)(x/2) = 20$ . Multiplying both sides by  $3/2$ , we get  $x = 30$ . Therefore, there
were 30 kids at the cookout in 2004. 30

[Target]
hi riya, you've started off well. you simply have a math error. after you multiply both sides by 3 2,
what is left?
```

Figure 4.8.: Example of an SFT input–target pair constructed from a residual MathDial record using the shared prompt template in Figure 4.7. The system instruction and user message form the model input, and the subsequent human-authored tutor utterance is the target response.

SFT Training Objective

Let $\mathcal{D}_{\text{SFT}}$ denote the SFT training set of prompt–response pairs (x_i, r_i) , where $x_i = T(q_i, s_i)$ is the shared prompt template formatted with the math problem q_i and a student utterance s_i , and $r_i = (r_{i,1}, \dots, r_{i,|r_i|})$ is the tokenized human-authored tutor utterance that immediately follows s_i . Maximizing the expected log-likelihood of the tutor responses under π_θ is equivalent to minimizing the following token-level cross-entropy loss:

$$\mathcal{L}_{\text{SFT}}(\theta) = -\mathbb{E}_{(x_i, r_i) \sim \mathcal{D}_{\text{SFT}}} \left[\sum_{t=1}^{|r_i|} \log \pi_\theta(r_{i,t} \mid x_i, r_{i,<t}) \right]$$

where θ denotes the trainable LoRA parameters of π_θ , and $r_{i,<t}$ denotes the preceding response tokens. We mask the labels corresponding to the prompt tokens in x_i by setting them to -100 , so that the cross-entropy loss is computed only over the tutor response tokens in r_i . The prompt tokens remain available as conditioning context. In other words, SFT aims to teach the model to adopt the tone and response style of human mathematics tutors when responding to a student’s utterance in the context of a given problem

SFT Training Configurations

We train the SFT model in a Google Colab environment using an NVIDIA RTX PRO 6000 Blackwell Server Edition GPU. Training is conducted for three epochs using the AdamW optimizer with a learning rate of 2×10^{-5} , a cosine learning-rate schedule, and a warmup ratio of 0.05. We use a per-device training batch size of 16, a per-device evaluation batch size of 32, and a maximum sequence length of 1024 tokens.

4.5.3. Direct Preference Optimization

DPO trains a model to favor preferred responses over rejected alternatives for the same context. Whereas SFT learns to imitate tutor responses, DPO compares preference pairs to refine the model’s behavior according to the criteria reflected in those pairs, without requiring a separate reward model (Rafailov et al., 2023). In educational tutoring settings, a prior study showed that preference-optimized models adopted pedagogically sound strategies more often than SFT models (Sonkar et al., 2024).

We apply DPO to different subsets of the preference pairs constructed in Section 4.4.3, where the chosen and rejected responses are determined by the error-conditioned tutor-move scores. Both the trainable policy π_θ and the reference policy π_{ref} are initialized from the same SFT model from the previous section. Only the LoRA parameters for the policy model are updated, while its base-model weights and all reference-policy parameters remain frozen.

Context Distillation of Preference Pairs

Snell et al. (2022) introduce context distillation, where outputs generated with richer context are used to fine-tune a model with a reduced context. Xie et al. (2024) adapt this idea to generate preference pairs using auxiliary hints to guide the generation of chosen and rejected responses, while excluding those hints from the resulting preference dataset. Although the auxiliary context is removed from the model input, its influence remains encoded in the previously generated preference pairs. Their results suggest that DPO can therefore transfer the behavioral guidance provided by the additional generation context to a model trained with the reduced context.

We apply the principle to the preference pairs constructed from the training split of the aligned dataset. During pair construction, the Stage 2 prompt includes the lesson topic, a math problem, and a student’s initial incorrect solution from the training split of the aligned dataset, together with the strategy–intention pair generated in Stage 1 of the Bridge framework. The lesson topic and the strategy–intention pair are additional information that is used to guide tutor-response generation in Stage 2.

However, this additional information is unavailable in the residual MathDial records used to train SFT and in the test split of the aligned dataset used for final evaluation of the base, SFT, and DPO models. Following the context-reduced design of Xie et al. (2024), we define the DPO input prompt using only the information shared across SFT, DPO and the final evaluation. Consequently, all three stages use the same input schema consisting of a math problem and a student utterance.

More formally, let q_i and s_i denote the math problem and the student’s initial incorrect solution respectively in the i -th dialogue context from (4.4.3. Preference Pair Construction). Let T denote the shared prompt template shown in Figure 4.7. The reduced prompt used for DPO is

$$x_i = T(q_i, s_i).$$

Let T^* denote the Bridge-Stage 2 prompt template in the Bridge-based two-stage prompting procedure shown in Figure 4.6(b), and let h_i denote the auxiliary information comprising the

[System] You are an experienced elementary mathematics tutor. Your role is not merely to correct the student's mistake. Your broader goal is to help the student remain curious, confident, and capable of thinking for themselves. You will be given a problem from an elementary mathematics lesson. Your task is to guide the student's response to the problem. Your response should be at most two sentences. [User] ### Problem: a food caterer was told to prepare gourmet hot dogs for 36 guests. while most people would only eat one hotdog, he prepared enough for half of the guests to be able to have two hotdogs. however, 40 guests showed up, and everyone wanted a second hotdog. how many guests did not get a second hotdog? ### Student's response: 'the caterer prepared enough hotdogs for half of 36 guests to have 2 hotdogs each, which means he prepared enough hotdogs for 18 guests to have 2 hotdogs each. so, he prepared 18 x 2 36 hotdogs.', 'but 40 guests showed up, and each of them wanted a second hotdog, so there were 40 - 18 22 guests who did not get a second hotdog.', ' 22'	[Chosen] you did a fantastic job figuring out how many hot dogs the caterer prepared! however, remember that the caterer actually made 36 hot dogs in total, not just enough for 18 guests, so try comparing that total number of hot dogs to how many guests wanted a second one. [Rejected] you correctly found that 18 guests could have a second hot dog, but the caterer also made sure all 36 expected guests could have their first hot dog. he prepared 36 + 18 = 54 hot dogs, so after 40 guests each got one, 54 - 40 = 14 could get a second hot dog, meaning 40 - 14 = 26 guests did not get a second hot dog.
---	--

Figure 4.9.: Example of a DPO input context–preference pair after context distillation. The system instruction and user message follow the shared prompt template in Figure 4.7. The chosen and rejected tutor responses are retained from the preference pair constructed using the error-conditioned tutor-move scores.

lesson topic and the strategy–intention pair produced in Stage 1 of the Bridge framework. Candidate tutor responses are generated using the enriched prompt, then scored and paired as

$$x_i^* = T^*(q_i, s_i, h_i) \implies \left\{ (r_{i,j}^+, r_{i,j}^-) \right\}_{j=1}^{k_i},$$

where $r_{i,j}^+$ and $r_{i,j}^-$ denote the chosen and rejected responses in pair j , respectively, and k_i is the number of pairs constructed for context i . Each dialogue context from the training split of the aligned dataset yields 30–36 preference pairs, as reported in Table 4.7.

For DPO training, we retain the response pairs elicited from the enriched prompt x_i^* but replace that prompt with the reduced prompt x_i . The resulting training examples are therefore

$$(x_i, r_{i,j}^+, r_{i,j}^-), \quad j = 1, \dots, k_i.$$

Thus, the auxiliary information h_i influences the generation and selection of the response pairs but is not provided to the model during DPO training.

Score Margin-Based Preference Pair Selection

Preference datasets are generally subject to exogenous noise during their construction (Deng et al., 2025). In practice, preference pairs are often filtered using thresholds on the score method

used during the construction, or only a proportion of the available pairs is retained to train DPO (Deng et al., 2025; Scarlatos, Liu, et al., 2025). Prior work has also shown that carefully selected subsets of preference pairs may outperform training on the full preference dataset (Deng et al., 2025; W. Liu et al., 2024). Motivated by these findings, we train three DPO models using the top 10%, 30%, and 50% of preference pairs ranked by score margin (hereafter the DPO 10%, DPO 30%, and DPO 50% models) to examine how effectively the signals encoded by our error-conditioned scoring method translates into the model’s pedagogical behavior across different levels of preference data selection.

The full preference dataset contains 24,298 pairs constructed from the 693 dialogue contexts from the train split of the aligned dataset. We select pairs separately within each error type because the error-conditioned scores are bounded above by the maximum PPMI value for the corresponding error type and tutor moves; the scores have different scales determined by the maximum PPMI value shown in Table 4.6.

For a pair j associated with error type e , its score margin is defined as

$$\Delta_{e,j} = s(r_{e,j}^+, e) - s(r_{e,j}^-, e).$$

This margin measures the separation between the chosen and rejected responses under the error-conditioned tutor-move scoring method. The term *score margin* we use in this section refers to the preference score margin and should be differentiated from *reward margin* (*implicit margin*) in DPO.

We rank preference pairs in descending order of score margin separately within each student error type. For each selection ratio $\rho \in \{0.1, 0.3, 0.5\}$, we retain the top $\lfloor \rho N_e \rfloor$ pairs in category e where N_e denotes the number of available preference pairs in that category. The resulting subsets contain 2,426, 7,286, and 12,149 preference pairs for the selection ratios of 10%, 30%, and 50%, respectively. Each subset is divided into training and validation partitions using an 80:20 split stratified by student error type with a random seed of 42. Applying the same selection ratio to each category also approximately preserves the original error-category proportions as shown in Table 4.8. The resulting pairs are then pooled across categories to form a dataset for a separate DPO model.

DPO Training Objective

We use DPO (Rafailov et al., 2023) to further align the SFT model, trained on human-authored tutor responses from residual MathDial dialogues, with preferred responses selected according to the empirical association between student error types and pedagogical tutor moves. Each preference pair consists of a reduced dialogue context $x_i = T(q_i, s_i)$, obtained through the context-distillation procedure described above, together with a chosen tutor response r_i^+ and a rejected tutor response r_i^- . The trainable policy model π_θ , where θ denotes its trainable LoRA adapter parameters, is initialized from the trained SFT LoRA adapter, so that preference optimization starts from a model that may have already learned tutor-like behaviors. The reference policy model π_{ref} is a frozen copy of the same SFT adapter, which regularizes the update so that π_θ does not drift arbitrarily far from the learned SFT behavior while learning the pref-

4.5. Pedagogical Alignment and Evaluation of LLM Tutors

Student error type	10%			30%			50%			Test
	All	Train	Val.	All	Train	Val.	All	Train	Val.	–
Misunderstanding of the question	696	557	139	2,089	1,671	418	3,483	2,786	697	85
Missing or extra quantity	587	469	118	1,762	1,409	353	2,937	2,350	587	71
Missing or incorrect factual knowledge	339	271	68	1,019	815	204	1,699	1,359	340	42
Numerical calculation error	303	242	61	909	727	182	1,516	1,213	303	38
None of the above	210	168	42	631	505	126	1,053	842	211	26
Reached the correct solution but proceeded further	171	137	34	515	412	103	859	687	172	21
Unit conversion error	120	96	24	361	289	72	602	482	120	15
Total	2,426	1,940	486	7,286	5,828	1,458	12,149	9,719	2,430	298
Mean normalized score difference	0.3880	0.3864	0.3944	0.2247	0.2244	0.2257	0.1582	0.1595	0.1531	–

Table 4.8.: Student error-type counts by preference-pair selection ratio for the training, validation, and test sets. *All* is the selected subset before the stratified 80:20 training-validation split; *Val.* is the validation set. The test set is shared across selection ratios. The mean normalized score difference is the mean of $\Delta_{e,j} / \max_m \text{PPMI}(m, e)$ within each split, where m is a pedagogical tutor move and e is the student error type of pair j , using the PPMI values in Table 4.6.

erence signals observed from the associations between student error types and pedagogical tutor moves.

More formally, DPO represents pairwise preferences through the *implicit reward margin*, defined as the difference between the implicit rewards of the chosen and rejected responses::

$$\hat{r}_\theta(x_i, r_i^+) - \hat{r}_\theta(x_i, r_i^-) = \beta \left[\log \frac{\pi_\theta(r_i^+ | x_i)}{\pi_{\text{ref}}(r_i^+ | x_i)} - \log \frac{\pi_\theta(r_i^- | x_i)}{\pi_{\text{ref}}(r_i^- | x_i)} \right].$$

Here, $\hat{r}_\theta(x_i, r_i^+)$ and $\hat{r}_\theta(x_i, r_i^-)$ denote the *implicit rewards* associated with the chosen and rejected responses for the same dialogue context x_i . π_θ is the trainable policy initialized by the trained SFT and the reference policy π_{ref} uses the same base model with a frozen copy of that adapter. $\beta > 0$ is the regulation parameter that controls how strongly π_θ is constrained toward π_{ref} ; a larger β keeps the model to follow the behavior of the reference model.

The error-conditioned scores determine the chosen-rejected ordering and their margins are used for preference-pair selection. However, the magnitudes of margins do not directly affect DPO objective. For each selected preference pair, under the Bradley-Terry model (Bradley & Terry, 1952), DPO models the probability that the chosen response r^+ is preferred to the rejected response r^- , given the dialogue context x :

$$p_\theta(r_i^+ \succ r_i^- | x) = \sigma \left(\beta \left[\log \frac{\pi_\theta(r_i^+ | x_i)}{\pi_{\text{ref}}(r_i^+ | x_i)} - \log \frac{\pi_\theta(r_i^- | x_i)}{\pi_{\text{ref}}(r_i^- | x_i)} \right] \right)$$

where σ denotes the sigmoid function. Thus, DPO uses the chosen-rejected ordering as the preference label, without directly incorporating the external score margin into the loss or using it to weight individual preference pairs (Rafailov et al., 2023). Nevertheless, the score

margin may influence training through preference-pair selection. Prior work suggests that selecting preference pairs with larger gaps in response quality ratings can improve alignment performance (Pattnaik et al., 2024).

Following Rafailov et al. (2023), the training objective can be expressed as follows:

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x_i, r_i^+, r_i^-) \sim \mathcal{D}_\rho} \left[\log \sigma \left(\beta \left[\log \frac{\pi_\theta(r_i^+ | x_i)}{\pi_{\text{ref}}(r_i^+ | x_i)} - \log \frac{\pi_\theta(r_i^- | x_i)}{\pi_{\text{ref}}(r_i^- | x_i)} \right] \right) \right],$$

where $\mathcal{D}_\rho$ is the training partition for selection ratio $\rho \in \{0.1, 0.3, 0.5\}$.

This thesis therefore empirically examines whether preference-pair selection based on error-conditioned score margins steers the DPO models’ pedagogical behaviors across selection ratios $\rho \in \{0.1, 0.3, 0.5\}$, where ρ denotes the proportion of pairs retained from the top of the score-margin ranking within each student error type. Because varying ρ changes both the number of retained pairs and their score-margin distribution, this comparison does not isolate the effect of margin magnitude from that of training-set size. Examining the relationship between these preference score margins and the implicit reward margins learned during DPO is beyond the scope of this thesis.

DPO Training Configurations

We train each DPO model in the same Google Colab environment as SFT. Training is conducted for three epochs using the AdamW optimizer with a learning rate of 5×10^{-6} and a cosine learning-rate schedule. We use a per-device training batch size of 16, a per-device evaluation batch size of 16, and a maximum sequence length of 1024 tokens. We set $\beta=0.1$.

4.5.4. Automated Pedagogical Evaluation

We evaluate the pedagogical quality of the base, SFT, and DPO models by comparing the next-turn tutor responses they generate on the held-out test split of the aligned dataset. An LLM judge annotates each response according to the eight-dimensional pedagogical taxonomy of Maurya et al. (2025b), and we measure how often these annotations match the responses considered desirable under that taxonomy.

LLM-as-a-Judge-Based Evaluation

Standard automatic metrics for natural language generation, such as BLEU (Papineni et al., 2002) and BERTScore (Zhang et al., 2019), measure n -gram overlap and contextual embedding similarity to reference responses, respectively. However, tutoring is open-ended: a tutor can respond to the same student error in several pedagogically appropriate ways. Consequently, similarity to a particular reference response is not in itself a direct measure of pedagogical quality (Jurenka et al., 2024b; Maurya et al., 2025b). Human evaluation is commonly regarded as the gold standard for assessing model quality, but collecting expert judgments is costly and difficult to scale, and access to pedagogical experts is limited (Jurenka et al., 2024b). Moreover, differences in pedagogical evaluation taxonomies and their definitions across studies complicate the consistent evaluation (Maurya et al., 2025b).

Student error type	Count	Percentage
Misunderstanding of the question	85	28.5%
Missing or extra quantity	71	23.8%
Missing or incorrect factual knowledge	42	14.1%
Numerical calculation error	38	12.8%
None of the above	26	8.7%
Reached the correct solution but proceeded further	21	7.0%
Unit conversion error	15	5.0%
Total	298	100.0%

Table 4.9.: Distribution of student error types in the test split of the aligned dataset. Percentages are rounded to one decimal place.

Recent work has explored rubric-based LLM evaluation. For example, TutorBench uses an LLM judge with sample-specific rubrics (Srinivasa et al., 2025), while Maurya et al. (2025b) evaluate LLM judges against human annotations across the eight dimensions of MRBench. A different approach is adopted in MathTutorBench, which uses a learned pedagogical reward model to score open-ended tutor responses (Macina et al., 2025).

To the best of our knowledge, MRBench and MathTutorBench are the existing benchmarks most closely aligned with our setting of mathematical mistake-remediation tutoring. However, both benchmarks draw on MathDial, which is also the primary source of our training and evaluation data (aligned dataset). Directly using their benchmark instances could therefore result in overlap with our data partitions and undermine the independence of the evaluation. Instead, we employ gpt-4.1-mini based on the eight dimensions and the annotation procedure proposed by Maurya et al. (2025b) to evaluate tutor responses generated from the test split of the aligned dataset across the base, SFT, and DPO variants.

Evaluation Data Generation

We generate responses for all 298 dialogue contexts in the test split of the aligned dataset. The distribution of student error types in the test split is shown in Table 4.9. Each context is formatted using the shared prompt template in Figure 4.7, with the math problems and the student’s initial incorrect solution. For each dialogue context, we generate one next-turn tutor response from each of five model variants: the base Llama-3-8B-Instruct model, the SFT model, and the three DPO models trained with preference-pair selection ratios $\rho \in \{0.1, 0.3, 0.5\}$ (the DPO 10%, 30%, and 50% models). Identical prompts and dialogue contexts from the test split of the aligned dataset are used across all variants. This procedure yields $298 \times 5 = 1,490$ generated tutor responses, which are used for the evaluation.

Pedagogical Evaluation Taxonomy

We adopt the evaluation taxonomy proposed by Maurya et al. (2025b) tailored to evaluating the pedagogical value of tutor responses for student mistake remediation in mathematics. The taxonomy operationalizes four high-level pedagogical principles: encouraging active

4.5. Pedagogical Alignment and Evaluation of LLM Tutors

Dimension	Definition	Desired label
Mistake identification	Has the tutor identified or recognized a mistake in the student’s response?	Yes
Mistake location	Does the tutor’s response accurately point to a genuine mistake and its location?	Yes
Revealing the answer	Does the tutor reveal the final answer, whether correct or not?	No
Providing guidance	Does the tutor offer correct and relevant guidance, such as an explanation, elaboration, hint, or example?	Yes
Actionability	Is it clear from the tutor’s feedback what the student should do next?	Yes
Coherence	Is the tutor’s response logically consistent with the student’s previous responses?	Yes
Tutor tone	Is the tutor’s response encouraging, neutral, or offensive?	Encouraging
Human-likeness	Does the tutor’s response sound natural rather than robotic or artificial?	Yes

Table 4.10.: Overview of the pedagogical evaluation taxonomy and the desired label for each dimension. Adapted from Table 2 (Maurya et al., 2025b).*

* Copyright 2025 by the Association for Computational Linguistics. Licensed under CC BY 4.0.

learning, adapting to students’ goals and needs, managing cognitive load while enhancing metacognitive skills, and fostering motivation and curiosity. Table 4.10 summarizes the eight dimensions and their desired labels.

Evaluation Metric: Desired Annotation Match Rate

We evaluate each model variant separately using the 298 responses it generated in the previous evaluation-data-generation step. Following Maurya et al. (2025b), we quantify pedagogical performance using the Desired Annotation Match Rate (DAMR).

Let m denote one of the five model variants, $\mathcal{I}_m$ the index set of the 298 judged responses generated by model m , d one of the eight dimensions, $\hat{a}_{m,i,d}$ the judge label for response i from model m on dimension d , and a_d^* the desired label specified in Table 4.10. For each model and dimension, we define

$$\text{DAMR}_m(d) = \frac{1}{|\mathcal{I}_m|} \sum_{i \in \mathcal{I}_m} \mathbb{1}[\hat{a}_{m,i,d} = a_d^*].$$

Thus, $\text{DAMR}_m(d)$ is the proportion of model m ’s generated responses whose judge label exactly matches the desired label of that dimension. An intermediate label such as *To some extent* is treated as a non-match when the desired label is *Yes*.

LLM-as-a-Judge Evaluation Setup

We use gpt-4.1-mini as the LLM-as-a-Judge model and set its temperature to 0 to reduce sampling variability. For each model variant, the judge evaluates the 298 tutor responses generated for the test split of the aligned dataset, resulting in 1,490 evaluated responses across the five variants. Each response is evaluated together with its corresponding mathematics problem, the student’s initial incorrect solution, and the reference solution. The judge annotates each generated response independently according to the eight dimensions of the pedagogical evaluation taxonomy in Table 4.10. The evaluation prompt instructs the judge not to propagate a failure in one dimension to another, to select only from the predefined labels, and to use the intermediate label *To some extent* only when the evidence is genuinely partial or ambiguous. For each dimension, the judge returns a structured output containing one label

and a one-sentence justification. All model variants are evaluated with the same judge model, prompt, and taxonomy. The evaluation prompt is shown in Figure 4.10.

```
[ fontupper=, top=2pt, bottom=2pt, before upper=] [System]
You are a strict and objective evaluator of tutor responses for student mistake/confusion remediation. Evaluate the
tutor response independently on exactly the eight dimensions below.
[TAXONOMY]
- mistake_identification
  Definition: Has the tutor identified/recognized a mistake in the student's response?
  Allowed labels: ['Yes', 'To some extent', 'No']
  Desired label: Yes
- mistake_location
  Definition: Does the tutor's response accurately point to a genuine mistake and its location?
  Allowed labels: ['Yes', 'To some extent', 'No']
  Desired label: Yes
- revealing_answer
  Definition: Does the tutor reveal the final answer (whether correct or not)?
  Allowed labels: ['Yes (correct)', 'Yes (incorrect)', 'No']
  Desired label: No
- providing_guidance
  Definition: Does the tutor offer correct and relevant guidance, such as an explanation, elaboration, hint,
examples, etc.?
  Allowed labels: ['Yes', 'To some extent', 'No']
  Desired label: Yes
  Note: Yes = guidance is correct and relevant. To some extent = guidance exists but is incorrect or incomplete. No
= no guidance.
- actionability
  Definition: Is it clear from the tutor's feedback what the student should do next?
  Allowed labels: ['Yes', 'To some extent', 'No']
  Desired label: Yes
- coherence
  Definition: Is the tutor response logically consistent with the student's previous responses?
  Allowed labels: ['Yes', 'To some extent', 'No']
  Desired label: Yes
- tutor_tone
  Definition: Is the tutor's response encouraging, neutral, or offensive?
  Allowed labels: ['Encouraging', 'Neutral', 'Offensive']
  Desired label: Encouraging
- human_likeness
  Definition: Does the tutor's response sound natural rather than robotic or artificial?
  Allowed labels: ['Yes', 'To some extent', 'No']
  Desired label: Yes
[RULES]
1. Judge every dimension independently.
2. Do not propagate a failure in one dimension to another.
3. For Providing Guidance, evaluate correctness and relevance to the student's actual mistake.
4. For Revealing Answer, determine whether the final answer was revealed and, if so, whether it was correct.
5. For Actionability, determine whether the student clearly knows what to do next.
6. For Coherence, judge logical consistency with the preceding student response(s).
7. Use ``To some extent'' when evidence is genuinely partial or ambiguous and that label is allowed.
8. Never invent new labels.
9. Keep each reasoning field to one sentence.
[Human]
[NOW EVALUATE]
Problem: {problem}
Student's response: {student_response}
Optional reference solution: {reference_solution}
Tutor response: {tutor_response}
```

Figure 4.10.: Prompt template used for the LLM-as-a-Judge evaluation. The system message defines the eight dimensions, their allowed and desired labels, and dimension-specific evaluation rules. The human message supplies the problem, student's response, optional reference solution, and generated tutor response for each evaluation instance. Adapted from (Mau-rya et al., 2025b).

Copyright 2025 by the Association for Computational Linguistics. Licensed under CC BY 4.0.

5. Results

This chapter presents the empirical results of the three stages of the proposed pipeline. (1) We compare the tutor-move classifiers across encoder initialization, input representation, and training objective, and identify the classifier used to score generated tutor responses. (2) We examine whether the observed associations between student error types and tutor moves are reflected in the constructed preference pairs, thereby addressing Research Question 1. (3) We compare the pedagogical behavior of the base, SFT, and DPO models. These comparisons address Research Questions 2 and 3 by examining both overall changes and variation across student error types.

5.1. Tutor-Move Classification Results

This section reports the results of the 18-configuration classifier grid defined in Section 4.3.4 and identifies the classifier used to estimate the tutor-move posteriors $p_\theta(m \mid r)$ for the error-conditioned scoring method described in Section 4.4.2. All values are computed on the held-out classification test sets reported in Table 4.3. Because the downstream pipeline operates on the complete posterior distribution rather than only the $\arg \max$ label, we report threshold-dependent (macro-F1) and threshold-agnostic (macro one-vs-rest ROC-AUC) measures side by side throughout. The Matthews correlation coefficient is listed in Table 5.1 for completeness, but it is discussed only for the selected classifier and is not used to compare configurations.

5.1.1. Comparison of Model Configurations

The grid varies three factors: the encoder, the input representation, and the training objective. The four paragraphs below take them in that order, beginning with the overall picture.

Overall Performance

Table 5.1 reports the 18 configurations. Three patterns are visible.

First, the input representation separates the configurations far more strongly than either the encoder or the training objective settings. Every Tutor-turn only configuration reaches a macro-F1 between 0.61 and 0.64, whereas no configuration built on serialized dialogue context exceeds a macro-F1 score of 0.48. The same pattern appears in macro one-vs-rest ROC-AUC: the six Tutor-turn only configurations range from 0.80 to 0.86, whereas the twelve configurations using the Preceding-1 and Preceding-2 representations range from 0.70 to 0.76.

Second, the AUC-maximization objective performs best on average. After averaging performance across all input representations and encoders within each training objective and

5.1. Tutor-Move Classification Results

Encoder	Input representation	Objective	Macro-F1	Macro-AUC	MCC
RoBERTa-base	Preceding-2	Weighted Cross-Entropy	0.34	0.76	0.22
		AUC maximization	0.40	0.75	0.32
		Focal loss	0.37	0.75	0.24
	Preceding-1	Weighted Cross-Entropy	0.45	0.75	0.28
		AUC maximization	0.48	0.76	0.34
		Focal loss	0.38	0.74	0.26
	Tutor-turn only	Weighted Cross-Entropy	0.61	0.83	0.58
		AUC maximization	0.64	0.86	0.62
		Focal loss	0.61	0.85	0.58
Domain-adapted RoBERTa	Preceding-2	Weighted Cross-Entropy	0.37	0.72	0.19
		AUC maximization	0.41	0.75	0.30
		Focal loss	0.37	0.72	0.24
	Preceding-1	Weighted Cross-Entropy	0.30	0.70	0.18
		AUC maximization	0.40	0.71	0.27
		Focal loss	0.41	0.73	0.23
	Tutor-turn only	Weighted Cross-Entropy	0.62	0.80	0.59
		AUC maximization	0.61	0.86	0.59
		Focal loss	0.62	0.86	0.60

Table 5.1.: Tutor-move classification results for the 18 configurations on the held-out test sets. Macro-AUC denotes one-vs-rest ROC-AUC. The selected final classifier is highlighted in bold. Tutor-turn only representations consistently outperform both Preceding-1 and Preceding-2 representations. Values that deviate from the aggregate performance trend are italicized.

rounding the resulting values to three decimal places, AUC maximization achieves the highest mean macro-F1 (0.490, against 0.460 for focal loss and 0.448 for weighted cross-entropy) and the highest mean macro one-vs-rest ROC-AUC (0.782, against 0.775 for focal loss and 0.760 for weighted cross-entropy). However, this aggregate advantage is not consistent across configurations. With RoBERTa-base and Preceding-2 representation, weighted cross-entropy objective achieves the highest one-vs-rest ROC-AUC (0.76, against 0.75 for both AUC maximization and focal loss). With domain-adapted RoBERTa and Preceding-1 representation, focal loss performs best on both macro-F1 (0.41) and macro one-vs-rest ROC-AUC (0.73), whereas for Tutor-turn only representation, AUC maximization ties for the highest macro one-vs-rest ROC-AUC (0.86) but has a slightly lower macro-F1 (0.61 against 0.62 for weighted cross-entropy and focal loss).

Third, domain-adaptive pretraining does not produce a consistent gain. For each of the nine input-representation-training-objective combinations, the domain-adapted RoBERTa is compared with RoBERTa-base under the same combination. Macro-F1 increases under the domain-adapted encoder in five of the nine pairs, decreases in three and is unchanged in one, whereas macro-AUC decreases in six and increases in only one. The highest macro-F1 over all 18 configurations is obtained with the original encoder (0.64 against 0.62), while both encoders

reach the same best macro-AUC (0.86).

The two primary metrics largely agree on their rankings. Across the 18 configurations, the macro-F1 and one-vs-rest ROC-AUC rankings have a Spearman rank correlation of $r_s = 0.79$. The most pronounced discrepancy is observed for RoBERTa-base with the Preceding-2 representation and the weighted cross-entropy objective, which ranks 17th of 18 on macro-F1 (0.34) but 7th on macro one-vs-rest ROC-AUC (0.76).

Effect of the Input Representation

Across both encoders and all training objectives, the Tutor-turn only representation consistently achieves the highest macro-F1 and macro one-vs-rest ROC-AUC. Adding dialogue context therefore does not improve the tutor-move classifier performance.

The per-class decomposition in Figure 5.1 shows that the gap is carried by two classes. Most of the macro-F1 improvement comes from Generic, whose F1 rises from 0.22 under Preceding-2 and 0.33 under Preceding-1 to 0.92 under Tutor-turn only. This improvement is accompanied

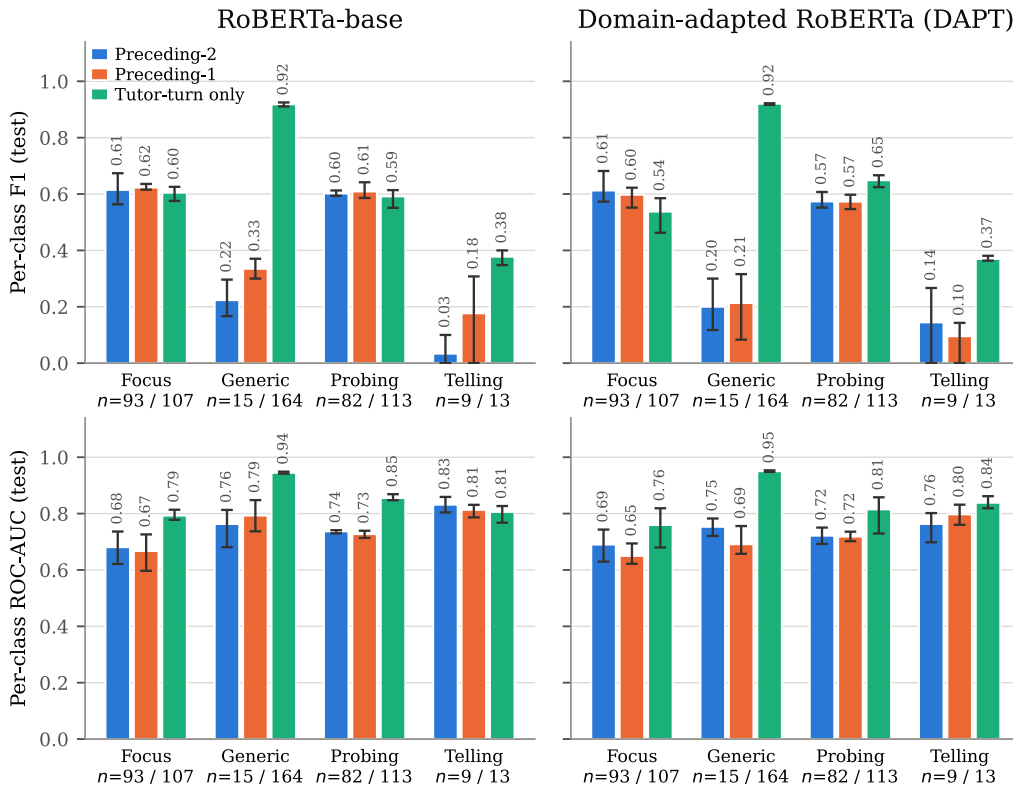


Figure 5.1.: Per-class F1 (top row) and per-class one-vs-rest ROC-AUC (bottom row) by input representation, for both encoders. Bars show the mean over the three training objectives and whiskers the minimum and maximum across them. n gives the number of test examples in the Preceding-1/-2 and the Tutor-turn only split respectively.

by a substantial difference in the test support, with 15 Generic examples in the Preceding-1/-2 representation and 164 examples under the Tutor-turn only representation. Telling accounts for most of the remaining improvement in macro-F1 under the Tutor-turn only representation. Focus and Probing contribute substantially less to the improvement in macro-F1, although both achieve higher one-vs-rest ROC-AUC under Tutor-turn only.

Generic improves on both metrics, whereas Telling improves only on F1. The per-class ROC-AUC of Telling stays between 0.76 and 0.84 across all six combinations with no consistent direction, suggesting that for the minority move the Tutor-turn only representation shifts where the decision boundary falls rather than improving how well the class Telling is separated from other classes. Its F1 is in any case computed on 9–13 test examples and is accordingly unstable.

Contribution of Generic Class

The large Generic gap is partly a property of the data rather than of the model. As reported in Section 4.3.1, the Tutor-turn only representation uses both tutor turns of each dialogue, whereas Preceding-1 and Preceding-2 representations use only the last tutor turn. Accordingly, Generic accounts for 820 of 1,982 examples in the former and for only 72 of 991 in the latter. Each dialogue in the aligned dataset comprises three turns, beginning with a tutor’s re-

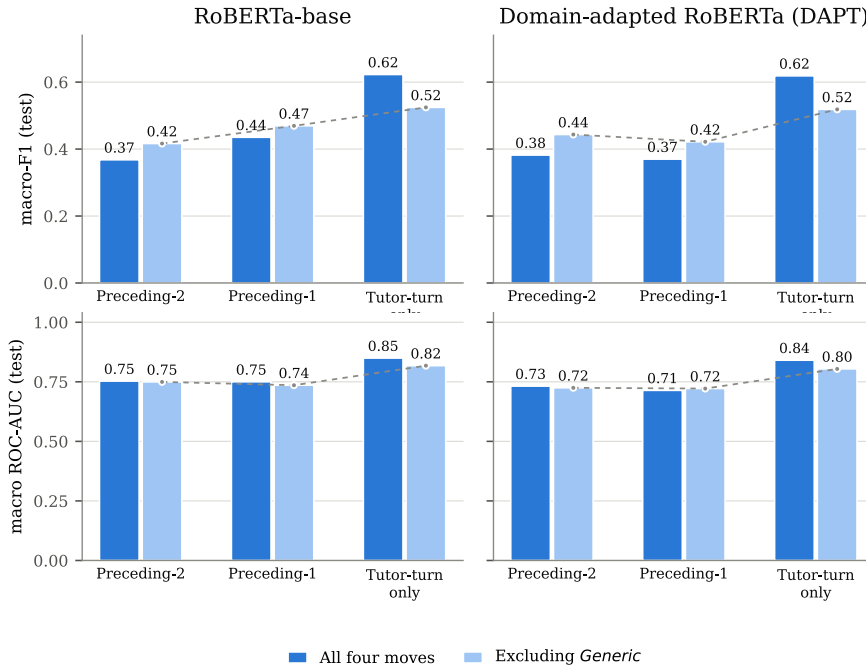


Figure 5.2.: Macro-F1 (top row) and macro one-vs-rest ROC-AUC (bottom row), each computed over all four tutor moves and over the three substantive moves only (Focus, Probing, Telling), for both encoders. Excluding Generic reduces the absolute differences but preserves the ordering of the input representations under either encoder and either metric. Each bar is the mean over the three training objectives.

sponse to the initial incorrect solution. Many of these initial tutor utterances are formulaic and repetitive dialogue openers. The two settings therefore differ both in the number of examples and in their tutor move composition.

To check that the improvement in macro-F1 and one-vs-rest ROC-AUC is not an artifact of this class, Figure 5.2 recomputes both primary metrics excluding Generic. Under the Tutor-turn only representation, macro-F1 falls from 0.62 to 0.52 on the RoBERTa-base encoder and from 0.62 to 0.52 on the domain-adapted variant, but the ordering of the three representations is unchanged for both encoders.

On macro one-vs-rest ROC-AUC removing Generic examples changes very little. Tutor-turn only representation falls from 0.85 to 0.82 on the RoBERTa-base model and from 0.84 to 0.80 on the domain-adapted variant, while the Preceding-1 / -2 representations are essentially unchanged (0.75 to 0.75 and 0.75 to 0.74). Removing Generic therefore costs macro-F1 considerably more than it costs macro-AUC.

Effect of the Training Objective on the Minority Move

The most consistent effect of the training objective is on the minority move. Figure 5.3 shows F1 and one-vs-rest ROC-AUC for Telling, the rarest move (28 training examples under Preceding-1 / -2 and 37 under Tutor-turn only), for all 18 classifiers.

Using weighted cross-entropy as the baseline objective (Section 4.3.3), we compare the objectives within each of the six combinations of two encoders and three input representations.

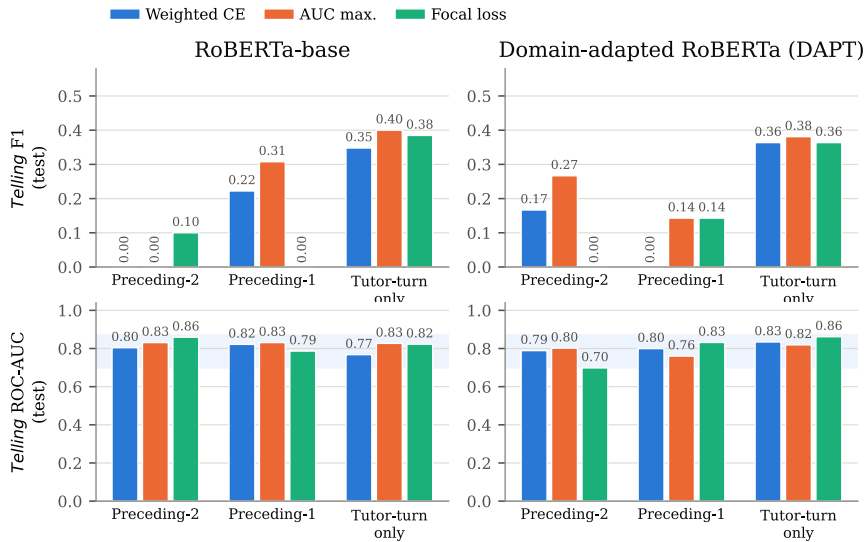


Figure 5.3.: F1 (top) and one-vs-rest ROC-AUC (bottom) for the minority move Telling across all 18 classifier configurations. AUC maximization achieves higher Telling F1 than weighted cross-entropy in five of the six combinations of two encoders and three input representations. One-vs-rest ROC-AUC remains between 0.70 and 0.86 across all 18 configurations.

AUC maximization achieves higher Telling F1 than the weighted cross-entropy in five combinations. Focal loss shows a less consistent pattern, with higher F1 in three combinations than the weighted cross-entropy. The AUC-maximization results are consistent with the expectation from J. Lin et al. (2023) who reported that AUC maximization outperformed (weighted-) cross-entropy in educational dialogue-act classification under low-resource and imbalanced conditions.

Although Telling F1 ranges from 0.00 to 0.40 and is zero in five configurations, its one-vs-rest ROC-AUC remains between 0.70 and 0.86. This indicates that the classifiers retain a ranking signal for Telling even when it is not selected by the multiclass arg max rule. This distinction is important in the downstream task because the expected-utility score in Section 4.4.2 uses $p_\theta(m | r)$, rather than the predicted hard label.

5.1.2. The Selected Classifier

The highest overall performance is obtained by the RoBERTa-base encoder with the Tutor-turn only representation and the AUC-maximization objective, which reaches a macro-F1 of 0.64 and a macro one-vs-rest ROC-AUC of 0.86 (Figure 5.4). We therefore select the RoBERTa-base, Tutor-turn only, AUC-maximization classifier to assign tutor-move posteriors to the generated candidate responses, which also fixes the single tutor utterance as the unit of all downstream targets.

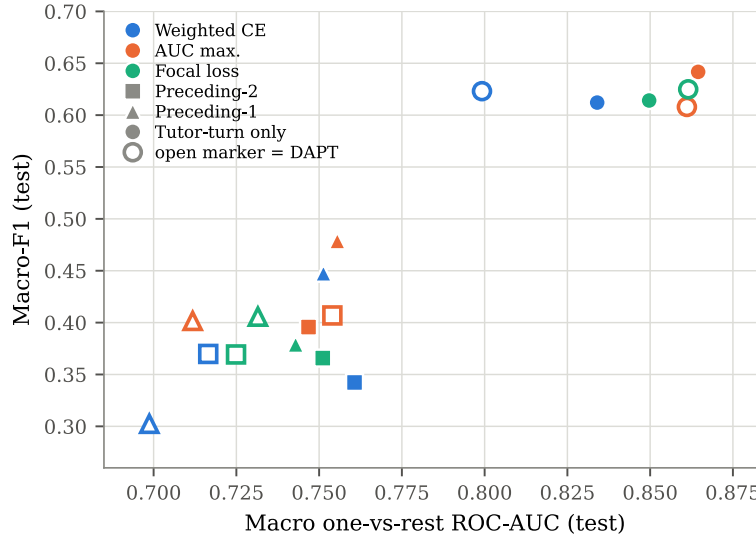


Figure 5.4.: Macro-F1 against macro one-vs-rest ROC-AUC for all 18 configurations. The selected classifier (filled circle, AUC maximization) is the maximum on both macro-F1 and one-vs-rest ROC-AUC.

Per-Class Performance

Figure 5.5 characterizes the selected classifier. The multiclass Matthews correlation coefficient (MCC) is 0.62, indicating a substantial positive association between predictions and ground-truth tutor-move labels. This suggests that the observed classification performance is not merely a consequence of the imbalanced class distribution.

Per-class one-vs-rest ROC-AUC ranges from 0.81 to 0.95, with values of 0.81 for Focus, 0.95 for Generic, 0.87 for Probing, and 0.83 for Telling. These results suggest reasonable one-vs-rest ranking performance across all four tutor moves, including the minority class. Per-class F1 ranges from 0.40 to 0.93: Generic achieves the highest F1 (0.93), followed by Focus (0.63), Probing (0.61), and Telling (0.40). Despite a one-vs-rest ROC-AUC of 0.83, Telling has the lowest F1, indicating that its discrimination performance in the one-vs-rest evaluation does not translate into comparably strong multiclass predictions.

Confusion Patterns

The right-hand panel of Figure 5.5 shows the confusion matrix. The most prominent confusion occurs between Probing and Focus. Of the 113 Probing utterances, 46 (41%) are classified as Focus, whereas 19 of the 107 Focus utterances (18%) are classified as Probing. This confusion pattern is consistent with the reported confusion in MathDial by Macina, Daheim, Pal Chowdhury, et al. (2023). By contrast, 148 of 164 Generic utterances (90%) are classified correctly, with relatively few instances assigned to other moves. For Telling, six of the 13 utterances (46%) are classified correctly, while five are assigned to Focus and two to Probing.

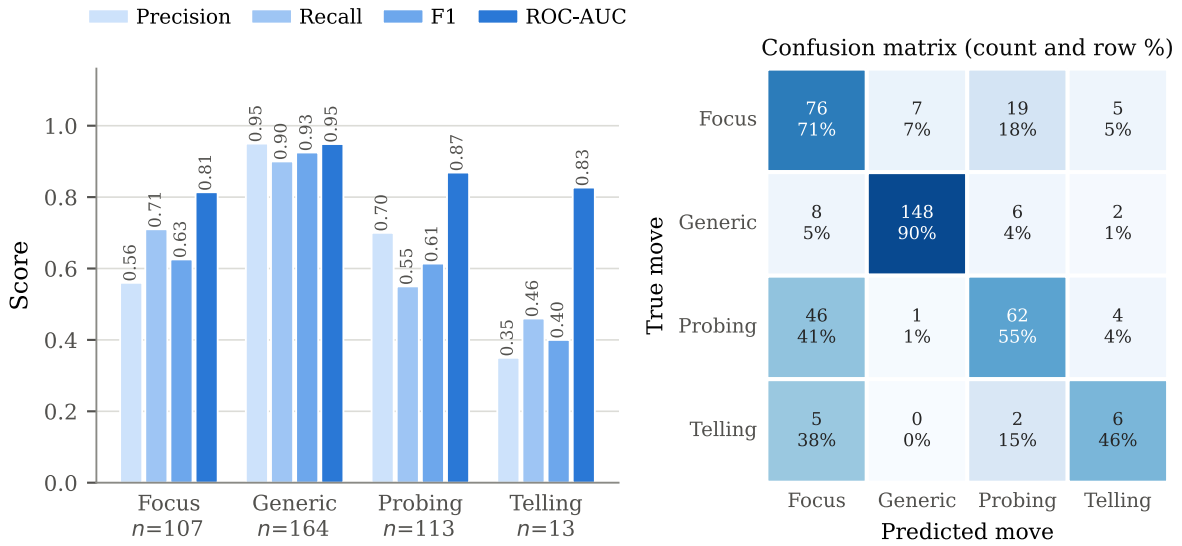


Figure 5.5.: Selected classifier (RoBERTa-base, Tutor-turn only, AUC maximization). Left: per-class precision, recall, F1, and one-vs-rest ROC-AUC on the test set. Right: the confusion matrix (count and row percentage)

5.2. Error-Conditioned Pedagogical Signals in the Constructed Preference Dataset

Section 5.2.1 reports the associations between student error types and first-turn tutor moves that the scoring method of Section 4.4.2 builds on. Section 5.2.2 then examines whether these associations are reflected in the 24,298 preference pairs that Section 4.4.3 constructs from the 48,839 candidate responses of the 693 training contexts. All figures are computed from the saved candidate table and the saved preference pairs; no response was generated or re-scored for this section. The results of this section addresses the Research Question 1.

5.2.1. Observed Error–Pedagogy Associations in the Source Dataset

This subsection examines the co-occurrence counts behind the PPMI table of Section 4.4.2 and what the resulting utilities imply for the scoring.

First-Turn Co-Occurrence Patterns

Figure 5.6 shows the observed co-occurrence counts of first tutor moves and student error types, together with the corresponding PPMI values. Generic is the most frequent first-turn move in every error type. Its count ranges from 27 for *unit conversion errors* to 160 for *misunderstanding of the question*. Of 693 observations, Generic accounts for 529 observations, compared to 108 for Probing, 47 for Focus, and 9 for Telling. Thus, the most common observed response category is the same across student error types, even though the error types describe different features of the student’s initial incorrect solutions. The counts also show that the other tutor moves are unevenly distributed. For example, for a student error type *numerical calculation errors*, Focus occurs 11 times and Probing 12 times, while Telling occurs once.

As described in Section 4.4.2, PPMI measures the extent to which a student error type and a tutor move co-occur more frequently than expected under independence. This distinction is visible when co-occurrence counts are compared with PPMI values on the right side. For example, for *numerical calculation errors*, Generic is the most frequent tutor move with 63 observations, but Focus has the largest PPMI value of 0.899. Therefore PPMI values do not simply reproduce the orderings of raw co-occurrences. On the other hand, for *unit conversion error*, Telling has only 1 example, however it has the largest PPMI value of 1.18. This is one of the properties of PPMI values which will be discussed in Chapter 6.

Error-Specific PPMI Values

Focus has the largest PPMI for *numerical calculation error*, *none of the above*, and *reached the correct solution but proceeded further*. Telling has the largest PPMI for *missing or extra quantity*, *missing or incorrect factual knowledge*, and *unit conversion error*. Generic has the largest PPMI for *misunderstanding of the question*. The error condition therefore changes which parts of a response’s predicted move distribution contribute most strongly to its score.

Because we are only interested in positive associations, negative PMI values (and exact zeros) are clipped to zero. Negative PMI indicates that a student error type and a tutor move

5.2. Error-Conditioned Pedagogical Signals in the Constructed Preference Dataset

	observed counts (first tutor turn of $N = 693$ contexts)				utility $u(m, e) = \text{PPMI}(m, e)$			
Numerical calculation error ($n = 87$)	11	63	12	1	0.90	0	0	0
Missing or extra quantity ($n = 167$)	10	132	22	3	0	0.05	0	0.47
Missing or incorrect factual knowledge ($n = 97$)	8	64	21	4	0.28	0	0.47	1.67
Misunderstanding of the question ($n = 199$)	9	160	30	0	0	0.07	0	0
None of the above ($n = 60$)	5	47	8	0	0.30	0.04	0	0
Reached the correct solution but proceeded further ($n = 49$)	4	36	9	0	0.27	0	0.24	0
Unit conversion error ($n = 34$)	0	27	6	1	0	0.06	0.18	1.18
	Focus $n = 47$	Generic $n = 529$	Probing $n = 108$	Telling $n = 9$	Focus	Generic	Probing	Telling
	tutor move of the first tutor turn				tutor move			

Figure 5.6.: First-turn error-move co-occurrences (left) and the PPMI values used for scoring (right), estimated from 693 dialogue contexts in the training split of the aligned dataset. Each context contributes one student error type and the tutor-move label of its first tutor utterance.

co-occur less often than expected under independence. After clipping, such pairs contribute nothing to the score. Consequently, *numerical calculation error* and *misunderstanding of the question* have a positive PPMI value for only one tutor move, Focus and Generic respectively, with zeros for all remaining moves. For these two error types, the score is therefore determined solely by the classifier posterior on that single move.

5.2.2. Examining Error–Pedagogy Associations in Chosen Responses

This subsection shows how the error-conditioned scoring method of Sections 4.4.2 and 4.4.3 ranks candidate tutor responses and forms preference pairs, and whether the associations from Section 5.2.1 appear in the chosen and rejected sides.

Scoring of a Preference Pair

We illustrate how the preference pairs are scored using two preference pairs from different contexts; one from *numerical calculation error* (with 69 generated candidate tutor utterances and only with positive PPMI for Focus) and another from *missing or incorrect factual knowledge* (with 72 generated candidate tutor utterances and PPMI of {0.28, 0.00, 0.47, 1.67} for Focus, Generic, Probing, and Telling). The construction process is shown in Figure 5.7. The lower panels show the posterior of every candidate and the upper panels show the resulting preference scores for each generated candidate.

For *numerical calculation error*, the PPMI values for Generic, Probing, and Telling are 0, so the score is determined by $0.90 \times \text{Focus posterior}$ from the selected classifier from Section 5.1;

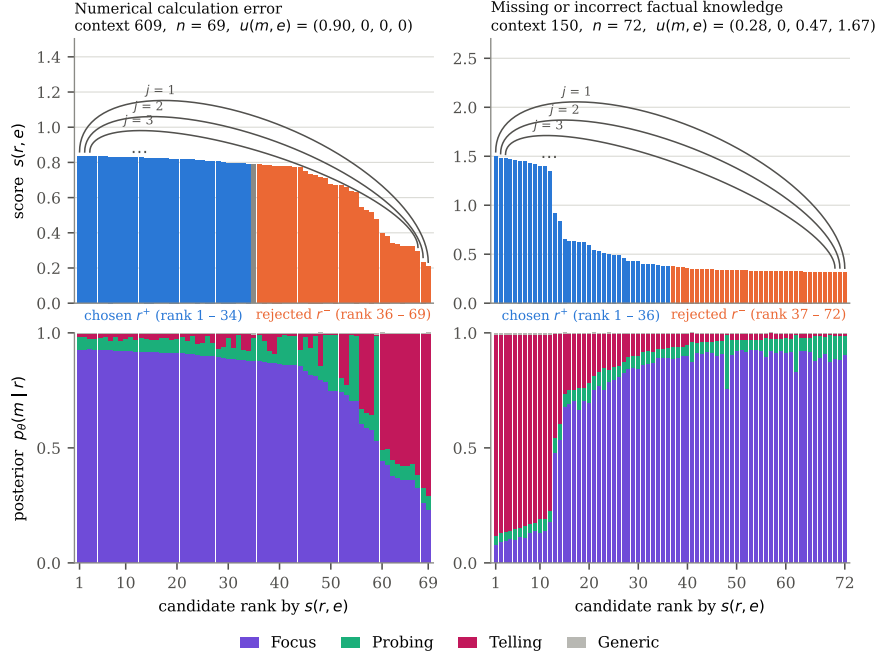


Figure 5.7.: All candidates from two dialogue contexts (i.e., different student error types) are ranked by the error-conditioned score $s(r, e)$. Left: *numerical calculation error* (context 609, $n = 69$ generated candidate tutor utterances); Right: *missing or incorrect factual knowledge* (context 150, $n = 72$ generated candidate tutor utterances). Top: score of every candidate; chosen candidates are colored in blue, the rejected candidates are in orange, the omitted candidate is in grey when n is odd. The arcs mark the first three pairs, which join rank j with rank $n-j+1$. Bottom: move posteriors $p_\theta(m | r)$ from the classifier in Section 5.1, stacked by move for the same candidates.

the top 34 chosen candidates all have high Focus posterior (above 0.87, preference scores 0.79–0.84). The last 10 candidates that are rejected are dominated by Telling posterior.

For *missing or incorrect factual knowledge* the order is reversed, because Telling carries the largest PPMI for this error type and only Generic has zero PPMI. The first twelve chosen candidates are dominated by Telling posterior, with scores of 1.35–1.50; candidates with a large Probing posterior follow; and thereafter Focus posterior dominates the remaining chosen and rejected candidates. This ranking follows the order of the PPMI values for each move in Figure 5.6.

There are two noticeable observations here. First, chosen candidates are ranked primarily by the posterior on the move with the largest PPMI for that error type, and thereafter by posteriors on moves with smaller PPMI values, in descending order of PPMI. Second, from at some point, the posteriors are dominated by Focus. The phenomenon is consistent across all error types except for *misunderstanding of the question* whose PPMI values are very small (0.07 for Generic and 0 for other tutor moves). For generated tutor candidates from *misunderstanding of the*

question context, the preference pairs do not follow the tutor moves; Generic is the top label of only 5 of its 13,997 candidates, all scores lie between 0.0001 and 0.07, and 98.6 % of its pairs are separated by less than 0.001. Appendix A.1 shows the same ranking procedure for one dialogue context from each of the seven student error types, selected from the 693 training contexts so that the preference scores and move posteriors can be compared across error types in how they reflect the PPMI utilities.

Score Margins of the Constructed Pairs

Figure 5.8 shows the mean normalized score margins for the first 30 preference pairs within each dialogue context, grouped by student error type. The analysis includes all 693 dialogue contexts in the training split of the aligned dataset. Because preference scores are bounded above by the maximum PPMI value for the corresponding student error type, we normalize the score margin of pair j in context i as $\Delta_{i,j} / \max_m \text{PPMI}(m, e_i)$, where e_i denotes the student error type associated with context c . At each pair index j , we then average the normalized margins across all contexts with the same student error type. The resulting mean normalized margin decreases monotonically with pair index for every student error type.

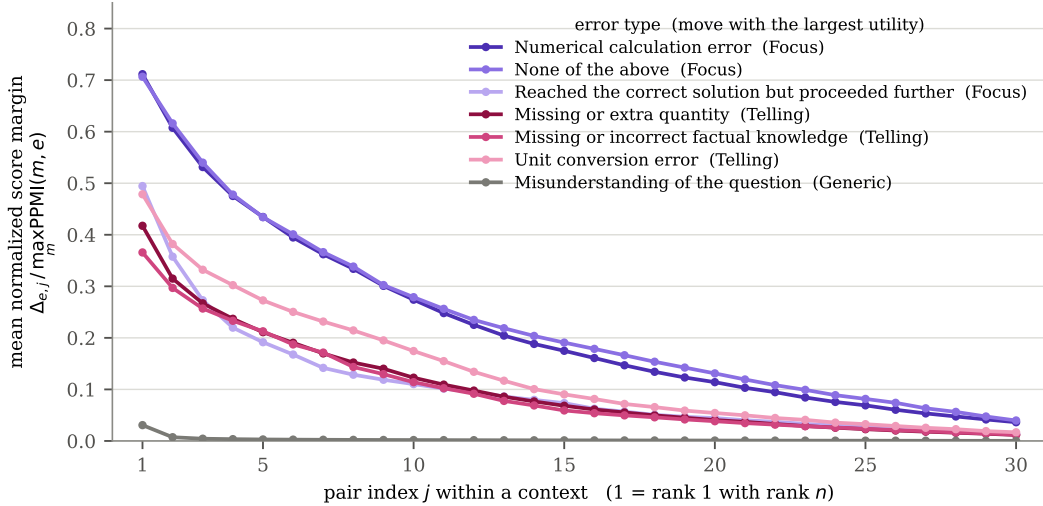


Figure 5.8.: Mean normalized score margins by pair index j , averaged at each index across dialogue contexts with the same student error type in the training split of the aligned dataset (693 contexts in total). Only pair indices $j = 1, \dots, 30$ are shown.

5.3. Pedagogical Alignment Results

This section reports the results of the two-stage alignment pipeline of Section 4.5. Five model variants are compared: the base Llama-3-8B-Instruct model, the SFT model, and the three DPO models trained on the top 10%, 30%, and 50% of preference pairs ranked by score margin within each student error type (the DPO 10%, DPO 30%, and DPO 50% models). The base model is not trained on any of the data of this thesis. All variants generate one next-turn tutor response for each of the 298 dialogue contexts in the test split of the aligned dataset, and every response is annotated by gpt-4.1-mini as the LLM judge along the eight dimensions of Table 4.10. The metric used throughout the section is the Desired Annotation Match Rate (DAMR) of Section 4.5.4. An intermediate label such as *To some extent* counts as a non-match. Sections 5.3.1 and 5.3.2 address Research Question 2, overall and within each student error type. Section 5.3.3 relates the tutor moves encoded in the selected preference pairs of each error type to the behavior of the DPO models and addresses Research Question 3.

5.3.1. Overall Pedagogical Quality

Figure 5.9 reports the DAMR of the five model variants on each of the eight dimensions, and Figure 5.10 shows the full distribution of judge labels behind these rates.

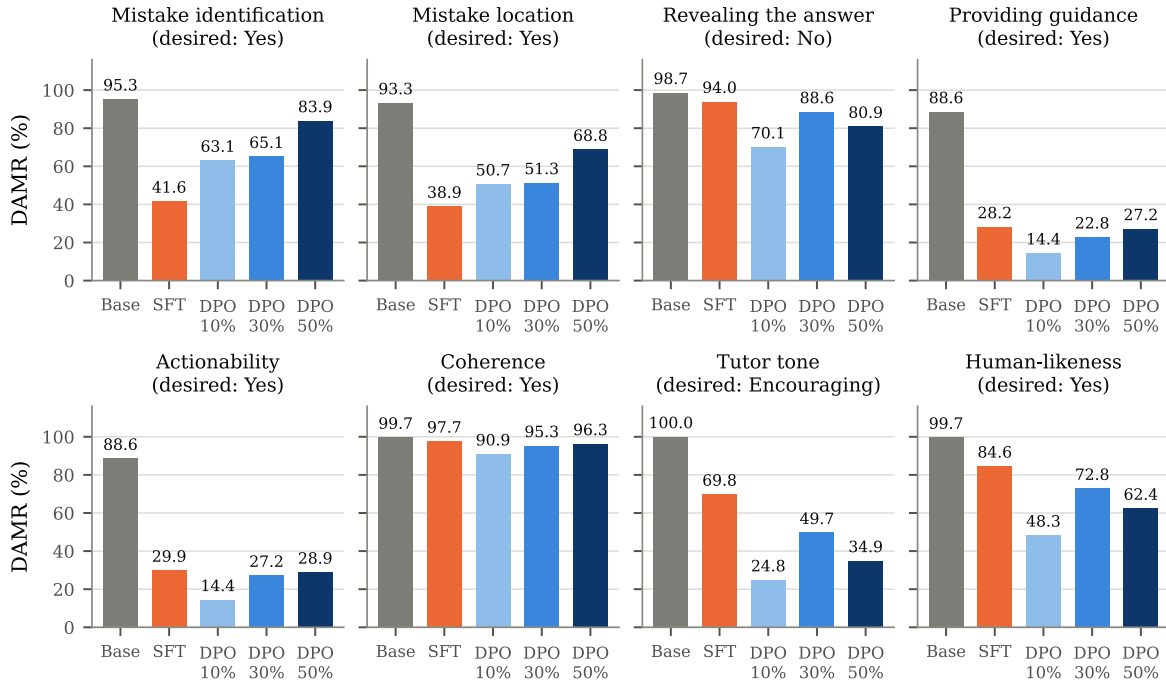


Figure 5.9.: DAMR (%) of the base, SFT, and DPO models on the eight dimensions of the pedagogical evaluation taxonomy, computed over the 298 test contexts of each model. *To some extent* counts as a non-match.

Base and SFT Model

The base model reaches the highest DAMR on all eight dimensions, between 88.6% (providing guidance, actionability) and 100% (tutor tone), as shown in Figure 5.9. The SFT model is below the base model on every dimension. The largest decreases are on mistake identification, mistake location, providing guidance, and actionability, each falling by more than 50%. Tutor tone and human-likeness also decline, whereas coherence and revealing the answer remain close to the base model.

DPO Models

Compared with the SFT model, DPO does not move all eight dimensions in the same direction (Figure 5.9). The clearest gains are on mistake identification and mistake location: both improve with the selection ratio and the DPO 50% model is the best fine-tuned model (i.e., of the SFT and DPO models) on these dimensions. In six dimensions, DPO models are below the SFT model: revealing the answer, human-likeness, tutor tone, coherence, and actionability. Among the three DPO models, the DPO 10% model has the lowest DAMR on all eight dimensions; the DPO 50% model is highest on mistake identification, mistake location, providing guidance, actionability, and coherence; and the DPO 30% model is highest on revealing the answer, tutor tone, and human-likeness.

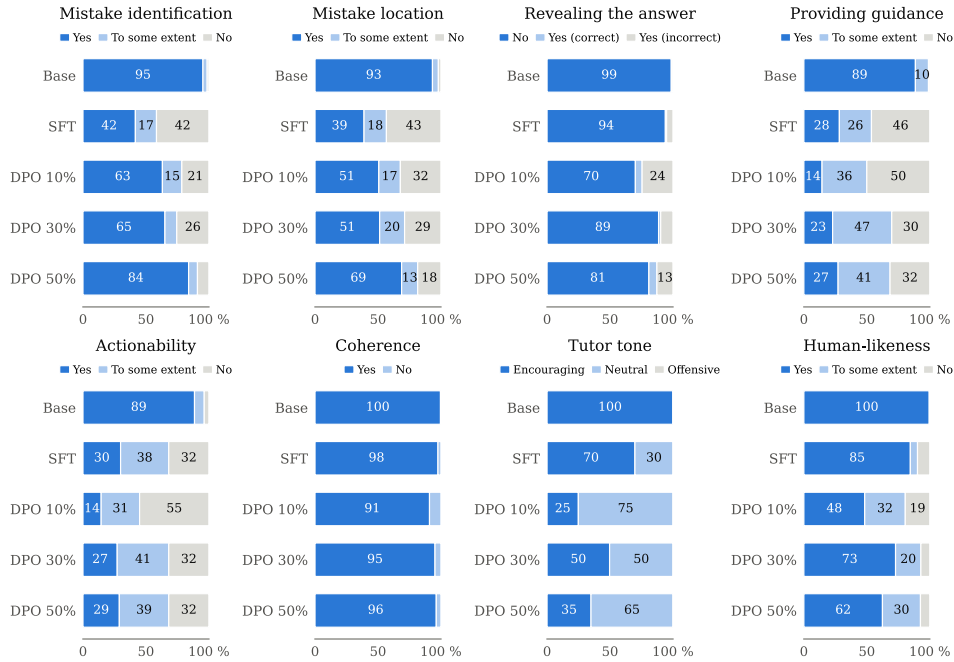


Figure 5.10.: Distribution of the judge labels per dimension and model (share of the 298 responses, rounded to whole percent; segments below 10% are not labeled). The first legend entry of each panel is the desired label, so its segment equals the DAMR of Figure 5.9.

Judge Label Distribution

Figure 5.10 adds three observations that the match rates alone do not show. First, no response of any model is labeled *Offensive*: every non-match on tutor tone is a *Neutral* label. Third, on providing guidance, the DPO 30% and 50% models move responses from *No* to *To some extent* rather than to *Yes*: partial guidance rises (76 responses for the SFT model, 140 and 123 for the DPO 30% and 50% models), while fully correct guidance does not (84, 68, and 81 responses). On actionability, the number of *No* responses stays at the SFT level (95 for the SFT, DPO 30%, and DPO 50% models), and the shift is from *Yes* to *To some extent*. The DPO 10% model instead has the most responses without guidance and without a clear next step. On mistake identification the picture is reversed: responses move from *No* toward *Yes* from the SFT model to the DPO 50% model.

5.3.2. Variation across Student Error Types

Research Question 2 asks how the changes in the pedagogical quality of tutor responses following preference optimization vary across student error types. Figure 5.11 reports the DAMR of every model within each of the seven error types of the test split of the aligned dataset, and Figure 5.12 shows the change of each DPO model relative to the SFT model, which is the reference policy of all three DPO runs. The error types have between 15 and 85 test contexts (Table 4.9).

Mistake Identification and Mistake Location

Figure 5.11 shows the DAMR within each student error type for the five model variants and the eight dimensions. On mistake identification and mistake location dimensions, the gains from the SFT model to the DPO models are noticeable in every error type. Figure 5.12 shows the gains in these two dimensions based on the SFT model. On mistake identification, the DPO 50% model improves from +19 points (missing or incorrect factual knowledge) to +57 points (none of the above). The DPO 30% model gains range from +9 to +38 points. The DPO 10% model shows gains in five error types and is unchanged for missing or incorrect factual knowledge and *unit conversion error*, where the SFT model already has the highest DAMR scores (60% and 53%, respectively). Mistake location follows the similar pattern and values: the DPO 50% model improves in all seven error types whose difference ranges +14 to +43 points, whereas the DPO 10% model is below the SFT model for missing or incorrect factual knowledge (−12) points and *unit conversion error* (−7) points. Within every error type, the DPO 50% model has the highest DAMR of the three DPO models on both dimensions.

Revealing the Answer, Tutor Tone, and Human-Likeness

The decreases on these dimensions from the SFT model to the DPO models are also noticeable across student error types. The DPO 10% model reveals the answer more often than the SFT model in all seven error types (−13 to −31 points), is less encouraging in all seven student error types (−14 to −55 points), and less human-like in all seven student error types (−23.1 to −42

5.3. Pedagogical Alignment Results

	Mistake identification (desired: Yes)					Mistake location (desired: Yes)					Revealing the answer (desired: No)					Providing guidance (desired: Yes)				
Numerical calculation error ($n = 38$)	97	39	68	63	84	97	37	61	50	71	100	95	68	84	71	95	24	21	29	26
Missing or extra quantity ($n = 71$)	94	35	59	58	75	92	31	49	45	58	99	93	66	87	82	85	21	13	15	24
Missing or incorrect factual knowledge ($n = 42$)	98	60	60	69	79	95	55	43	50	69	98	86	64	83	79	88	40	10	24	33
Misunderstanding of the question ($n = 85$)	95	38	62	65	87	93	38	47	52	69	98	98	75	92	82	87	31	11	22	26
None of the above ($n = 26$)	100	35	81	73	92	100	35	65	58	77	100	96	65	100	81	96	27	31	38	23
Reached the correct solution but proceeded further ($n = 21$)	90	48	62	76	95	90	43	57	67	86	100	95	76	86	86	90	24	19	19	43
Unit conversion error ($n = 15$)	87	53	53	67	93	80	47	40	53	73	100	93	80	87	93	87	33	7	20	20
	Base	SFT	DPO	DPO	DPO	Base	SFT	DPO	DPO	DPO	Base	SFT	DPO	DPO	DPO	Base	SFT	DPO	DPO	DPO
			10%	30%	50%			10%	30%	50%			10%	30%	50%			10%	30%	50%
	Actionability (desired: Yes)					Coherence (desired: Yes)					Tutor tone (desired: Encouraging)					Human-likeness (desired: Yes)				
Numerical calculation error ($n = 38$)	95	24	21	29	29	100	100	95	92	95	100	76	21	58	34	100	92	50	76	63
Missing or extra quantity ($n = 71$)	85	25	13	21	25	100	97	89	99	99	100	66	23	45	35	99	80	42	72	61
Missing or incorrect factual knowledge ($n = 42$)	88	40	10	33	33	100	98	88	98	93	100	74	19	48	40	100	81	48	74	60
Misunderstanding of the question ($n = 85$)	87	32	11	27	28	100	100	92	96	96	100	76	24	49	33	100	89	48	72	62
None of the above ($n = 26$)	96	27	31	38	27	100	100	88	85	96	100	65	35	62	35	100	85	62	77	69
Reached the correct solution but proceeded further ($n = 21$)	90	24	19	24	43	95	100	90	95	95	100	57	38	38	43	100	81	52	62	81
Unit conversion error ($n = 15$)	87	40	7	20	20	100	73	100	93	100	100	47	33	53	20	100	73	47	80	40
	Base	SFT	DPO	DPO	DPO	Base	SFT	DPO	DPO	DPO	Base	SFT	DPO	DPO	DPO	Base	SFT	DPO	DPO	DPO
			10%	30%	50%			10%	30%	50%			10%	30%	50%			10%	30%	50%

Figure 5.11.: DAMR (%) within each student error type of the test split, for the five model variants and the eight dimensions. Rows give the number of test contexts per error type. Cell values are rounded to whole percent.

5.3. Pedagogical Alignment Results

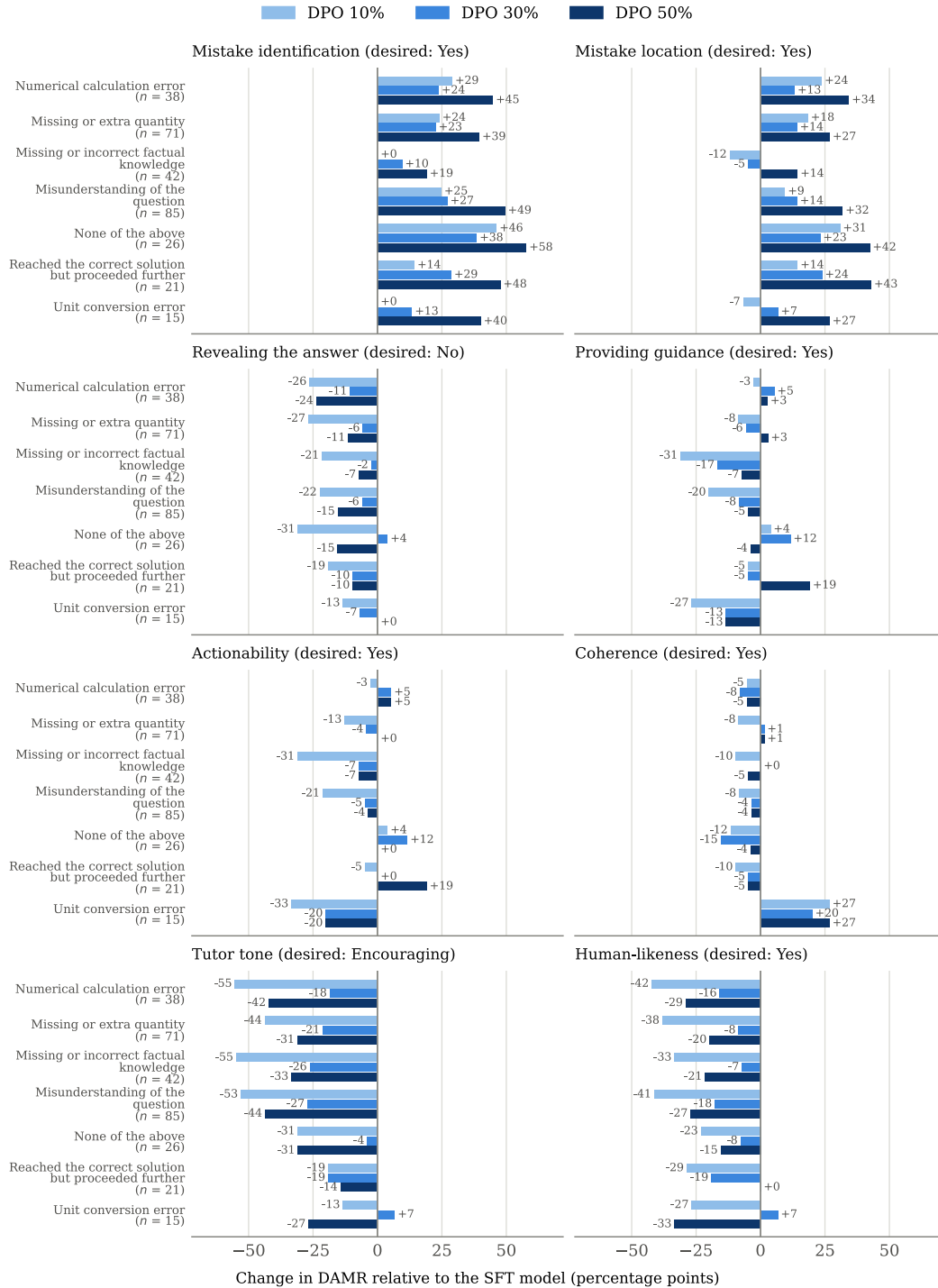


Figure 5.12.: Change in DAMR of the three DPO models relative to the SFT model, in percentage points, within each student error type and dimension. Positive values are improvements toward the desired label.

points). DPO 50% also reveals the answer often than the SFT model in 6 student error types (it recovers in *unit conversion error*), less speaks like a tutor in all error types and like a human in six error types (unchanged for reached the correct solution but proceeded further) based on the evaluation taxonomy. The DPO 30% model shows the smallest decrease on revealing the answer in most student error types (and improves for none of the above by +4 points); on tutor tone and human-likeness it also decreases slightly relative to the SFT model in most student error types, except for *unit conversion error*, where tutor tone and human-likeness increase by 6 and 7 points, respectively. The largest decrease is observed in tutor tone from SFT model to the DPO 10% model for the numerical calculation error (−55), missing or incorrect factual knowledge (−55), and misunderstanding of the question (−52).

Providing Guidance, Actionability, and Coherence

On providing guidance and actionability, the DPO 30% and 50% models change little relative to the SFT model, and the changes are in both directions (between −20 and +19 points). The largest gains occur for reached the correct solution but proceeded further (+19 points on both dimensions for the DPO 50% model), and the largest decreases are observed for *unit conversion error* (−13 and −20.0 points) for the DPO 50% model on both dimensions. The DPO 10% model decreases on both dimensions in six error types (except for none of the above), and drops most for missing or incorrect factual knowledge (−31 on both) and for unit conversion (−26 and −33). Coherence is relatively consistent across all model variants and student error types; it changes by at most 15 points in six error types; the exception is unit conversion, where the SFT model has the lowest coherence of all variants.

5.3.3. Student Error Types behind the Steering

Research Question 3 asks which student error types exert the influence when a preference-optimized model is steered. The preference pairs were selected separately for each student error type (Section 4.4). Each error type therefore has its own set of pairs, and these sets can express different preferences. If the DPO models learned from each set separately, they would behave differently in the test contexts of different error types from the test split of the aligned dataset. This section examines whether these learned behaviors appear in the preference-optimized models.

Tutor-Move Preferences by Student Error Type

Each pair has a chosen and a rejected response, and the tutor-move classifier assigns every response a posterior for each move, which is used to compute the preference scores (Section 4.4). For each student error type, Figure 5.13 shows the difference between the average posterior of the chosen responses and that of the rejected responses across tutor moves. A positive value means that the chosen responses of this error type carry more of the move than the rejected ones, that is, the pairs have more property of that move. Two groups appear, and the separation of the groups is the same across the three selection ratios ρ .

5.3. Pedagogical Alignment Results

		DPO 10% pairs ($\rho = 0.1$) pairs				DPO 30% pairs ($\rho = 0.3$) pairs				DPO 50% pairs ($\rho = 0.5$) pairs			
pairs prefer Focus	Numerical calculation error	+0.70	-0.26	-0.43	242	+0.47	-0.22	-0.24	727	+0.35	-0.17	-0.18	1,213
	None of the above	+0.70	-0.41	-0.27	168	+0.48	-0.30	-0.17	505	+0.36	-0.23	-0.12	842
	Reached the correct solution but proceeded further	+0.52	-0.04	-0.46	137	+0.27	-0.04	-0.22	412	+0.21	-0.04	-0.16	687
pairs prefer Telling	Missing or extra quantity	-0.40	-0.09	+0.48	469	-0.19	-0.07	+0.26	1,409	-0.09	-0.08	+0.17	2,350
	Missing or incorrect factual knowledge	-0.55	+0.03	+0.52	271	-0.35	+0.09	+0.27	815	-0.30	+0.13	+0.17	1,359
	Misunderstanding of the question	-0.49	+0.17	+0.31	557	-0.34	+0.14	+0.19	1,671	-0.26	+0.12	+0.14	2,786
	Unit conversion error	-0.57	+0.02	+0.55	96	-0.37	+0.06	+0.31	289	-0.28	+0.08	+0.20	482
		Focus	Probing	Telling		Focus	Probing	Telling		Focus	Probing	Telling	
		Tutor move				Tutor move				Tutor move			

Figure 5.13.: Each cell is the difference between the average posterior of the chosen responses and that of the rejected responses for the tutor moves Focus, Probing, and Telling, computed on the training split of the preference dataset at $\rho \in \{0.1, 0.3, 0.5\}$ and grouped by student error type. Positive values (blue) mean that the chosen responses carry more of the move, negative values (red) mean that they carry less. Generic is omitted (absolute difference is at most 0.02). Rows are ordered by group: the three Focus-favored error types above the dashed line, the four Telling-favored error types below it. The number to the right of each panel is the number of training pairs of the error type; the training/validation split is given in Table 4.8.

- **Focus-favored group** (numerical calculation error, none of the above, reached the correct solution but proceeded further): the pairs prefer Focus and disfavor Telling.
- **Telling-favored group** (the other four types): the pairs prefer Telling and disfavor Focus.

The Telling-favored group is much larger: it holds 71.8% of the training pairs at every ratio and most of them are from misunderstanding of the question and missing or extra quantity. Here, attention should be given to misunderstanding of the question; it has a PPMI value of 0.07 with 0s for all other tutor moves (Figure 5.6).

The values are strongest at $\rho = 0.1$ (for example +0.70 for Focus in three Focus-favored error types, and from +0.70 to +0.52 for Telling in the four others) and smaller at $\rho = 0.3$ and 0.5, because the pairs added at larger ratios have smaller score margins. For six out of seven error types, this grouping matches the PPMI values in Figure 5.6. For misunderstanding of the question, only Generic has a positive PPMI weight. Nevertheless, the chosen responses have a higher mean Telling posterior than the rejected responses, even though Telling contributes nothing to the preference score for this error type.

Expected Patterns of Error-Type-Specific Steering

The two groups are rewarded for opposite things with respect to Telling. Of the eight dimensions, only revealing the answer separates the two moves by definition, although this distinction is imperfect given the discrepancy between the evaluation taxonomy (Table 4.10) and the MathDial tutor-move taxonomy (Table 4.1). Despite this limitation, the test is made on revealing the answer. If the DPO models followed the signal of each error type, they would reveal the answer more often in the test contexts of the Telling-favored error types, and not in the test contexts of the Focus-favored error types.

Figure 5.14 compares the behavior of the DPO models trained on the pairs of Figure 5.13 between the Telling-favored and the Focus-favored group, measured in the test contexts of the corresponding error types using DAMR. The figure takes the DAMR changes of these models relative to the SFT model (Figure 5.12) and, for each DAMR dimension and each DPO model, shows the changes of the four Telling-favored error types as a grey range and the changes of the three Focus-favored error types as individual markers. If the steering were error-type-specific, the markers would lie on the opposite side of the grey range on the dimensions related to Telling.

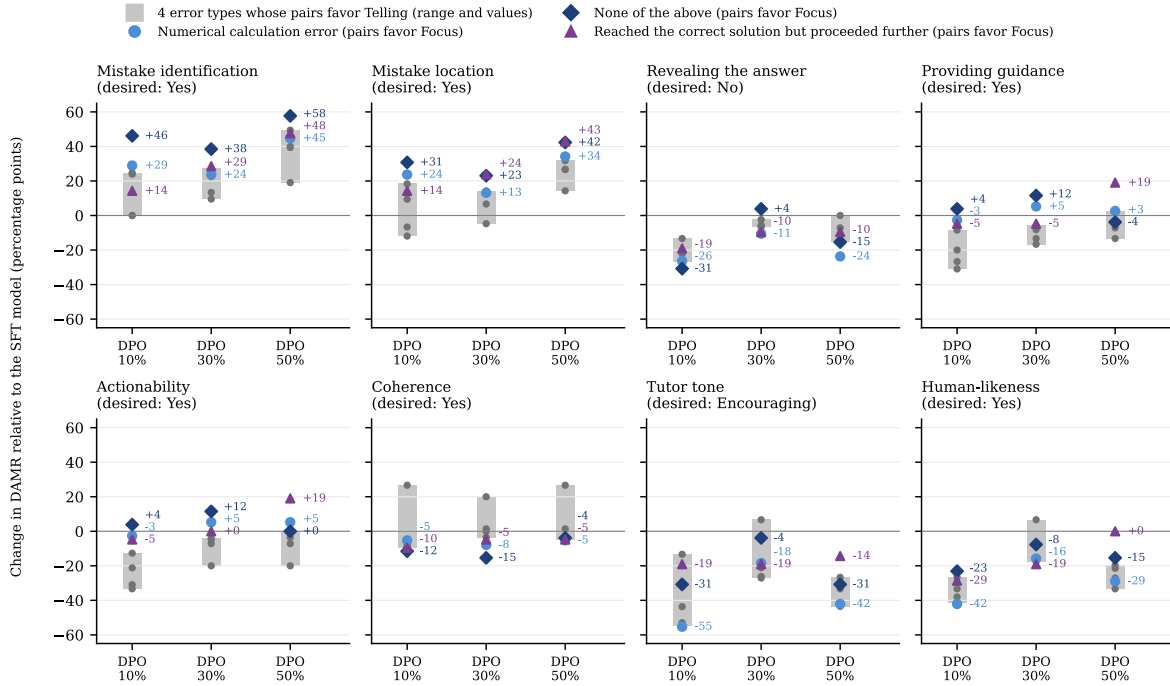


Figure 5.14.: Change in DAMR of the three DPO models relative to the SFT model, in percentage points, with the error types grouped by the move their pairs prefer (Figure 5.13). Grey bar and dots: range and values of the four Telling-favored error types; markers: the three Focus-favored error types. The values are those of Figure 5.12.

Observed Steering across Preference Groups

The DAMR changes do not show a consistent separation between the two preference groups (Figure 5.14). On revealing the answer, three Focus-favored error types show decreases under the DPO 10% and 50% models, indicating that these models reveal the final answer more often than the SFT model. For the DPO 10% model, the decreases are 26.3 percentage points for numerical calculation error and 30.8 points for none of the above, and 19.0 points for reached the correct solution but proceeded further, compared with 13.3–26.8 points among the Telling-favored error types. The DPO 30% model produces a mixed pattern: numerical calculation error and reached the correct solution but proceeded further decrease by 10.5 and 9.5 points, whereas none of the above improves by 3.8 points. Thus, the error types whose pairs favor Focus do not preserve the distinction expected from their training preferences.

Tutor tone and human-likeness show changes shared across the two groups. The three Focus-favored error types decline on these dimensions under all three DPO models (the only exception is human-likeness for reached the correct solution but proceeded further under the DPO 50% model, which is unchanged), as do most Telling-favored error types. The magnitudes vary, but the opposing preferences of the groups are not accompanied by a consistent separation on these dimensions.

Some differences appear on mistake identification, providing guidance, and actionability. On mistake identification, two of the three Focus-favored error types, numerical calculation error and none of the above, have the largest gains under the DPO 10% model, at 28.9 and 46.2 points, compared with at most 24.7 points among the Telling-favored error types; the third, reached the correct solution but proceeded further, gains 14.3 points and lies within the Telling-favored range. At the larger selection ratios, none of the above retains the largest gain, whereas the other two Focus-favored error types fall within or close to the Telling-favored range. On providing guidance and actionability, numerical calculation error and none of the above are the only error types with positive changes under the DPO 30% model, gaining 5.3 and 11.5 points on both dimensions; reached the correct solution but proceeded further changes by –4.8 and 0.0 points. This advantage is not maintained under the DPO 50% model. With 38 and 26 test contexts, respectively, the DPO 30% gains correspond to net increases of two and three responses matching the desired label on each dimension.

The predominant shift toward revealing answers is broadly consistent with the preferences of the Telling-favored group, which accounts for 71.8% of the training pairs (Table 4.8). However, the present design does not isolate the contributions of individual error types or distinguish the influence of group size from the content of the preference pairs.

6. Discussion

This chapter highlights the main findings of the experiments in Chapter 5 and then collects the limitations of the pipeline, including those noted in Chapter 4, together with the changes that would address them. The research questions themselves are answered in Chapter 7.

6.1. Findings

The Input Representation Matters More than the Encoder or the Objective

The classifier grid varies three factors: the encoder, the input representation, and the training objective (Section 4.3.3). Of the three, the input representation separates the configurations most strongly (Table 5.1). With the Tutor-turn only representation, every combination of encoder and objective reaches a higher macro-F1 and a higher macro one-vs-rest ROC-AUC than any configuration with the Preceding-1 or Preceding-2 representation. Adding the preceding dialogue turns, with speaker prefixes, therefore does not improve tutor-move classification. Part of this gap comes from the data rather than from the model. The Tutor-turn only representation uses both labeled tutor utterances of each dialogue, whereas Preceding-1 and Preceding-2 use only the last one (Section 4.3.1). It therefore has twice as many labeled examples, and it contains many more Generic utterances, most of which are formulaic dialogue openers (Table 4.3). Excluding Generic from the metrics lowers the absolute values but does not change the ordering of the three representations under either encoder (Figure 5.2). The advantage of the Tutor-turn only representation is thus not an artifact of the Generic class. The difference in the number of labeled examples, however, was not controlled, so the comparison is between complete experimental configurations rather than a controlled estimate of the effect of dialogue context. Within this grid, the single tutor utterance is the most useful input for the classifier, and it is the unit on which all downstream scoring operates.

AUC Maximization Helps the Minority Move; Domain Pretraining Does Not Guarantee a Gain

The training objective matters most for the minority move. Telling is the rarest tutor move in the aligned dataset (Table 4.3). Among the three objectives, AUC maximization achieves the highest macro-F1 and macro one-vs-rest ROC-AUC on average. Its clearest effect is on Telling: it achieves a higher Telling F1 than weighted cross-entropy in most encoder–representation combinations (Figure 5.3). This agrees with J. Lin et al. (2023), who reported the same advantage for educational dialogue-act classification under low-resource and imbalanced conditions. A second observation is more important for the downstream task. The Telling F1 is

zero in several configurations, but the Telling one-vs-rest ROC-AUC stays well above chance in all of them. The classifiers therefore still rank Telling utterances above the other moves even when the multiclass arg max never selects Telling. The error-conditioned scoring method uses the posterior $p_{\theta}(m \mid r)$ rather than the predicted label (Section 4.4.2). This ranking signal for Telling therefore reaches the preference dataset even though the hard label rarely does. Domain-adaptive pretraining, in contrast, produced no consistent gain. The domain-adapted RoBERTa raises macro-F1 in some input-representation-objective combinations but lowers macro one-vs-rest ROC-AUC in most of them, and the best classifier uses the original RoBERTa-base encoder (Table 5.1). The domain corpus built from QATD2k and MathMentorDB differs from the aligned dataset in register and in mathematical level (Section 4.3.2). Adapting to a large

The Scoring Method Turns Observed Associations into Error-Conditioned Preferences

Within each context, candidates are ranked by the weighted sum of their predicted tutor-move probabilities, using the error-specific PPMI values as weights. For numerical calculation errors, only Focus has a positive weight, so candidate ranking depends solely on the predicted probability of Focus. For factual-knowledge errors, Focus, Probing, and Telling all contribute to the score, with Telling receiving the largest weight. In the two contexts illustrated in Figure 5.7, these weights yield different ranking patterns: candidates with high Focus probabilities occupy the highest ranks in the calculation-error context, whereas candidates dominated by Telling occupy the highest ranks in the factual-knowledge context. These examples illustrate how the scoring method translates observed associations into pairwise preferences; they do not establish the pedagogical superiority of the chosen responses. The same formulation could, in principle, be applied to other annotated student attributes, but its usefulness beyond error types remains untested.

The Observed Associations Are Interpretable under the Move Taxonomy

Several of the PPMI values in Figure 5.6 have a plausible reading under the tutor-move definitions of Table 4.1. For *numerical calculation error*, Focus is the only move with a positive PPMI. Focus covers moves that direct the student’s attention to a specific step or calculation, such as asking the student to recalculate a value, which is a natural response to a calculation error. For *missing or extra quantity*, *missing or incorrect factual knowledge*, and *unit conversion error*, Telling has the largest PPMI. Telling covers moves that reveal a strategy or the answer, such as stating the missing quantity or the needed fact, which matches what these three error types lack. Generic has a low PPMI for every error type. This follows from how PPMI is computed: Generic is the most frequent first move in every error type and its share is similar across error types (Section 5.2.1), so its co-occurrence is close to what independence predicts, and PPMI measures only the deviation from independence. For *missing or incorrect factual knowledge* and for *reached the correct solution but proceeded further*, both Focus and Probing have positive PPMI. These are the two moves that the selected classifier confuses most often (Figure 5.5) and that Macina, Daheim, Pal Chowdhury, et al. (2023) report as the hardest to distinguish, so their joint association is consistent with the known overlap between the two moves. All of these

readings interpret empirical co-occurrence in the training split. They do not show that the associated move is the effective one for

DPO Recovers What SFT Lost, and the Selection Ratio Decides How

The base model has the highest DAMR on all eight dimensions (Figure 5.9). SFT on the residual MathDial records adapts the model to the response format of human tutors (Section 4.5.2), but it lowers the DAMR on every dimension. The largest decreases are on mistake identification, mistake location, providing guidance, and actionability. DPO then moves the eight dimensions in different directions. It recovers two of the four dimensions on which SFT loses the most, mistake identification and mistake location. On these two dimensions, the DAMR increases with the selection ratio, and the DPO 50% model is the best fine-tuned model in every student error type (Figure 5.11). On revealing the answer, tutor tone, human-likeness, coherence, providing guidance, and actionability, however, the DPO models remain below the SFT model. The selection ratio also determines the size of these side effects. The DPO 10% model, trained on the pairs with the largest score margins, has the lowest DAMR on all eight dimensions. The DPO 30% model reveals the answer least often and has the best tutor tone and human-likeness of the three DPO models. The DPO 50% model is the best of the three on mistake identification, mistake location, providing guidance, actionability, and coherence. The DAMR counts only whether a response received the desired label. The full distribution of the judge labels (Figure 5.10) therefore shows two things that the DAMR hides. First, when a DPO model reveals the final answer, the judge labels the revealed answer as *Yes (incorrect)* far more often than as *Yes (correct)*: the DPO models give away answers, and the answers they give away are mostly wrong. Second, on providing guidance and actionability, the DPO models move responses from *No* to *To some extent* rather than from *No* to *Yes*. The share of responses with partial or incomplete guidance rises, while the share with fully correct guidance stays at the level of the SFT model. The DAMR on these two dimensions therefore does not improve even though fewer responses give no guidance at all.

The Score Margin Selects Pairs but Never Weights Them

The score margin of a preference pair is the difference between the error-conditioned scores of its chosen and rejected responses (Section 4.4.3). In this thesis, the score margin is used only to select pairs. The DPO objective of Section 4.5.3 uses only the chosen–rejected order of a pair. A pair with a score margin of 0.0001 and a pair with a score margin of 0.7 therefore receive the same weight in training. This design has two consequences that are visible in the results. The first concerns the pairs with the largest score margins. Within a dialogue context, the score margin falls steeply with the pair index: the mean normalized margin of the first pair is 0.37–0.71 in six of the seven error types, but it is at most 0.04 by the thirtieth pair (Figure 5.8). The smallest selection ratio therefore keeps mainly the first few pairs of each context. In these pairs, the chosen and rejected responses differ most in their Focus and Telling posteriors (Figure 5.13). The DPO 10% model, trained on these pairs, has the lowest DAMR on all eight dimensions among the DPO models (Figure 5.9). As the selection ratio grows, pairs with smaller score margins are added, and the difference between the chosen

and rejected posteriors shrinks. The DPO 30% and 50% models stay closer to the SFT model on revealing the answer, tutor tone, and human-likeness. This comparison, however, does not isolate the effect of the margin magnitude from that of the training-set size, because a larger selection ratio also means more update steps (Section 4.5.3). The second consequence concerns *misunderstanding of the question*. Pairs are selected within each error type, so this error type contributes the same share of pairs at every selection ratio. Its pairs make up 28.7 % of the preference dataset, but 98.6 % of them have a score margin below 0.001 (Section 5.2.2). More than a quarter of every DPO training set therefore consists of pairs whose chosen and rejected responses have nearly identical error-conditioned scores, and these pairs are weighted like all others. Prior work suggests that pairs with larger quality gaps improve alignment (Pattnaik et al., 2024). Under the error-conditioned score, this expectation does not hold. A large score margin marks a strong contrast in tutor-move posteriors, not a more reliable preference.

The Steering Follows the Majority of the Pairs, Not the Error Type

The preference pairs were selected separately for each student error type (Section 4.4). The pairs of different error types therefore encode different preferences. Figure 5.13 shows two groups. In the Focus-favored group, numerical calculation error, none of the above, and reached the correct solution but proceeded further, the chosen responses have a higher average Focus posterior and a lower average Telling posterior than the rejected responses. In the Telling-favored group, the other four error types, the chosen responses have a higher average Telling posterior and a lower average Focus posterior. The Telling-favored group holds 71.8% of the training pairs at every selection ratio. If the DPO models had learned the preference of each error type separately, their behavior would differ between the two groups. In particular, they would reveal the answer more often than the SFT model in the test contexts of the Telling-favored error types, but not in the test contexts of the Focus-favored error types. Figure 5.14 shows that this separation does not occur. The DPO models reveal the answer more often than the SFT model in the test contexts of both groups. *Numerical calculation error*, a Focus-favored error type, is among the error types with the largest increases under every DPO model. Tutor tone and human-likeness also decrease relative to the SFT model in the three Focus-favored error types and in most Telling-favored error types. Some differences between the two groups appear on mistake identification, providing guidance, and actionability. However, these differences do not persist across the three DPO models. The predominant change, revealing the answer more often, matches the preference of the Telling-favored group, which holds most of the training pairs. The DPO models therefore learn the preference that dominates the training pairs. They do not learn a preference that depends on the student error type. Whether the size of the Telling-favored group or the content of its pairs drives this result cannot be separated in the present design.

6.2. Limitations and Future Work

Source Data and Annotation Transfer

Few datasets carry both fine-grained student-state and tutor-move labels. The aligned dataset was therefore assembled by transferring MathDial’s tutor-move labels to StepVerify through string similarity rather than by re-annotation (Section 4.2.2). The matching imposes no turn-position constraint, so some accepted matches took their label from a MathDial tutor turn at a different position, including a subset of the first-turn labels that feed the PPMI table (none of them Telling, so the effect is confined to the Focus and Probing cells). MathDial itself labels identical formulaic openers inconsistently: openers such as *“walk me through your solution”* carry different labels across dialogues, many first-turn matches involve such texts, and a few of the first-turn Telling labels are this opener, so the largest utilities partly rest on mislabeled turns. The student utterances are moreover synthetic, and no human verified the re-assembled labels. Future work should constrain matching to the same turn index and have annotators adjudicate a sample of transferred labels before the association is estimated.

Text Normalization

Text normalization is applied to every utterance before annotation transfer, classifier training, and domain-adaptive pretraining (Section 4.3.1). It lowercases the text and removes all characters that are not letters, digits, or basic punctuation. This removes LaTeX markup, arithmetic operators, and equals signs. As a result, part of the mathematical content of an utterance is lost. A student step such as $3 \times 4 = 12$ survives only as *“3 x 4 12”*, and a division such as $2400 \div 5 = 480$ survives only as *“2400 5 480”*. For the tutor-move classifier, two tutor utterances that differ only in an equation therefore become nearly identical. The limitation is not confined to the classifier. The aligned dataset was built from the normalized text, so the normalized student solutions are also what the candidate-generation prompts, the SFT and DPO training data, the shared inference prompt, and the LLM judge received. None of the student solutions in the test split contains an equals sign, and none of the student turns in the SFT data contains an arithmetic operator. Every model in the pipeline therefore had to identify the student’s mistake from a solution whose operators were missing, and the judge assessed the generated responses against the same damaged solution. A math-aware normalization that protects operators, numerals, and LaTeX spans while still removing emoji, URLs, and markup would remove this problem at every stage.

Truncated Dialogue Context

StepVerify limits each dialogue to three turns and annotates only the initial incorrect solution, so the associations are estimated from the first tutor turn alone and the shared prompt of Section 4.5.3 contains a problem and a single student utterance without dialogue history. Extending the error annotation to later student turns of MathDial would give richer contexts for both the association and the aligned model.

Statistical Strength of the Association and PPMI

The utilities describe tendencies in 693 dialogues rather than an established relationship: a chi-square test on the contingency table of Figure 5.6 gives $\chi^2 = 25.94$ ($df = 18$, $p = 0.101$; 10 of 28 expected counts below 5) and Cramér’s $V = 0.112$, so independence between error type and first tutor move cannot be rejected at the 5 % level. PPMI further amplifies rare cells (Section 4.4.2): the three largest utilities, all for Telling, rest on one to four first tutor turns, and clipping negative PMI discards the information that a move is avoided for an error type. The Telling-led preferences of four error types, and hence the fewer-questions, more-correction behavior of the DPO models, therefore rest on a table whose association is not statistically distinguishable from independence. Future work should apply a minimum-count threshold, smoothing, or a discounted PPMI, and report confidence intervals for the utilities before they are used as a training signal.

Prompt Defects, Prompt Mismatch, and Pair Selection

Candidate generation adapts the Bridge framework of R. Wang et al. (2024), in which the tutor’s strategy and intention are chosen on the basis of the student’s error type. Our adaptation omits the error-inference step because the error types are already annotated, but it does not pass the annotated error type to either prompt (Figure 4.6). Stage 1 therefore chooses the strategy and intention from the problem and the student’s solution alone, and the error type enters the pipeline only at scoring time. Thus the candidate student responses are generated without student error information. The prompt text itself also contains defects: the Stage 1 system message states the task twice, once with a typo, and both stages contain ungrammatical slot sentences. Their effect on the selected strategies and on the generated candidates was not measured. The pairs were then generated with the Bridge Stage 2 prompt, which contains the lesson topic and the strategy–intention pair, but DPO was trained with the reduced prompt of Figure 4.7, following the context-distillation argument of Xie et al. (2024). This argument assumes that the chosen response remains the better response when that information is absent from the prompt; we did not test this assumption. Finally, selecting pairs by score margin keeps the pairs whose chosen and rejected responses differ most in their move posteriors, and the DPO 10% model, trained on the largest-margin pairs, has the lowest DAMR on all eight dimensions (Figure 5.9). Preference data carry construction noise (Deng et al., 2025), so a large margin does not guarantee a correct label. Future work should pass the error type to both stages, correct the prompts, test the preference under the reduced prompt on a held-out sample, and combine margin-based selection with diversity constraints or noise-robust objectives.

Problem Overlap across Splits

The aligned dataset was split into training and test records, and the residual MathDial records were used for SFT (Section 4.1). These splits are disjoint at the level of records: no dialogue appears in more than one split. They are not disjoint at the level of math problems. Both MathDial and StepVerify contain several dialogues for the same GSM8k problem, each with a different student solution. Consequently, of the 298 test contexts, 140 pose a problem that also

appears in the DPO training split, 171 pose a problem that also appears in the SFT data, and only 43 pose a problem that appears in neither. The evaluation therefore measures generalization to unseen dialogues, not generalization to unseen problems. A fine-tuned model may have seen the same problem with a different student solution and a human tutor response during training. We checked whether this overlap inflates the reported results. For each fine-tuned model and each dimension, we compared the DAMR on the 140 test contexts whose problem appears in the DPO training split with the DAMR on the remaining 158 test contexts. The two values differ by at most ten percentage points, and the difference is positive for some dimensions and negative for others; for example, the DPO 50% model reaches 85.7 % mistake identification on the overlapping contexts and 82.3 % on the remaining ones. On the 43 test contexts whose problem appears in neither training set, the ordering of the fine-tuned models on mistake identification and mistake location is the same as on the full test split. The overlap therefore does not appear to inflate the gains reported in Chapter 5, but the results should be read as results on familiar problems. Splitting the data by math problem before any stage of the pipeline would remove this caveat.

Evaluation

The evaluation relies on a single LLM judge, `gpt-4.1-mini`, whose agreement with human annotators was not measured; benchmarks such as MathTutorBench and MRBench could not be used because they draw on MathDial (Section 4.5.4). The judge’s taxonomy is also not the one the preference signal was built on: the signal rewards Telling for three error types, whereas the judge penalizes revealing the answer, so a model that follows the signal can be scored down by design; the base model’s top scores on all eight dimensions, with 51-word responses that always open with praise, suggest that the judge also rewards length and tone; and no dimension measures learning outcomes. Future work should use several judges and a human study with reported agreement, add a move-level evaluation in the MathDial taxonomy, and measure the effect on simulated or real students.

Design of the Error-Type Analysis

To test whether the steering of the DPO models depends on the student error type, Section 5.3.3 groups the seven error types by the tutor move their preference pairs favor, the Focus-favored group and the Telling-favored group, and compares the DAMR changes of the DPO models relative to the SFT model between the two groups. Four properties of this design limit what the comparison can show. First, each DPO model was trained on the pooled pairs of all seven error types. The contribution of a single error type to the trained model can therefore not be isolated, and the effect of the size of the Telling-favored group cannot be separated from the effect of the content of its pairs. Second, of the eight dimensions, only revealing the answer distinguishes Focus from Telling by definition, so the test of error-type-specific steering rests on a single dimension. Third, the error types have between 15 and 85 test contexts, so one response changes the DAMR of an error type by 1.2 to 6.7 percentage points, and small differences between the groups cannot be distinguished from noise. Fourth, the group assignment of one error type does not follow from its utilities: for *misunderstanding of the question*, Telling

has a PPMI of zero, yet its chosen responses carry more Telling posterior mass than its rejected responses (Figure 5.13), because its near-zero margins leave the pairing to the composition of the candidate pool. Future work should train separate DPO models per error type or on balanced subsets, evaluate the generated responses at the level of tutor moves with the classifier so that Focus and Telling can be measured directly, and enlarge the per-type test sets.

7. Conclusion

This thesis investigated whether empirical associations between student error types and tutor moves in tutoring dialogues could be transformed into a preference signal for aligning an LLM-based tutor, and how training on this signal affected the resulting tutor behavior. To answer this, we built an end-to-end pipeline: an aligned dataset of 991 dialogues that pairs StepVerify’s student error types with MathDial’s corresponding tutor-move labels through annotation transfer; a tutor-move classifier selected from 18 configurations of encoder, input representation, and training objective; an error-conditioned scoring method that combines PPMI-based utilities with classifier posteriors to rank 48,839 generated candidate responses and form 24,298 preference pairs; and a two-stage alignment of Llama-3-8B-Instruct via supervised fine-tuning and Direct Preference Optimization. The resulting tutor behavior was evaluated per error type with an LLM judge on the eight dimensions from Maurya et al. (2025b). The three research questions are answered below.

RQ 1. What associations are observed between student error types and tutor moves, and how are these associations reflected in the constructed preference dataset? In the training split of the aligned dataset, Generic is the most frequent first tutor move for every student error type. The frequencies of Focus, Probing, and Telling vary across error types (Section 5.2.1). The PPMI values identify different associations within this distribution. Focus has the highest PPMI for numerical calculation error, none of the above, and reached the correct solution but proceeded further. Telling has the highest PPMI for missing or extra quantity, missing or incorrect factual knowledge, and unit conversion error. For misunderstanding of the question, Generic is the only move with a positive PPMI.

These associations enter the preference dataset through the scoring method. Within each dialogue context, each candidate response receives a score calculated by weighting its tutor-move posteriors with the PPMI values for the student error type. Within each selected pair, the response with the higher score becomes the chosen response, and the other becomes the rejected response. The PPMI values therefore determine which tutor-move profiles receive higher scores for each error type (Section 5.2.2).

However, the underlying associations have limited statistical support. The association between student error type and first tutor move is not statistically significant ($\chi^2 = 25.94$, $df = 18$, $p = 0.101$), and its effect size is small (Cramér’s $V = 0.112$). The largest PPMI values, which favor Telling, are also based on very few first tutor turns (Chapter 6).

A further limitation concerns misunderstanding of the question, the largest error type. Only Generic has a positive PPMI for this error type, but the generated candidates have almost no Generic posterior mass. Their scores are therefore nearly identical, leaving negligible margins

between the chosen and rejected responses in almost all pairs (Appendix A.1). These pairs carry little information about the error type’s associated tutor move.

Thus, the preference dataset encodes the observed PPMI-based associations through its ranking procedure, but the resulting error-conditioned preference signal is limited by weak associations and sparse observations.

RQ 2. How do changes in the pedagogical quality of tutor responses following preference optimization vary across student error types? To answer this question, we compare the DAMR of each DPO model with the DAMR of the SFT model. We make this comparison separately for each dimension of pedagogical quality and for each student error type (Section 5.3.2). The resulting changes do not follow one single pattern. Instead, the dimensions fall into three groups, and the pattern is different in each group.

The first group consists of mistake identification, revealing the answer, tutor tone, and human-likeness. On these four dimensions, the DPO models change in the same direction across student error types, with three exceptions that we describe below. The DAMR increases on mistake identification and decreases on revealing the answer, tutor tone, and human-likeness. Two results show this most clearly. The DPO 50% model improves mistake identification in all seven error types (Figure 5.11). The DPO 10% model scores lower than the SFT model on revealing the answer, tutor tone, and human-likeness in all error types (Figure 5.12). The three exceptions all belong to the DPO 30% model. On revealing the answer, this model scores higher than the SFT model in none of the above. On tutor tone and on human-likeness, it scores higher than the SFT model in unit conversion error. These two error types have the fewest test contexts of all error types: none of the above has 26, and unit conversion error has 15. In a test set of this size, a single response changes the DAMR by several percentage points, and each of these three exceptions amounts to a single response.

The second group consists of mistake location alone. On this dimension, the direction of the change depends on the DPO model. For the DPO 50% model, the DAMR is higher than that of the SFT model in every error type. For the DPO 10% model, the direction is not the same in every error type: this model scores lower than the SFT model in missing or incorrect factual knowledge and in unit conversion error.

The third group consists of providing guidance, actionability, and coherence. On these three dimensions, the changes are small. They also have no consistent direction: for the same DPO model, the DAMR increases in some error types and decreases in others.

The size of the changes differs between error types. Part of this difference comes from the small size of some test sets. Some error types have only a small number of test contexts, and in such a test set a single response changes the DAMR by several percentage points. Small differences between error types therefore provide only limited evidence that the models behave differently on those error types.

Two limitations apply to this answer. First, the pedagogical quality was judged by a single LLM judge. The agreement between this judge and human annotators was not measured, so it is not known whether human annotators would give the same scores. Second, the three DPO models were trained with different selection ratios, and a different selection ratio changes two

things at the same time: the score margin of the training pairs and the number of training pairs. When two DPO models differ in DAMR, we therefore cannot say whether the difference comes from the score margin or from the number of training pairs. The effect of the selection ratio cannot be attributed to either factor alone (Chapter 6).

We therefore answer RQ 2 as follows. On mistake identification, revealing the answer, tutor tone, and human-likeness, the changes vary across student error types in their size but not in their direction; the three exceptions each amount to a single response in a small test set. On mistake location, the direction of the change is the same in every error type for the DPO 50% model, but not for the DPO 10% model. On providing guidance, actionability, and coherence, the changes are small in every error type, and their direction is not consistent. Because several error types have small test sets, differences in the size of the changes between error types should be interpreted with caution.

RQ 3. If a preference-optimized model is pedagogically steered, which student error types exert the influence on this steering? The preference optimization steers the behavior of DPO models but the DPO models do not show consistent pedagogical improvement based on DAMR. Compared with the SFT model, the DPO models have higher DAMR on mistake identification and mistake location, and lower DAMR on revealing the answer, tutor tone, and human-likeness (Section 5.3.1).

To examine whether the steering reflects the preference of each error type, Section 5.3.3 compares the DAMR changes of the Focus-favored and Telling-favored groups relative to the SFT model. The main comparison uses the revealing the answer dimension. A lower DAMR on this dimension means that answers are revealed more often. The result shows no consistent separation between the Focus-favored and Telling-favored groups; lower scores on revealing answer are observed among error types whose preference pairs flavored Focus (Figure 5.14).

The overall shift toward revealing answer is consistent with the preference direction of the Telling-favored group. This group accounts for a majority of training pairs, mainly from two error types: misunderstanding of the question and missing or extra quantity. These the samples from these two error types are plausible sources of the observed steering. However, all training samples were used to train the DPO models, so the current experiment design cannot isolate their individual contributions. Additionally, the result can also not say that whether the observed shift reflects the size of the Telling-favored group or the content of its pairs (Chapter 6).

A further limitation concerns the difference between the tutor-move taxonomy used for training and the taxonomy used for evaluation. The Focus tutor move directs the student to a specific step without giving the answer, whereas Telling reveals the answer on the solution strategy (Table 4.1). Revealing the answer is the dimension closest to this distinction by definition, but it measures whether the final tutor utterances generated from the DPO models reveal the answer. The revealing the answer dimension therefore measures only part of the behavior covered by Telling.

Within this limitation, the results show an overall shift consistent with the Telling-favored group's preference. But this result does not establish which individual error types drove that

or whether the DPO models learned the distinct preferences encoded by individual student error-tutor move associations.

Message of this thesis

This thesis shows how empirical associations between student error types and tutor moves can be turned into a preference signal for an LLM-based tutor. The aligned dataset used for this construction pairs student error types with tutor-move labels from dialogues between human tutors and LLM-simulated students. Training on the resulting preference dataset changes tutor behavior. Compared with the SFT model, the DPO models show an overall shift toward revealing the answer more often. This shift is consistent with the preference of the Telling-favored group, which accounts for most of the training pairs. However, the experimental design and structural issues in the available data limit what can be concluded about individual error types. The results do not establish which error types drove the shift or whether the DPO models learned the distinct preferences encoded for each error type.

The same method may be used to construct preference datasets from observed associations between other student characteristics and tutor behaviors in another context. Educational dialogue datasets with annotations of both student characteristics and tutor behaviors can therefore provide the ingredients for this construction. For example, the TalkMoves dataset contains annotations of both teacher and student talk moves in the same lesson transcripts (Suresh, Jacobs, Harty, et al., 2022). This extension requires a well-curated dataset. Annotators must have verified the student characteristics and tutor behaviors. Their association must also be reliable enough to serve as a utility for preference scoring.

With such data, a preference signal as simple as the one developed here could help adapt LLM tutors to what individual students know, misunderstand, and intend. Whether such adaptation improves student learning remains to be established. Future evaluation could examine whether the student’s next answer is correct, whether a knowledge component is subsequently mastered, or whether performance improves across sessions. The educational value of this approach would ultimately depend on what students learn from the trained tutor.

A. Appendix

A.1. Ranked Candidates for Each Student Error Type

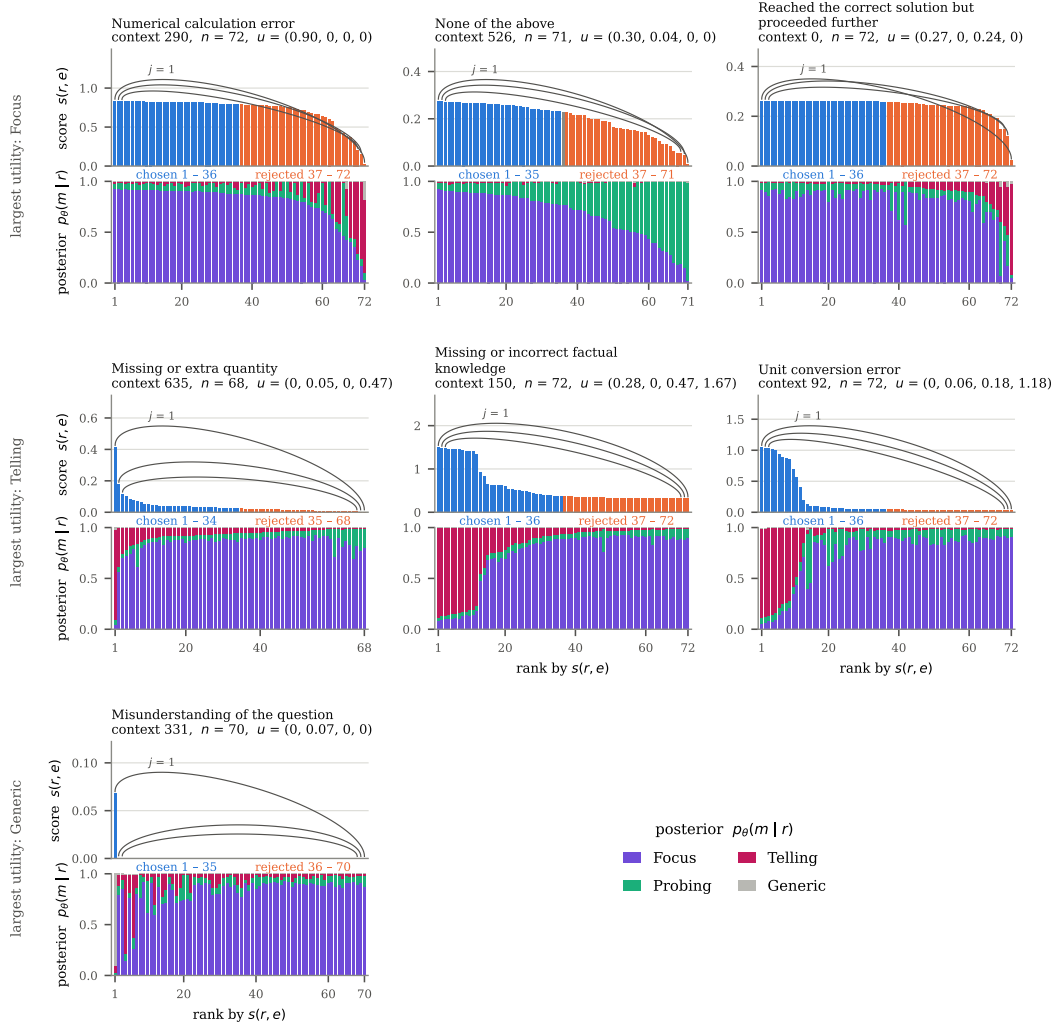


Figure A.1.: One dialogue context per student error type from the 693 training contexts of the aligned dataset. In each block, all candidates are ranked by the error-conditioned score $s(r, e)$ defined in Section 4.4.2. Upper panel: chosen candidates are colored in blue, rejected candidates in orange, and the omitted middle candidate in grey when n is odd. Lower panel: move posteriors $p_\theta(m | r)$ from the selected classifier in Section 5.1, stacked by move. Rows are grouped by the student error type with the largest utility (PPMI). n denotes the number of candidates, and the utility vector lists $u(m, e)$ in the order $u_{\text{Focus}}, u_{\text{Generic}}, u_{\text{Probing}}, u_{\text{Telling}}$.

List of Figures

4.1. Overview of the methodology pipeline, including preference dataset construction using error-conditioned scoring, LLM-based tutor training through supervised fine-tuning and preference optimization, and rubric-based pedagogical evaluation.	20
4.2. An example from the 1,932nd record in MathDial and the 969th record in StepVerify, showing shared and dataset-specific fields. Tutor tutor-move annotations in parentheses appear only in MathDial, while the student error type appears only in StepVerify. Other metadata are omitted for readability.	24
4.3. An example record from the aligned dataset, combining student error annotations from the 969th record in StepVerify with tutor-move labels transferred from the 1,932nd record in MathDial. Only the contents of fields relevant to illustrating the annotation transfer are shown.	26
4.4. An example of a complete tutoring session from QATD2k (Zent et al., 2025). Tutor messages are shown with their talk-move labels in parentheses.	33
4.5. An example of a complete question-answering session from MathMentorDB (Ion, 2026). The student shares a screenshot of a proof problem, three volunteer helpers offer complementary explanations, and the student then closes the channel with .close. Speaker labels follow the source transcript.	34
4.6. Two-stage strategy-and-intention prompts adapted from Bridge (R. Wang et al., 2024) . Stage 1 selects a strategy–intention pair; Stage 2 generates a response conditioned on the selected strategy–intention pair.	42
4.7. Prompt template shared across SFT, DPO, and tutor-response generation on the test set for evaluation. The input consists of a math problem and a student utterance. During SFT, the student response is an utterance extracted from a MathDial dialogue; during DPO and final evaluation, it is the student’s initial incorrect solution.	50
4.8. Example of an SFT input–target pair constructed from a residual MathDial record using the shared prompt template in Figure 4.7. The system instruction and user message form the model input, and the subsequent human-authored tutor utterance is the target response.	51
4.9. Example of a DPO input context–preference pair after context distillation. The system instruction and user message follow the shared prompt template in Figure 4.7. The chosen and rejected tutor responses are retained from the preference pair constructed using the error-conditioned tutor-move scores.	53

4.10.	Prompt template used for the LLM-as-a-Judge evaluation. The system message defines the eight dimensions, their allowed and desired labels, and dimension-specific evaluation rules. The human message supplies the problem, student’s response, optional reference solution, and generated tutor response for each evaluation instance. Adapted from (Maurya et al., 2025b).	59
5.1.	Per-class F1 (top row) and per-class one-vs-rest ROC-AUC (bottom row) by input representation, for both encoders. Bars show the mean over the three training objectives and whiskers the minimum and maximum across them. n gives the number of test examples in the Preceding-1 / -2 and the Tutor-turn only split respectively.	62
5.2.	Macro-F1 (top row) and macro one-vs-rest ROC-AUC (bottom row), each computed over all four tutor moves and over the three substantive moves only (Focus, Probing, Telling), for both encoders. Excluding Generic reduces the absolute differences but preserves the ordering of the input representations under either encoder and either metric. Each bar is the mean over the three training objectives.	63
5.3.	F1 (top) and one-vs-rest ROC-AUC (bottom) for the minority move Telling across all 18 classifier configurations. AUC maximization achieves higher Telling F1 than weighted cross-entropy in five of the six combinations of two encoders and three input representations. One-vs-rest ROC-AUC remains between 0.70 and 0.86 across all 18 configurations.	64
5.4.	Macro-F1 against macro one-vs-rest ROC-AUC for all 18 configurations. The selected classifier (filled circle, AUC maximization) is the maximum on both macro-F1 and one-vs-rest ROC-AUC.	65
5.5.	Selected classifier (RoBERTa-base, Tutor-turn only, AUC maximization). Left: per-class precision, recall, F1, and one-vs-rest ROC-AUC on the test set. Right: the confusion matrix (count and row percentage)	66
5.6.	First-turn error-move co-occurrences (left) and the PPMI values used for scoring (right), estimated from 693 dialogue contexts in the training split of the aligned dataset. Each context contributes one student error type and the tutor-move label of its first tutor utterance.	68
5.7.	All candidates from two dialogue contexts (i.e., different student error types) are ranked by the error-conditioned score $s(r, e)$. Left: <i>numerical calculation error</i> (context 609, $n = 69$ generated candidate tutor utterances); Right: <i>missing or incorrect factual knowledge</i> (context 150, $n = 72$ generated tutor utterances). Top: score of every candidate; chosen candidates are colored in blue, the rejected candidates are in orange, the omitted candidate is in grey when n is odd. The arcs mark the first three pairs, which join rank j with rank $n-j+1$. Bottom: move posteriors $p_{\theta}(m \mid r)$ from the classifier in Section 5.1, stacked by move for the same candidates.	69
5.8.	Mean normalized score margins by pair index.	70

5.9.	DAMR (%) of the base, SFT, and DPO models on the eight dimensions of the pedagogical evaluation taxonomy, computed over the 298 test contexts of each model. <i>To some extent</i> counts as a non-match.	71
5.10.	Distribution of the judge labels per dimension and model (share of the 298 responses, rounded to whole percent; segments below 10% are not labeled). The first legend entry of each panel is the desired label, so its segment equals the DAMR of Figure 5.9.	72
5.11.	DAMR (%) within each student error type of the test split, for the five model variants and the eight dimensions. Rows give the number of test contexts per error type. Cell values are rounded to whole percent.	74
5.12.	Change in DAMR of the three DPO models relative to the SFT model, in percentage points, within each student error type and dimension. Positive values are improvements toward the desired label.	75
5.13.	Each cell is the difference between the average posterior of the chosen responses and that of the rejected responses for the tutor moves Focus, Probing, and Telling, computed on the training split of the preference dataset at $\rho \in \{0.1, 0.3, 0.5\}$ and grouped by student error type. Positive values (blue) mean that the chosen responses carry more of the move, negative values (red) mean that they carry less. Generic is omitted (absolute difference is at most 0.02). Rows are ordered by group: the three Focus-favored error types above the dashed line, the four Telling-favored error types below it. The number to the right of each panel is the number of training pairs of the error type; the training/validation split is given in Table 4.8.	77
5.14.	Change in DAMR of the three DPO models relative to the SFT model, in percentage points, with the error types grouped by the move their pairs prefer (Figure 5.13). Grey bar and dots: range and values of the four Telling-favored error types; markers: the three Focus-favored error types. The values are those of Figure 5.12.	78
A.1.	One dialogue context per student error type from the 693 training contexts of the aligned dataset. In each block, all candidates are ranked by the error-conditioned score $s(r, e)$ defined in Section 4.4.2. Upper panel: chosen candidates are colored in blue, rejected candidates in orange, and the omitted middle candidate in grey when n is odd. Lower panel: move posteriors $p_\theta(m r)$ from the selected classifier in Section 5.1, stacked by move. Rows are grouped by the student error type with the largest utility (PPMI). n denotes the number of candidates, and the utility vector lists $u(m, e)$ in the order $u_{\text{Focus}}, u_{\text{Generic}}, u_{\text{Probing}}, u_{\text{Telling}}$	92

List of Tables

2.1. Publicly accessible educational dialogue datasets. Not all publicly available educational dialogue datasets are shown. C = classroom, T = tutoring.	8
4.1. Teacher moves in MathDial	22
4.2. Distribution of student error types and tutor-move labels in the aligned dataset. Tutor-move counts include both tutor turns in each dialogue. Each dialogue has one student error type for the initial incorrect student solution.	27
4.3. Train-validation-test counts by tutor-move label. Tutor-turn only includes both labeled tutor utterances. Preceding-1 and Preceding-2 use only last-turn labels and therefore share the same stratified split.	31
4.4. Classifier training configurations. All classifiers are trained using the AdamW optimizer, with a warmup ratio of 0.10, weight decay of 0.01, and a random seed of 42. OvR AUC denotes macro one-vs-rest ROC-AUC.	38
4.5. First-turn co-occurrence of student error types and tutor-move labels in the training split of the aligned dataset. Each dialogue contributes one student error type and the tutor-move label of the first tutor utterance.	44
4.6. Positive pointwise mutual information (PPMI) between student error types and first-turn tutor moves in the training split of the aligned dataset. Values are rounded to three decimal places for display only.	45
4.7. Composition of the preference dataset by student error type. Dialogues are the 693 training-split contexts. Generated responses are unique tutor responses after duplicate removal. Percentages are within-column shares.	47
4.8. Student error-type counts by preference-pair selection ratio for the training, validation, and test sets. <i>All</i> is the selected subset before the stratified 80:20 training-validation split; <i>Val.</i> is the validation set. The test set is shared across selection ratios. The mean normalized score difference is the mean of $\Delta_{e,j} / \max_m \text{PPMI}(m, e)$ within each split, where m is a pedagogical tutor move and e is the student error type of pair j , using the PPMI values in Table 4.6.	55
4.9. Distribution of student error types in the test split of the aligned dataset. Percentages are rounded to one decimal place.	57
4.10. Pedagogical evaluation taxonomy	58

5.1. Tutor-move classification results for the 18 configurations on the held-out test sets. Macro-AUC denotes one-vs-rest ROC-AUC. The selected final classifier is highlighted in bold. Tutor-turn only representations consistently outperform both Preceding-1 and Preceding-2 representations. Values that deviate from the aggregate performance trend are italicized.	61
---	----

Bibliography

- Alic, S., Demszky, D., Mancenido, Z., Liu, J., Hill, H., & Jurafsky, D. (2022). Computationally identifying funneling and focusing questions in classroom discourse. *Proceedings of the 17th Workshop on Innovative Use of NLP for Building Educational Applications (BEA 2022)*, 224–233. <https://doi.org/10.18653/v1/2022.bea-1.27>
- Anderson, J. R., Corbett, A. T., Koedinger, K. R., & Pelletier, R. (1995). Cognitive tutors: Lessons learned. *Journal of the Learning Sciences*, 4(2), 167–207. https://doi.org/10.1207/s15327809jls0402_2
- Ashok Kumar, N., & Lan, A. (2024). Improving Socratic question generation using data augmentation and preference optimization. *Proceedings of the 19th Workshop on Innovative Use of NLP for Building Educational Applications (BEA 2024)*, 108–118. <https://aclanthology.org/2024.bea-1.10/>
- Austin, J. L. (1962). *How to do things with words*. Clarendon Press. <https://books.google.com/books?id=4soTvAEACAAJ>
- Bastani, H., Bastani, O., Sungu, A., Ge, H., Kabakçı, Ö., & Mariman, R. (2025). Generative AI without guardrails can harm learning: Evidence from high school mathematics. *Proceedings of the National Academy of Sciences*, 122(26), e2422633122. <https://doi.org/10.1073/pnas.2422633122>
- Bloom, B. S. (1984). The 2 sigma problem: The search for methods of group instruction as effective as one-to-one tutoring. *Educational Researcher*, 13(6), 4–16. <https://doi.org/10.3102/0013189X013006004>
- Boyer, K. E., Phillips, R., Ingram, A., Ha, E. Y., Wallis, M., Vouk, M., & Lester, J. (2011). Investigating the relationship between dialogue structure and tutoring effectiveness: A hidden Markov modeling approach. *International Journal of Artificial Intelligence in Education*, 21(1–2), 65–81. <https://doi.org/10.3233/JAI-2011-018>
- Bradley, R. A., & Terry, M. E. (1952). Rank analysis of incomplete block designs. <https://doi.org/10.1093/biomet/41.3-4.502>
- Briggs, R. A. (2014). Normative theories of rational choice: Expected utility.
- Brown, J. S., & Burton, R. R. (1978). Diagnostic models for procedural bugs in basic mathematical skills. *Cognitive Science*, 2(2), 155–192. https://doi.org/10.1207/s15516709cog0202_4
- Buckley, M., & Wolska, M. (2008). A classification of dialogue actions in tutorial dialogue. *Proceedings of the 22nd International Conference on Computational Linguistics (Coling 2008)*, 73–80.
- Bunt, H., Alexandersson, J., Choe, J.-W., Fang, A. C., Hasida, K., Petukhova, V., Popescu-Belis, A., & Traum, D. (2012). ISO 24617-2: A semantically-based standard for dialogue anno-

- tation. *Proceedings of the Eighth International Conference on Language Resources and Evaluation (LREC)*, 430–437. <https://aclanthology.org/L12-1296/>
- Caines, A., Yannakoudakis, H., Allen, H., Pérez-Paredes, P., Byrne, B., & Buttery, P. (2022). The teacher-student chatroom corpus version 2: More lessons, new annotation, automatic detection of sequence shifts. *Proceedings of the 11th Workshop on NLP for Computer Assisted Language Learning (NLP4CALL)*, 23–35. <https://aclanthology.org/2022.nlp4call-1.3/>
- Caines, A., Yannakoudakis, H., Edmondson, H., Allen, H., Pérez-Paredes, P., Byrne, B., & Buttery, P. (2020). The teacher-student chatroom corpus. *Proceedings of the 9th Workshop on Natural Language Processing for Computer Assisted Language Learning (NLP4CALL 2020)*, 10–19. <https://aclanthology.org/2020.nlp4call-1.2/>
- Cao, J., Suresh, A., Jacobs, J., Clevenger, C., Howard, A., Brown, C., Milne, B., Fischhaber, T., Sumner, T., & Martin, J. H. (2025). Enhancing talk moves analysis in mathematics tutoring through classroom teaching discourse. *Proceedings of the 31st International Conference on Computational Linguistics*, 7671–7684.
- Cao, J., Tanana, M., Imel, Z., Poitras, E., Atkins, D., & Srikumar, V. (2019). Observing dialogue in therapy: Categorizing and forecasting behavioral codes. *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 5599–5611.
- Chi, M. T. H., Siler, S. A., Jeong, H., Yamauchi, T., & Hausmann, R. G. M. (2001). Learning from human tutoring. *Cognitive Science*, 25(4), 471–533. https://doi.org/10.1207/s15516709cog2504_1
- Chi, M. T. H., & Wylie, R. (2014). The ICAP framework: Linking cognitive engagement to active learning outcomes. *Educational Psychologist*, 49(4), 219–243. <https://doi.org/10.1080/00461520.2014.965823>
- Chi, M., VanLehn, K., Litman, D., & Jordan, P. (2011). Empirically evaluating the application of reinforcement learning to the induction of effective and adaptive pedagogical strategies. *User Modeling and User-Adapted Interaction*, 21(1–2), 137–180. <https://doi.org/10.1007/s11257-010-9093-1>
- Church, K. W., & Hanks, P. (1990). Word association norms, mutual information, and lexicography. *Computational Linguistics*, 16(1), 22–29. <https://aclanthology.org/J90-1003/>
- Clavié, B., & Gal, K. (2019). Edubert: Pretrained deep language models for learning analytics. *arXiv preprint arXiv:1912.00690*.
- Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., et al. (2021). Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.
- Corbett, A. T., & Anderson, J. R. (1995). Knowledge tracing: Modeling the acquisition of procedural knowledge. *User Modeling and User-Adapted Interaction*, 4(4), 253–278. <https://doi.org/10.1007/BF01099821>
- Core, M. G., & Allen, J. F. (1997). Coding dialogs with the DAMSL annotation scheme. *Working Notes of the AAAI Fall Symposium on Communicative Action in Humans and Machines*. <https://www.cs.rochester.edu/research/cisd/pubs/>

- Daheim, N., Macina, J., Kapur, M., Gurevych, I., & Sachan, M. (2024). Stepwise verification and remediation of student reasoning errors with large language model tutors. *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 8386–8411.
- Demszky, D., & Hill, H. (2023). The NCTE transcripts: A dataset of elementary math classroom transcripts. *Proceedings of the 18th Workshop on Innovative Use of NLP for Building Educational Applications (BEA 2023)*, 528–538. <https://doi.org/10.18653/v1/2023.bea-1.44>
- Demszky, D., & Liu, J. (2023). M-Powering teachers: Natural language processing powered feedback improves 1:1 instruction and student outcomes. *Proceedings of the Tenth ACM Conference on Learning @ Scale*, 59–69. <https://doi.org/10.1145/3573051.3593379>
- Demszky, D., Liu, J., Mancenido, Z., Cohen, J., Hill, H., Jurafsky, D., & Hashimoto, T. (2021). Measuring conversational uptake: A case study on student-teacher interactions. *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, 1638–1653. <https://doi.org/10.18653/v1/2021.acl-long.130>
- Deng, X., Zhong, H., Ai, R., Feng, F., Wang, Z., & He, X. (2025). Less is more: Improving LLM alignment via preference data selection. *Advances in Neural Information Processing Systems*, 38, 161259–161285. https://proceedings.neurips.cc/paper_files/paper/2025/hash/ebf95a6f3c575322da15d4fd0fc2b3c8-Abstract-Conference.html
- Dinucu-Jianu, D., Macina, J., Daheim, N., Hakimi, I., Gurevych, I., & Sachan, M. (2025). From problem-solving to teaching problem-solving: Aligning LLMs with pedagogy using reinforcement learning. *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 272–292. <https://doi.org/10.18653/v1/2025.emnlp-main.15>
- D’Mello, S., & Graesser, A. (2012). AutoTutor and affective AutoTutor: Learning by talking with cognitively and emotionally intelligent computers that talk back. *ACM Transactions on Interactive Intelligent Systems*, 2(4), 1–39. <https://doi.org/10.1145/2395123.2395128>
- Forbes-Riley, K., & Litman, D. (2011). Benefits and challenges of real-time uncertainty detection and adaptation in a spoken dialogue computer tutor. *Speech Communication*, 53(9–10), 1115–1136. <https://doi.org/10.1016/j.specom.2011.02.006>
- Ganesh, A., Palmer, M., & von der Wense, K. (2021). What would a teacher do? predicting future talk moves. *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, 4739–4751.
- Gorodkin, J. (2004). Comparing two K-category assignments by a K-category correlation coefficient. *Computational Biology and Chemistry*, 28(5–6), 367–374.
- Graesser, A. C., Person, N. K., & Magliano, J. P. (1995). Collaborative dialogue patterns in naturalistic one-to-one tutoring. *Applied cognitive psychology*, 9(6), 495–522.
- Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Vaughan, A., et al. (2024). The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.
- Gururangan, S., Marasović, A., Swayamdipta, S., Lo, K., Beltagy, I., Downey, D., & Smith, N. A. (2020). Don’t stop pretraining: Adapt language models to domains and tasks.

- Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 8342–8360.
- Han, J., Yoo, H., Myung, J., Kim, M., Lee, T. Y., Ahn, S.-Y., & Oh, A. (2024). Recipe4u: Student-chatgpt interaction dataset in efl writing education. *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, 13666–13676.
- Hanley, J. A., & McNeil, B. J. (1982). The meaning and use of the area under a receiver operating characteristic (ROC) curve. *Radiology*, 143(1), 29–36. <https://doi.org/10.1148/radiology.143.1.7063747>
- Hattie, J., & Timperley, H. (2007). The power of feedback. *Review of Educational Research*, 77(1), 81–112. <https://doi.org/10.3102/003465430298487>
- He, L., & Xu, J. (2026). Automated classification of tutors' dialogue acts using generative ai. *Reflecting Digital Learning*, 1(1).
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2022). LoRA: Low-rank adaptation of large language models. *International Conference on Learning Representations*. <https://openreview.net/forum?id=nZeVKeeFYf9>
- Ikram, F., Scarlatos, A., & Lan, A. (2025). Exploring llms for predicting tutor strategy and student outcomes in dialogues. *Proceedings of the 20th Workshop on Innovative Use of NLP for Building Educational Applications (BEA 2025)*, 765–779.
- Ion, M. (2026). MathMentorDB: A massive dataset of authentic online mathematics tutoring dialogues [ACL Rolling Review, under review]. <https://openreview.net/forum?id=hw76uAXOgg>
- Jensen, E., Dale, M., Donnelly, P. J., Stone, C., Kelly, S., Godley, A., & D'Mello, S. K. (2020). Toward automated feedback on teacher discourse to enhance teacher learning. *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems*, 1–13. <https://doi.org/10.1145/3313831.3376418>
- Jurafsky, D., Shriberg, E., & Biasca, D. (1997). *Switchboard SWBD-DAMSL shallow-discourse-function annotation coders manual, draft 13* (tech. rep. No. 97-02). University of Colorado at Boulder, Institute of Cognitive Science. <https://web.stanford.edu/~jurafsky/ws97/manual.august1.html>
- Jurenka, I., Kunesch, M., McKee, K. R., Gillick, D., Zhu, S., Wiltberger, S., Phal, S. M., Hermann, K., Kasenberg, D., Bhoopchand, A., et al. (2024a). Towards responsible development of generative AI for education: An evaluation-driven approach. arXiv: 2407.12687. <https://arxiv.org/abs/2407.12687>
- Jurenka, I., Kunesch, M., McKee, K. R., Gillick, D., Zhu, S., Wiltberger, S., Phal, S. M., Hermann, K., Kasenberg, D., Bhoopchand, A., et al. (2024b). Towards responsible development of generative ai for education: An evaluation-driven approach. *arXiv preprint arXiv:2407.12687*.
- Jurman, G., Riccadonna, S., & Furlanello, C. (2012). A comparison of MCC and CEN error measures in multi-class prediction. *PLoS ONE*, 7(8), e41882. <https://doi.org/10.1371/journal.pone.0041882>

- Kakarla, S., Thomas, D. R., Lin, J., Gupta, S., & Koedinger, K. R. (2024). Using large language models to assess tutors' performance in reacting to students making math errors. *Proceedings of the AI for Education Workshop at AAAI 2024*, 257, 77–84. <https://proceedings.mlr.press/v257/kakarla24a.html>
- Kelly, S., Olney, A. M., Donnelly, P., Nystrand, M., & D'Mello, S. K. (2018). Automatically measuring question authenticity in real-world classrooms. *Educational Researcher*, 47(7), 451–464. <https://doi.org/10.3102/0013189X18785613>
- Kestin, G., Miller, K., Kiales, A., Milbourne, T., & Ponti, G. (2025). Ai tutoring outperforms in-class active learning: An rct introducing a novel research-based design in an authentic educational setting. *Scientific Reports*, 15(1), 17458.
- Khan Academy. (2023). Khanmigo [Accessed: 2026-09-21]. <https://www.khanmigo.ai/>
- Kim, T., & Vossen, P. (2021). Emoberta: Speaker-aware emotion recognition in conversation with roberta. *arXiv preprint arXiv:2108.12009*.
- Kochmar, E., Maurya, K. K., Petukhova, K., Srivatsa, K. A., Tack, A., & Vasselli, J. (2025). Findings of the BEA 2025 shared task on pedagogical ability assessment of AI-powered tutors. *Proceedings of the 20th Workshop on Innovative Use of NLP for Building Educational Applications (BEA 2025)*, 1011–1033. <https://doi.org/10.18653/v1/2025.bea-1.77>
- Kulik, J. A., & Fletcher, J. D. (2016). Effectiveness of intelligent tutoring systems: A meta-analytic review. *Review of Educational Research*, 86(1), 42–78. <https://doi.org/10.3102/0034654315581420>
- LearnLM Team. (2024). LearnLM: Improving Gemini for learning. *arXiv preprint arXiv:2412.16429*.
- LearnLM Team. (2025). Evaluating gemini in an arena for learning. *arXiv preprint arXiv:2505.24477*.
- Lin, J., Singh, S., Sha, L., Tan, W., Lang, D., Gašević, D., & Chen, G. (2022). Is it a good move? Mining effective tutoring strategies from human–human tutorial dialogues. *Future Generation Computer Systems*, 127, 194–207. <https://doi.org/10.1016/j.future.2021.09.001>
- Lin, J., Tan, W., Nguyen, N. D., Lang, D., Du, L., Buntine, W., Beare, R., Chen, G., & Gašević, D. (2023). Robust educational dialogue act classifiers with low-resource and imbalanced datasets. *International Conference on Artificial Intelligence in Education*, 114–125.
- Lin, T.-Y., Goyal, P., Girshick, R., He, K., & Dollár, P. (2017). Focal loss for dense object detection. *Proceedings of the IEEE International Conference on Computer Vision*, 2980–2988.
- Litman, D., & Silliman, S. (2004). Itspoke: An intelligent tutoring spoken dialogue system. *Demonstration papers at HLT-NAACL 2004*, 5–8.
- Litman, D. J., & Forbes-Riley, K. (2006). Correlations between dialogue acts and learning in spoken tutoring dialogues. *Natural Language Engineering*, 12(2), 161–176. <https://doi.org/10.1017/S1351324906004165>
- Liu, J., Huang, Z., Xiao, T., Sha, J., Wu, J., Liu, Q., Wang, S., & Chen, E. (2024). SocraticLM: Exploring socratic personalized teaching with large language models. *Advances in Neural Information Processing Systems 37 (NeurIPS)*, 37, 85693–85721. https://proceedings.neurips.cc/paper_files/paper/2024/hash/9bae399d1f34b8650351c1bd3692aeae-Abstract-Conference.html

- Liu, W., Zeng, W., He, K., Jiang, Y., & He, J. (2024). What makes good data for alignment? a comprehensive study of automatic data selection in instruction tuning. *International Conference on Learning Representations*, 2024, 22353–22373.
- Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., & Stoyanov, V. (2019). RoBERTa: A robustly optimized BERT pretraining approach. *arXiv preprint arXiv:1907.11692*.
- Lu, X., & Wang, X. (2024). Generative students: Using LLM-simulated student profiles to support question item evaluation. *Proceedings of the Eleventh ACM Conference on Learning @ Scale*, 16–27. <https://doi.org/10.1145/3657604.3662031>
- Ma, W., Adesope, O. O., Nesbit, J. C., & Liu, Q. (2014). Intelligent tutoring systems and learning outcomes: A meta-analysis. *Journal of Educational Psychology*, 106(4), 901–918. <https://doi.org/10.1037/a0037123>
- Macina, J., Daheim, N., Hakimi, I., Kapur, M., Gurevych, I., & Sachan, M. (2025). MathTutorBench: A benchmark for measuring open-ended pedagogical capabilities of LLM tutors. *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 204–221. <https://doi.org/10.18653/v1/2025.emnlp-main.11>
- Macina, J., Daheim, N., Pal Chowdhury, S., Sinha, T., Kapur, M., Gurevych, I., & Sachan, M. (2023). MathDial: A dialogue tutoring dataset with rich pedagogical properties grounded in math reasoning problems. *Findings of the Association for Computational Linguistics: EMNLP 2023*, 5602–5621. <https://doi.org/10.18653/v1/2023.findings-emnlp.372>
- Macina, J., Daheim, N., Wang, L., Sinha, T., Kapur, M., Gurevych, I., & Sachan, M. (2023). Opportunities and challenges in neural dialog tutoring. *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*, 2357–2372. <https://doi.org/10.18653/v1/2023.eacl-main.173>
- Manning, C. D., & Schütze, H. (1999). *Foundations of statistical natural language processing*. MIT Press.
- Marineau, J., Wiemer-Hastings, P., Harter, D., Olde, B., Chipman, P., Karnavat, A., Pomeroy, V., Rajan, S., Graesser, A., & Tutoring Research Group. (2000). Classification of speech acts in tutorial dialog. *Proceedings of the Workshop on Modeling Human Teaching Tactics and Strategies at ITS 2000*, 65–71. <http://users.sussex.ac.uk/~bend/its2000/marineau.pdf>
- Markel, J. M., Opferman, S. G., Landay, J. A., & Piech, C. (2023). GPTEach: Interactive TA training with GPT-based students. *Proceedings of the Tenth ACM Conference on Learning @ Scale*, 226–236. <https://doi.org/10.1145/3573051.3593393>
- Maurya, K. K., Srivatsa, K. A., Petukhova, K., & Kochmar, E. (2025a). Unifying AI tutor evaluation: An evaluation taxonomy for pedagogical ability assessment of LLM-powered AI tutors. *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, 1234–1251. <https://doi.org/10.18653/v1/2025.naacl-long.57>
- Maurya, K. K., Srivatsa, K. A., Petukhova, K., & Kochmar, E. (2025b). Unifying ai tutor evaluation: An evaluation taxonomy for pedagogical ability assessment of llm-powered

- ai tutors. *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, 1234–1251.
- Mehan, H. (1979). *Learning lessons: Social organization in the classroom*. Harvard University Press. <https://doi.org/10.4159/harvard.9780674420106>
- Metcalfe, J. (2017). Learning from errors. *Annual Review of Psychology*, 68, 465–489. <https://doi.org/10.1146/annurev-psych-010416-044022>
- Michaels, S., O'Connor, C., & Resnick, L. B. (2008). Deliberative discourse idealized and realized: Accountable talk in the classroom and in civic life. *Studies in Philosophy and Education*, 27(4), 283–297. <https://doi.org/10.1007/s11217-007-9071-1>
- Mitrovic, A., & Ohlsson, S. (1999). Evaluation of a constraint-based tutor for a database language. *International Journal of Artificial Intelligence in Education*, 10(3–4), 238–256. <https://hdl.handle.net/10092/327>
- Moreau-Pernet, B., Tian, Y., Sawaya, S., Foltz, P., Cao, J., Milne, B., & Christie, T. (2024). Classifying tutor discursive moves at scale in mathematics classrooms with large language models. *Proceedings of the eleventh ACM conference on Learning at Scale*, 361–365.
- Murray, T. (2003). An overview of intelligent tutoring system authoring tools: Updated analysis of the state of the art. *Authoring tools for advanced technology learning environments: Toward cost-effective adaptive, interactive and intelligent educational software*, 491–544.
- Nguyen, N. D., Tan, W., Du, L., Buntine, W., Beare, R., & Chen, C. (2023). Auc maximization for low-resource named entity recognition. *Proceedings of the AAAI Conference on Artificial Intelligence*, 37(11), 13389–13399.
- Nye, B. D., Graesser, A. C., & Hu, X. (2014). AutoTutor and family: A review of 17 years of natural language tutoring. *International Journal of Artificial Intelligence in Education*, 24(4), 427–469. <https://doi.org/10.1007/s40593-014-0029-5>
- Nystrand, M., Gamoran, A., Kachur, R., & Prendergast, C. (1997). *Opening dialogue: Understanding the dynamics of language and learning in the english classroom*. Teachers College Press. <https://eric.ed.gov/?id=ED404651>
- Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., et al. (2022). Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems 35 (NeurIPS)*, 27730–27744. https://proceedings.neurips.cc/paper_files/paper/2022/hash/b1efde53be364a73914f58805a001731-Abstract-Conference.html
- Pal Chowdhury, S., Zouhar, V., & Sachan, M. (2024). AutoTutor meets large language models: A language model tutor with rich pedagogy and guardrails. *Proceedings of the Eleventh ACM Conference on Learning @ Scale*, 5–15. <https://doi.org/10.1145/3657604.3662041>
- Pan, Y., Cai, Z., Zhong, H., Chen, G., & Wang, C. (2025). What matters in data for DPO? *Advances in Neural Information Processing Systems*, 38, 44689–44716. https://proceedings.neurips.cc/paper_files/paper/2025/hash/3f37b8fbd43303106dd141a602838ad5-Abstract-Conference.html

- Papineni, K., Roukos, S., Ward, T., & Zhu, W.-J. (2002). Bleu: A method for automatic evaluation of machine translation. *Proceedings of the 40th annual meeting of the Association for Computational Linguistics*, 311–318.
- Park, M., Kim, S., Lee, S., Kwon, S., & Kim, K. (2024). Empowering personalized learning through a conversation-based tutoring system with student modeling. *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*, 1–10. <https://doi.org/10.1145/3613905.3651122>
- Pattnaik, P., Maheshwary, R., Ogueji, K., Yadav, V., & Madhusudhan, S. T. (2024). Enhancing alignment using curriculum learning & ranked preferences. *Findings of the Association for Computational Linguistics: EMNLP 2024*, 12891–12907.
- Pavlik, P., Brawner, K., Olney, A., & Mitrovic, A. (2013). A review of student models used in intelligent tutoring systems. *Design recommendations for intelligent tutoring systems*, 1, 39–68.
- Petukhova, K., & Kochmar, E. (2025). Intent matters: Enhancing AI tutoring with fine-grained pedagogical intent annotation. *Proceedings of the 20th Workshop on Innovative Use of NLP for Building Educational Applications (BEA 2025)*, 860–872. <https://doi.org/10.18653/v1/2025.bea-1.63>
- Petukhova, K., Nguyen, T. D., & Kochmar, E. (2026). Towards pedagogically aligned LLM tutors for math mistake remediation. In E. Kochmar, B. Alhafni, S. Bannò, M. Bexte, J. Burstein, A. Horbach, R. Laarmann-Quante, A. Tack, V. Yaneva, & Z. Yuan (Eds.), *Proceedings of the 21st workshop on innovative use of NLP for building educational applications (BEA 2026)* (pp. 118–140). Association for Computational Linguistics. <https://doi.org/10.18653/v1/2026.bea-1.10>
- Piech, C., Bassen, J., Huang, J., Ganguli, S., Sahami, M., Guibas, L. J., & Sohl-Dickstein, J. (2015). Deep knowledge tracing. *Advances in Neural Information Processing Systems 28 (NeurIPS)*, 505–513. <https://proceedings.neurips.cc/paper/2015/hash/bac9162b47c56fc8a4d2a51980Abstract.html>
- Radatz, H. (1979). Error analysis in mathematics education. *Journal for Research in Mathematics Education*, 10(3), 163–172. <https://doi.org/10.5951/jresmetheduc.10.3.0163>
- Rafailov, R., Sharma, A., Mitchell, E., Manning, C. D., Ermon, S., & Finn, C. (2023). Direct preference optimization: Your language model is secretly a reward model. *Advances in Neural Information Processing Systems*, 36, 53728–53741. https://proceedings.neurips.cc/paper_files/paper/2023/hash/a85b405ed65c6477a4fe8302b5e06ce7-Abstract.html
- Reiser, B. J. (2018). Scaffolding complex learning: The mechanisms of structuring and problematizing student work. In *Scaffolding* (pp. 273–304). Psychology Press.
- Röder, M., Both, A., & Hinneburg, A. (2015). Exploring the space of topic coherence measures. *Proceedings of the Eighth ACM International Conference on Web Search and Data Mining*, 399–408.
- Samei, B., Li, H., Keshtkar, F., Rus, V., & Graesser, A. C. (2014). Context-based speech act classification in intelligent tutoring systems. *Intelligent Tutoring Systems (ITS 2014)*, 236–241. https://doi.org/10.1007/978-3-319-07221-0_28

- Scarlatos, A., Baker, R. S., & Lan, A. (2025). Exploring knowledge tracing in tutor-student dialogues using LLMs. *Proceedings of the 15th International Learning Analytics and Knowledge Conference (LAK 2025)*, 249–259. <https://doi.org/10.1145/3706468.3706501>
- Scarlatos, A., Liu, N., Lee, J., Baraniuk, R., & Lan, A. (2025). Training llm-based tutors to improve student learning outcomes in dialogues. *International Conference on Artificial Intelligence in Education*, 251–266.
- Scarlatos, A., Smith, D., Woodhead, S., & Lan, A. (2024). Improving the validity of automatically generated feedback via reinforcement learning. *Artificial Intelligence in Education (AIED 2024)*, 14829, 280–294. https://doi.org/10.1007/978-3-031-64302-6_20
- Schmucker, R., Xia, M., Azaria, A., & Mitchell, T. (2024). Ruffle&Riley: Insights from designing and evaluating a large language model-based conversational tutoring system. *Artificial Intelligence in Education (AIED 2024)*, 14829, 75–90. https://doi.org/10.1007/978-3-031-64302-6_6
- Searle, J. R. (1969). *Speech acts: An essay in the philosophy of language*. Cambridge University Press. <https://doi.org/10.1017/CBO9781139173438>
- Shen, J. T., Yamashita, M., Prihar, E., Heffernan, N., Wu, X., Graff, B., & Lee, D. (2021). Mathbert: A pre-trained language model for general nlp tasks in mathematics education. *arXiv preprint arXiv:2106.07340*.
- Sinclair, J. M., & Coulthard, M. (1975). *Towards an analysis of discourse: The english used by teachers and pupils*. Oxford University Press. https://books.google.com/books/about/Towards_an_Analysis_of_Discourse.html?id=zYucAAAAMAAJ
- Skinner, B. F. (1954). The science of learning and the art of teaching. *Harvard Educational Review*, 24(2), 86–97. https://books.google.com/books/about/The_Science_of_Learning_and_the_Art_of_T.html?id=55BLAAAAYAAJ
- Snell, C., Klein, D., & Zhong, R. (2022). Learning by distilling context. *arXiv preprint arXiv:2209.15189*.
- Sonkar, S., Liu, N., Mallick, D. B., & Baraniuk, R. G. (2023). CLASS: A design framework for building intelligent tutoring systems based on learning science principles. *Findings of the Association for Computational Linguistics: EMNLP 2023*, 1941–1961. <https://doi.org/10.18653/v1/2023.findings-emnlp.130>
- Sonkar, S., Ni, K., Chaudhary, S., & Baraniuk, R. (2024). Pedagogical alignment of large language models. *Findings of the Association for Computational Linguistics: EMNLP 2024*, 13641–13650.
- Srinivasa, R. S., Che, Z., Zhang, C. B. C., Mares, D., Hernandez, E., Park, J., Lee, D., Mangialardi, G., Ng, C., Cardona, E.-Y. H., et al. (2025). Tutorbench: A benchmark to assess tutoring capabilities of large language models. *arXiv preprint arXiv:2510.02663*.
- Stasaski, K., Kao, K., & Hearst, M. A. (2020). CIMA: A large open access dialogue dataset for tutoring. *Proceedings of the Fifteenth Workshop on Innovative Use of NLP for Building Educational Applications*, 52–64. <https://doi.org/10.18653/v1/2020.bea-1.5>
- Steenbergen-Hu, S., & Cooper, H. (2013). A meta-analysis of the effectiveness of intelligent tutoring systems on K–12 students’ mathematical learning. *Journal of Educational Psychology*, 105(4), 970–987. <https://doi.org/10.1037/a0032447>

- Stolcke, A., Ries, K., Coccaro, N., Shriberg, E., Bates, R., Jurafsky, D., Taylor, P., Martin, R., Van Ess-Dykema, C., & Meteer, M. (2000). Dialogue act modeling for automatic tagging and recognition of conversational speech. *Computational Linguistics*, 26(3), 339–373. <https://doi.org/10.1162/089120100561737>
- Suresh, A., Jacobs, J., Harty, C., Perkoff, M., Martin, J. H., & Sumner, T. (2022). The TalkMoves dataset: K-12 mathematics lesson transcripts annotated for teacher and student discursive moves. *Proceedings of the Thirteenth Language Resources and Evaluation Conference*, 4654–4662. <https://aclanthology.org/2022.lrec-1.497/>
- Suresh, A., Jacobs, J., Perkoff, M., Martin, J. H., & Sumner, T. (2022). Fine-tuning transformers with additional context to classify discursive moves in mathematics classrooms. *Proceedings of the 17th Workshop on Innovative Use of NLP for Building Educational Applications (BEA 2022)*, 71–81.
- Suresh, A., Sumner, T., Jacobs, J., Foland, B., & Ward, W. (2019). Automating analysis and feedback to improve mathematics teachers’ classroom discourse. *Proceedings of the AAAI Conference on Artificial Intelligence*, 33, 9721–9728. <https://doi.org/10.1609/aaai.v33i01.33019721>
- Sweller, J. (1988). Cognitive load during problem solving: Effects on learning. *Cognitive Science*, 12(2), 257–285. https://doi.org/10.1207/s15516709cog1202_4
- Tack, A., Kochmar, E., Yuan, Z., Bibauw, S., & Piech, C. (2023). The BEA 2023 shared task on generating AI teacher responses in educational dialogues. *Proceedings of the 18th Workshop on Innovative Use of NLP for Building Educational Applications (BEA 2023)*, 785–795. <https://doi.org/10.18653/v1/2023.bea-1.64>
- Tack, A., & Piech, C. (2022). The AI teacher test: Measuring the pedagogical ability of Blender and GPT-3 in educational dialogues. *Proceedings of the 15th International Conference on Educational Data Mining (EDM 2022)*. <https://doi.org/10.5281/zenodo.6853187>
- VanLehn, K. (1990). *Mind bugs: The origins of procedural misconceptions*. MIT Press. https://books.google.com/books/about/Mind_Bugs.html?id=owY9seMJ4RwC
- VanLehn, K. (2006). The behavior of tutoring systems. *International Journal of Artificial Intelligence in Education*, 16(3), 227–265. [https://doi.org/10.3233/IRG-2006-16\(3\)02](https://doi.org/10.3233/IRG-2006-16(3)02)
- VanLehn, K. (2011). The relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems. *Educational Psychologist*, 46(4), 197–221. <https://doi.org/10.1080/00461520.2011.611369>
- VanLehn, K., Jordan, P. W., Rosé, C. P., Bhembé, D., Böttner, M., Gaydos, A., Makatchev, M., Pappuswamy, U., Ringenberg, M., Roque, A., Siler, S., & Srivastava, R. (2002). The architecture of Why2-Atlas: A coach for qualitative physics essay writing. *Intelligent Tutoring Systems: 6th International Conference, ITS 2002*, 2363, 158–167. https://doi.org/10.1007/3-540-47987-2_20
- Vygotsky, L. S. (1978). *Mind in society: The development of higher psychological processes*. Harvard University Press. <https://doi.org/10.2307/j.ctvjf9vz4>
- Wang, R., Zhang, Q., Robinson, C., Loeb, S., & Demszky, D. (2024). Bridging the novice-expert gap via models of decision-making: A case study on remediating math mistakes. *Pro-*

- ceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), 2174–2199.
- Wang, R. E., Ribeiro, A. T., Robinson, C. D., Loeb, S., & Demszky, D. (2024). Tutor CoPilot: A human-AI approach for scaling real-time expertise [arXiv preprint].
- Wood, D., Bruner, J. S., & Ross, G. (1976). The role of tutoring in problem solving. *Journal of child psychology and psychiatry*, 17(2), 89–100.
- Woolf, B. P. (2009). *Building intelligent interactive tutors: Student-centered strategies for revolutionizing e-learning*. Morgan Kaufmann. <https://doi.org/10.1016/B978-0-12-373594-2.X0001-9>
- Xie, Q., Wang, Z., Feng, Y., & Xia, R. (2024). Ask again, then fail: Large language models' vacillations in judgment. *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 10709–10745.
- Yuan, Z., Yan, Y., Sonka, M., & Yang, T. (2021). Large-scale robust deep AUC maximization: A new surrogate loss and empirical studies on medical image classification. *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 3040–3049.
- Zent, M., Smith, D., & Woodhead, S. (2025). Piivot: A lightweight nlp anonymization framework for question-anchored tutoring dialogues. *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 27467–27476.
- Zhang, T., Kishore, V., Wu, F., Weinberger, K. Q., & Artzi, Y. (2019). Bertscore: Evaluating text generation with bert. *arXiv preprint arXiv:1904.09675*.
- Zhou, C., Liu, P., Xu, P., Iyer, S., Sun, J., Mao, Y., Ma, X., Efrat, A., Yu, P., Yu, L., Zhang, S., Ghosh, G., Lewis, M., Zettlemoyer, L., & Levy, O. (2023). LIMA: Less is more for alignment. *Advances in Neural Information Processing Systems*, 36, 55006–55021.